



OWASP
Smart Contract Security

scs.owasp.org ↗

Security Handbook : Infrastructure Security

07

Security Handbook Series

GRANT SUPPORT • 2025-2026



**ecosystem
support
program**

Supported by Ethereum Foundation's
Ecosystem Support Program (ESP)



Contents

Frontispiece	4
About This Handbook	4
Copyright and License	5
Author and Creative Credits	5
Purpose and Scope	6
Target Audience	6
How to Use This Handbook	6
Relationship to SCSVS, SCSTG, and SCWE	7
Part I: Foundations	8
1. Introduction to Infrastructure Security in Web3	8
2. Asset Inventory	12
Part II: Node and Compute Infrastructure	17
3. Node Infrastructure Security	17
4. Cloud Infrastructure	25
5. Operating System Security	34
Part III: Network and Availability	38
6. Network Security	38
7. DDoS Protection	42
8. Zero-Trust Principles	48
Part IV: DNS and Domain Security	60
9. DNS and Domain Registration	60
10. DNS Security (DNSSEC, CAA, etc.)	63
11. Email and Communication Security (SPF, DKIM, DMARC)	66
12. DNS and Hosting Attack Playbooks (Overview)	68
Part V: Identity and Access Management	71
13. Identity and Access Management for Infrastructure	71
14. Integration with IAM Framework	83
Part VI: Checklists and Operations	86
15. Node Infrastructure Security Checklist (Consolidated)	87
16. Infrastructure as Code (IaC) Security	90
17. Monitoring and Logging	93

Part VII: Appendices	99
18. Appendix A: Glossary	100
19. Appendix B: Node Infrastructure Security Checklist (Printable)	106
20. Appendix C: DNS and Domain Security Checklist	110
21. Appendix D: Tool and Provider References	116
22. Appendix E: References and Further Reading	118
Appendix C: References and Further Reading	121
Standards and Frameworks	121
Incidents and Case Studies	122
Tools and Documentation	124
Further Reading	125

Frontispiece

OWASP SMART CONTRACT SECURITY PROJECT

Infrastructure Security

About This Handbook

Infrastructure Security is part of the OWASP Smart Contract Security Handbook Series. The series extends smart contract security beyond contract code into the people, process, application, infrastructure, and operational layers surrounding Web3 systems.

Each handbook is designed as a defensive field reference for architects, engineers, security practitioners, incident responders, auditors, and organizational leaders. It complements the OWASP Smart Contract Security Verification Standard (SCSVS), Smart Contract Security Testing Guide (SCSTG), Smart Contract Weakness Enumeration (SCWE), Smart Contract Top 10, and SCS Checklist without replacing those resources.

SERIES EDITION **2026**

Copyright and License

Copyright © 2026 The OWASP Foundation and contributors.



This document is released under the [Creative Commons Attribution-ShareAlike 4.0 International License](#). You may share and adapt the material with appropriate attribution and under the same license. For reuse or distribution, make the license terms clear and identify material changes.

OWASP publications are community-developed educational resources. References to organizations, products, or services do not constitute endorsement, and the material is provided without warranty.

Author and Creative Credits

PROJECT LEAD

Shashank | CredShields

CEO and Co-Founder

AUTHOR | PROJECT MAINTAINER

Pratik Lagaskar

Security Researcher

COVER PAGE DESIGNS

Siddharth Neekher

Lead UI Designer @ CredShields

OWASP Smart Contract Security Project

Purpose and Scope

This handbook secures the infrastructure layer underneath a Web3 project: the servers, cloud accounts, nodes, and network paths that keep contracts reachable and front ends online. It covers asset inventory, cloud and node infrastructure (including remote procedure call (RPC) and archive nodes), distributed denial-of-service (DDoS) protection, DNS and domain security, network hardening, operating system baselines, and zero-trust design, aligned with current operational practice rather than a theoretical ideal.

The guidance draws on NIST's Zero Trust Architecture (SP 800-207 and SP 1800-35), cloud security posture checklists built around least-privilege identity and access management (IAM), policy-as-code enforcement, Zero Trust Network Access (ZTNA) and Secure Access Service Edge (SASE) concepts, and continuous monitoring practice. Each chapter translates these frameworks into checklists a node operator, DevOps engineer, or site reliability engineer (SRE) can act on directly, rather than leaving them as abstract compliance language.

Target Audience

Engineers and developers building on Web3 infrastructure, security specialists reviewing that infrastructure, and DevOps, cloud, and SRE practitioners who operate it day to day. Anyone accountable for keeping an RPC node, a validator, or a project's cloud footprint online and defensible will find direct guidance here.

How to Use This Handbook

Start with Part I to establish why infrastructure sits underneath every other Web3 security control and to build an asset inventory before anything else. Parts II through V are reference material organized by domain: node and compute infrastructure, network and availability, DNS and domain security, and identity and access management. Part VI consolidates the checklists into operational form for pre-deployment, ongoing operations, and incident readiness. Part VII holds the glossary, printable checklists, tool references, and bibliography. An SRE hardening a fresh RPC deployment can jump straight to Part II and the checklists in Part VI; a team preparing for a domain-security review should start with Part IV.

Relationship to SCSVS, SCSTG, and SCWE

The Smart Contract Security Verification Standard (SCSVS) defines the properties a secure system must satisfy at the application and infrastructure boundary; the controls in this handbook, cloud IAM, node hardening, and DNS integrity among them, are the operational implementation of those infrastructure-facing requirements. The Smart Contract Security Testing Guide (SCSTG) describes how to test a system's security; the checklists in Parts II, III, and VI extend that testing discipline to cloud posture, network configuration, and DNS records rather than contract code. The Smart Contract Weakness Enumeration (SCWE) catalogs weaknesses in contract logic, and the failure modes covered here (exposed RPC endpoints, missing DNSSEC, stale IAM grants) sit one layer below the contract and fall largely outside SCWE's scope. This handbook overlaps most closely with the DNS and Hosting Security Handbook (02), which goes deeper on domain and hosting-provider risk, and feeds directly into the Incident Response Handbook (06), which owns the response process once an infrastructure control fails.

Part I: Foundations

This part argues a single point: the smart contract is rarely where a Web3 project's infrastructure actually fails. It fixes the vocabulary for the rest of the handbook, works through the recurring single-provider-versus-multi-provider decision, and builds the asset inventory that every later control, node hardening, network segmentation, DNS integrity, identity and access management, depends on to know what it is protecting.

1. Introduction to Infrastructure Security in Web3

This chapter explains why infrastructure sits underneath every other Web3 security control and orients you to the rest of the handbook.

1.1 Why Infrastructure Is Critical (Centralized Front-Ends and Services)

A smart contract can be immutable, formally verified, and audited by three independent firms, and a project can still lose everything through the layer surrounding it. That layer, the cloud account, the domain name system (DNS) records, the content delivery network (CDN), the remote procedure call (RPC) endpoints wallets talk to, is almost always centralized, mutable, and administered by a small number of people with normal, phishable credentials. It re-creates the entire Web2 attack surface underneath a system marketed as trustless.

The incident record backs this up. In December 2021, an attacker compromised a Cloudflare application programming interface (API) key belonging to BadgerDAO and used it to inject a malicious script into the project's front end for roughly ten hours, ultimately draining about \$120 million from users who had granted the resulting spend approvals ([Halborn, BadgerDAO hack explained](#); [CoinDesk](#)). In August 2022, Curve Finance's `curve.fi` domain was DNS-hijacked; attackers pointed the nameservers at a cloned site and collected roughly \$570,000 in malicious approvals before the team could redirect users to the unaffected `curve.exchange` domain, which ran on a separate DNS provider ([The Block](#); [Decrypt](#)). In July 2024, Compound Finance and Celer

Network were both hit by DNS hijacks targeting domains registered through Squarespace, part of a wave that put over 220 DeFi front ends at risk of the same technique ([CoinDesk](#); [BleepingComputer](#)). None of these incidents touched a line of Solidity.

Availability failures tell the same story from the other direction. When Infura, the RPC provider behind most MetaMask installs, suffered an outage on November 11, 2020 caused by outdated client software and an unreviewed code change, MetaMask, Uniswap, and Compound degraded simultaneously, and Binance and Coinbase halted Ethereum withdrawals until service was restored ([Decrypt](#); [CoinMarketCap Academy](#)). Five years later, on October 20, 2025, a domain name system and database fault inside a single Amazon Web Services (AWS) region, `us-east-1`, took Coinbase's trading platform and its Base layer-2 network offline alongside Robinhood, and disrupted Infura's Ethereum, Polygon, Optimism, Arbitrum, Linea, Base, and Scroll endpoints at once ([CCN](#); [CoinDesk](#)). A French regulator's tracking of Ethereum node hosting found the share running on AWS alone climbing from roughly 43% in mid-2022 to over 49% by late 2023, meaning a single cloud region's health has become a proxy for the health of the base chain most Web3 infrastructure sits on ([CryptoSlate](#)). Infrastructure is not a secondary concern behind contract security. For the users who lost funds in every incident above, it was the only layer that mattered.

1.2 Design Choices: Single Provider vs. Multi-Provider

Nearly every infrastructure decision in this handbook eventually reduces to one question: concentrate on a single vendor for a given function (one cloud account, one CDN, one RPC provider, one DNS registrar) or spread that function across two or more. Neither answer is universally correct. The decision trades attack surface against blast radius, and it should be made deliberately, per function, with the tradeoff written down rather than defaulted into by whichever provider a founding engineer happened to sign up with first.

Dimension	Single provider (consolidated)	Multi-provider (diversified)
Attack surface	Smaller: one account, one set of credentials, one configuration surface to secure and monitor	Larger: N accounts, N credential sets, N configurations that can drift out of sync
Blast radius of a compromise	Full: one stolen API key or session can take down or redirect the entire service, as in the BadgerDAO incident	Partial: a compromised provider affects only the traffic routed to it, if failover is configured and tested
Blast radius of an outage	Full: a provider-wide incident takes the whole service down with it	Partial: healthy providers absorb the shifted load
Operational load	Lower: one dashboard, one on-call runbook, one billing and support relationship	Higher: more credentials to rotate, more configurations to keep consistent, more places integrity pins must be updated
Cost	Lower steady-state cost, higher tail-risk cost when the single provider fails	Higher steady-state cost (duplicated infrastructure, cross-provider data transfer)

Dimension	Single provider (consolidated)	Multi-provider (diversified)
Best fit	Early-stage teams, internal tooling, non-critical services	Front ends and RPC paths that sit directly between users and funds

Use the sensitivity tiering built in Chapter 2 to decide where diversification earns its cost: a critical-tier asset (Section 2.2) that is the sole path to signing a transaction justifies a second provider; an internal staging dashboard usually does not.

1.2.1 Consolidation (Lower Surface) vs. Diversification (Resilience)

The Fastly outage of June 8, 2021 is the clearest illustration of concentrated-provider risk in the industry's recent memory. A latent software defect, triggered by one customer's valid configuration change, crashed edge nodes across Fastly's global network and dropped roughly 85% of that CDN's traffic within minutes, taking Amazon, Reddit, Twitch, PayPal, and the UK government's website offline for about an hour ([Fastly post-incident summary](#); [ThousandEyes analysis](#)). None of the affected sites had a code defect of their own. They had a dependency on a single provider whose defect became their defect. The AWS `us-east-1` outage of October 20, 2025 repeated the pattern a region at a time rather than a CDN edge at a time, and it repeated again nine days later on October 29 ([CoinDesk](#)).

Consolidation is the right default when the asset in question is not on the critical path to user funds: it lowers the number of things that must be independently monitored, patched, and defended, and every credential you do not issue is a credential that cannot be phished or leaked. Diversification earns its added complexity when the asset is on that critical path, when the org's risk tolerance for a multi-hour outage is low, or when a single incident (compromise or outage) would be existential rather than merely embarrassing. The failure mode to avoid is diversification in name only: two CDN accounts that share the same DNS provider, or two cloud regions with the same root IAM credentials, diversify nothing, because the shared dependency is still a single point of failure. Test failover before you need it. A secondary provider that has never actually served production traffic is a hypothesis, not a control.

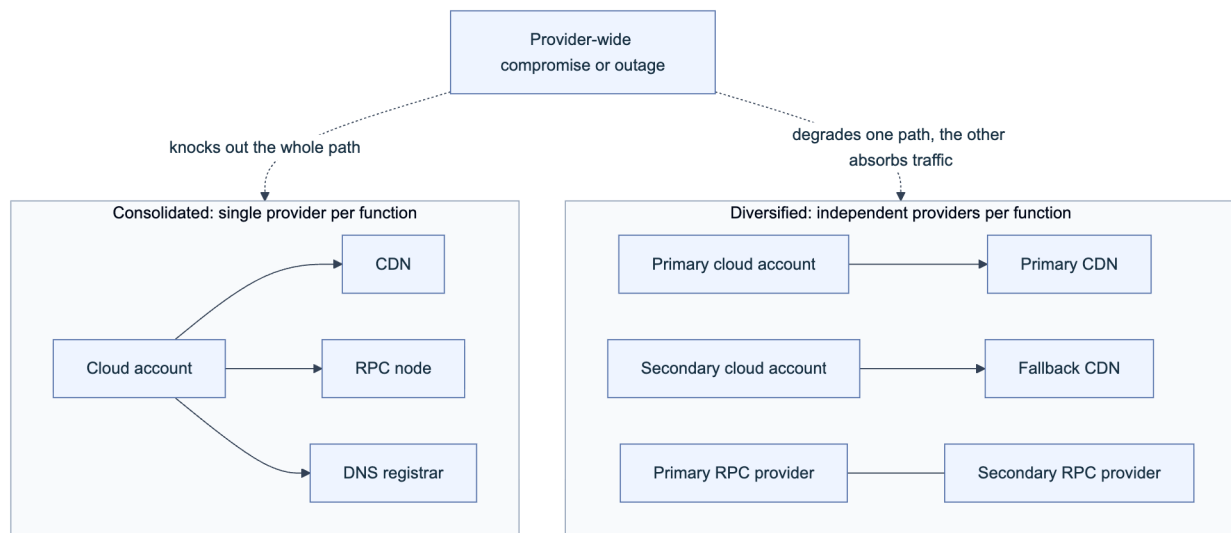


Figure 1. The same failure event against a consolidated architecture (left) versus a diversified one (right). Diversification only reduces blast radius if the redundant providers do not share an underlying dependency.

1.3 Relationship to SCSVS, SCSTG, and SCWE

The OWASP Smart Contract Security (SCS) Project’s three companion resources, the [Smart Contract Security Verification Standard \(SCSVS\)](#), the [Smart Contract Security Testing Guide \(SCSTG\)](#), and the [Smart Contract Weakness Enumeration \(SCWE\)](#), define, test, and catalog security properties of contract code and its immediate on-chain environment. This handbook covers the layer those resources assume is already trustworthy: the cloud accounts, nodes, networks, and DNS records that keep the contract reachable in the first place. The relationship is complementary rather than overlapping. SCSVS asks whether a system’s verifiable security requirements are met; the IAM, node-hardening, and DNS-integrity controls in this handbook are the infrastructure-facing requirements that sit alongside SCSVS’s contract-facing ones, so that “the system is secure” does not quietly narrow to “the bytecode is secure” while the DNS registrar account has no multi-factor authentication (MFA). SCSTG describes testing methodology; the checklists in Parts II, III, and VI of this handbook extend that discipline to cloud configuration posture, network segmentation, and DNS record hygiene, none of which SCSTG’s contract-focused test cases reach. SCWE enumerates weaknesses in contract logic; the failure modes this handbook addresses (an exposed RPC admin port, a missing DNS Certification Authority Authorization (CAA) record, a stale cloud IAM grant) sit one layer below SCWE’s scope and are not catalogued there. This handbook overlaps most closely with the DNS and Hosting Security Handbook (02), which goes deeper on domain- and hosting-provider-specific risk, and it feeds directly into the Incident Response Handbook (06), which owns the response process once one of these infrastructure controls fails.

1.4 How to Use This Handbook

Read this part once, in order, to fix the four ideas that recur throughout the rest of the handbook: infrastructure is an attack surface equal in weight to the contract, the single-versus-multi-provider decision is made per function against a documented risk tolerance, and nothing downstream can

be secured, monitored, or recovered without the asset inventory built in Chapter 2. After that, treat the handbook as reference material organized by domain rather than as a narrative to read cover to cover. Part II covers node and compute infrastructure, including RPC and archive node hardening and cloud account structure. Part III covers network security, distributed denial-of-service (DDoS) protection, and zero-trust design. Part IV is DNS and domain security in depth. Part V covers identity and access management for infrastructure specifically, cross-referencing the organization's broader IAM program. Part VI consolidates every checklist in the handbook into operational form for pre-deployment gates, ongoing operations, and incident readiness, and Part VII holds the glossary, printable checklists, and full bibliography. An engineer hardening a fresh RPC deployment should jump straight to Part II and the checklist in Part VI; a team preparing a DNS registrar lockdown should start at Part IV. Wherever a topic touches a sibling handbook, DNS specifics (02), the response process (06), SDK-side node interaction (08), this part and the ones that follow link to it directly rather than duplicating its content.

2. Asset Inventory

Every control in the rest of this handbook, hardening a node, segmenting a network, rotating a secret, presupposes that you know the asset exists. This chapter builds that precondition.

2.1 Identifying All Infrastructure Components

You cannot defend what you have not enumerated, and you cannot enumerate what you have never systematically looked for. The Center for Internet Security's Critical Security Controls open with exactly this principle: Control 1, Inventory and Control of Enterprise Assets, requires an organization to actively track every enterprise asset, physical, virtual, cloud-hosted, or remote, precisely because unmanaged and unauthorized assets are where monitoring, patching, and incident response first break down ([CIS Control 1](#)). The National Institute of Standards and Technology (NIST) Cybersecurity Framework 2.0 codifies the same requirement as the Asset Management (ID.AM) function, with subcategories for hardware inventories (ID.AM-01), software and services (ID.AM-02), network communication flows (ID.AM-03), and supplier-provided services (ID.AM-04) ([NIST CSF 2.0](#), [ID.AM](#)).

For a Web3 project, that enumeration spans categories a generic checklist will not surface on its own, because “infrastructure” here includes the blockchain-facing layer alongside the conventional one.

Category	Examples	Typical owner
Compute	Virtual machines, containers, serverless functions, bare-metal validator hosts	Platform / DevOps

Category	Examples	Typical owner
Network	Virtual private clouds (VPCs), subnets, load balancers, firewalls, VPN gateways	Network / site reliability engineering (SRE)
Blockchain node infrastructure	RPC endpoints, archive nodes, validator and staking nodes, light clients	Protocol / infrastructure engineering
Storage	Object storage buckets, databases, backup snapshots, block explorer indexes	Platform / DevOps
DNS and domains	Registrar accounts, zone files, subdomains, TXT and CAA records	Security / IT
CDN and edge	CDN accounts, edge configuration, web application firewall (WAF) rules	Front-end / Platform
Secrets and keys	Key management service (KMS) keys, hardware-backed signing keys, API tokens	Security
Continuous integration / continuous deployment (CI/CD)	Source repositories, pipelines, self-hosted runners, package registries	Engineering / DevOps
Identity	Single sign-on (SSO) provider, service accounts, break-glass accounts	Security / IT
Monitoring	Log aggregators, alerting channels, dashboards, on-chain monitors	SRE / Security
Third-party software as a service	Analytics, support tooling, status pages, transactional email	IT / Operations

Build the first version of this list from every angle at once, cloud provider billing exports, DNS zone transfers, CI/CD runner registrations, employee offboarding records, rather than from a single source, because no single source is complete. A validator node spun up manually for a testnet migration and never torn down will not appear in a Terraform state file; a legacy subdomain pointed at a decommissioned service will not appear in a cloud billing export. Treat the first pass as a floor, not a finished inventory.

2.2 Ownership, Purpose, and Sensitivity

An inventory entry that lists an asset but not who is accountable for it is only slightly more useful than no entry at all. Every asset needs three fields beyond its identifier: an owner (a named individual or team, not a distribution list that has gone stale), a documented purpose (why the asset exists and what depends on it), and a sensitivity classification that determines the baseline controls it must carry. NIST CSF 2.0's ID.AM-05 makes this explicit: assets are prioritized based on classification, criticality, and their impact on the mission, not treated uniformly ([NIST CSF 2.0, ID.AM](#)).

Tier	Definition	Example assets	Baseline controls
Critical	Compromise or loss directly enables theft of user or protocol funds	Signing keys, deployer wallets, front-end origin, RPC nodes that broadcast transactions	Hardware-backed MFA, break-glass process, dedicated on-call, quarterly access review
Sensitive	Compromise enables escalation toward a critical asset or exposes user data	Cloud root and IAM accounts, CI/CD secrets, DNS registrar account, the inventory system itself	MFA, least privilege, logged access, mandatory change review
Internal	Needed for operations, limited direct exploit value on its own	Internal dashboards, staging environments, ticketing systems	Standard access control, routine patch cadence
Public	Intended for public consumption	Documentation sites, marketing pages, public status pages	Baseline integrity monitoring

Unowned assets are where this classification most often collapses in practice. A subdomain created for a marketing campaign and left pointing at a decommissioned cloud resource is a dangling DNS record: an attacker can claim the abandoned resource and serve content under your domain, a subdomain takeover technique covered in depth in the DNS and Hosting Security Handbook (02). The reason it happens is rarely malice; it is an asset whose owner left the company, whose purpose was never written down, and whose sensitivity was never assessed, so no one noticed it still existed. Every inventory review should explicitly hunt for entries with a blank owner field, because a blank owner field is itself a finding.

2.3 Keeping Inventory Current (IaC, CMDB, Spreadsheets)

An inventory that is accurate on the day it is built and stale six months later provides false confidence, which is worse than no inventory at all, because it tells an incident responder to trust data that no longer reflects reality. Three approaches dominate in practice, and they are not mutually exclusive.

A spreadsheet is the honest starting point for a small team: it costs nothing, requires no tooling investment, and is immediately understandable. It also degrades the fastest, because nothing enforces that a change to infrastructure produces a corresponding edit to the spreadsheet. A configuration management database (CMDB), the Information Technology Infrastructure Library (ITIL) term for a system of record that stores assets as configuration items along with their relationships, formalizes what a spreadsheet leaves informal: dependency graphs between assets, structured change history, and integration points for other tooling ([Red Hat, what is a CMDB](#)). A CMDB is worth the operational overhead once the asset count or the team size makes a spreadsheet unreliable, typically well before either feels large by headcount alone.

Infrastructure as code (IaC) offers a third path that many Web3 infrastructure teams underuse: if every cloud resource is provisioned through Terraform, Pulumi, or an equivalent tool, the IaC state file and its enforced tagging convention become a living inventory that cannot drift from reality without also failing a `plan` or `apply`.

```
# Mandatory tags, enforced by policy-as-code in CI (see Part II, Section 4.7.2),
# so every resource created through Terraform self-registers into the inventory.
resource "aws_instance" "rpc_node_archive_01" {
  # ... instance configuration ...
  tags = {
    Owner       = "infra-team"
    Purpose     = "ethereum-archive-rpc"
    Sensitivity  = "critical"
    Environment = "production"
    ManagedBy   = "terraform"
    InventoryId = "AST-0142"
  }
}
```

Resources created outside this workflow, through a console click during an incident, for instance, are exactly the assets a manual inventory misses, so pair IaC with active and passive discovery: CIS Control safeguards 1.3 and 1.5 call for an active discovery tool that scans for connected assets and a passive tool that identifies assets from observed network traffic, catching what no provisioning workflow declared ([CIS Control 1](#)). Open-source options fit both roles well: [Cartography](#), originally built at Lyft, pulls assets and their relationships from AWS, Google Cloud Platform (GCP), Azure, Kubernetes, GitHub, and dozens of other platforms into a queryable graph database, and [Steampipe](#) exposes live cloud provider APIs as SQL tables so you can query current state directly rather than trust a snapshot. Cloud-native equivalents, AWS Config's resource aggregator, GCP's Cloud Asset Inventory, and Azure Resource Graph, provide the same continuous, provider-verified view without third-party tooling. Whichever combination you choose, run a reconciliation on a fixed schedule: diff the declared inventory against what active discovery actually finds, and treat every undeclared asset as an incident candidate until its owner and purpose are established.

2.4 Integration with Incident Response and Change Management

An inventory that exists only as a reference document, consulted occasionally and updated rarely, fails at the exact moment it matters most. Its value is proportional to how tightly it is wired into the two processes that need it in real time.

During an incident, the first questions a responder asks are which asset is affected, who owns it, what depends on it, and what its sensitivity tier implies about urgency. If those answers require paging three people and searching a wiki, the inventory has already failed its purpose; the [Incident Response Handbook \(06\)](#) assumes the inventory built here can answer them in seconds, not minutes, because minutes are the difference between containing a BadgerDAO-style front-end compromise and watching it run for ten hours. During change management, every modification

to an asset, a new IAM policy, a DNS record update, a cloud region added, should reference the asset's inventory identifier in the change ticket, so that the inventory updates as a side effect of the normal deployment workflow rather than through a separate, easily skipped step. Configuration drift, where the live state of an asset no longer matches what change management or IaC believes it to be, is itself a signal worth alerting on: it means either an unauthorized change occurred or the inventory's record of authorized changes is incomplete, and either case warrants investigation.

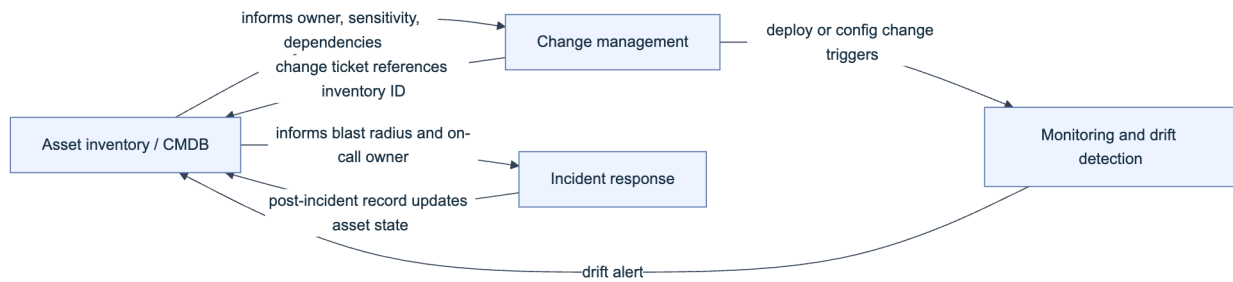


Figure 2. The asset inventory sits at the center of a closed loop with change management and incident response. A change that bypasses this loop, or an incident that cannot query it, is the failure mode Section 2.3's reconciliation step exists to catch.

Key controls for Part 1

- Treat infrastructure (cloud accounts, DNS, CDN, RPC endpoints) as an attack surface equal in weight to the smart contract; the BadgerDAO, Curve, Compound, and Infura incidents all bypassed audited contract code entirely.
- Make the single-provider-versus-multi-provider decision per function, explicitly, against the sensitivity tier of what that function protects, not as an unexamined default.
- Test failover paths before an outage forces you to; an untested secondary provider is a hypothesis, not a resilience control.
- Build the asset inventory from multiple sources (cloud billing, DNS zones, CI/CD registrations, IaC state) and flag every asset with a blank owner field as a finding.
- Wire the inventory into change management (every change references an asset ID) and incident response (every responder can query owner, sensitivity, and dependencies in seconds), and reconcile declared state against active discovery on a fixed schedule.

Part II: Node and Compute Infrastructure

Part 1 mapped what infrastructure a Web3 project owns. This Part hardens the pieces that actually run: the nodes that read and write chain state, the cloud accounts hosting everything else, and the operating systems underneath both. Each chapter follows the same pattern, access first, then patching and configuration, then the monitoring that catches what slips through.

3. Node Infrastructure Security

A node is the one component in this handbook that talks directly to the chain, and a compromised node can serve forged data, leak the operator's funds, or vanish and take an entire dApp's read path with it. This chapter covers the node itself, RPC and archive endpoints and validator infrastructure, plus the boundary keeping development traffic away from production chain state.

3.1 RPC and Archive Nodes

A full node keeps recent chain state and validates new blocks. An archive node additionally retains every historical state root, which lets it answer queries like “what was this address's balance at block 12,000,000” that a pruned full node cannot. That extra retention is expensive: as of 2025, an Erigon archive node needs on the order of 1.8 to 4 TB (terabyte) depending on configuration, while a Geth (go-ethereum) archive node using the newer path-based state scheme needs roughly 1.9 TB, down from the 12 TB or more that hash-based storage required ([7Block-Labs storage comparison](#)). Both node types expose a JSON-RPC interface, and that interface is the attack surface this section is about: it is how wallets submit transactions, how indexers pull events, and, unprotected, how an attacker enumerates internal state or exhausts the node's resources.

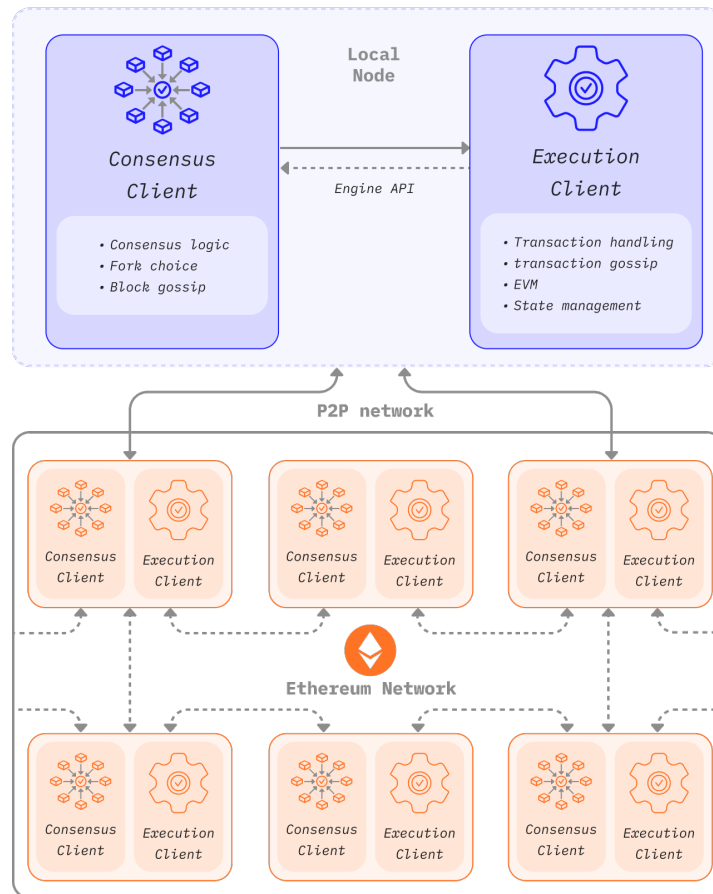


Figure. A post-Merge Ethereum node is a two-client system: the execution and consensus clients have distinct peer networks and communicate through an authenticated local API. This separation creates multiple patch, credential, network, and telemetry boundaries that operators must secure. Source: [ethereum.org, Node architecture](https://ethereum.org/en/node-architecture/), licensed under CC BY 4.0.

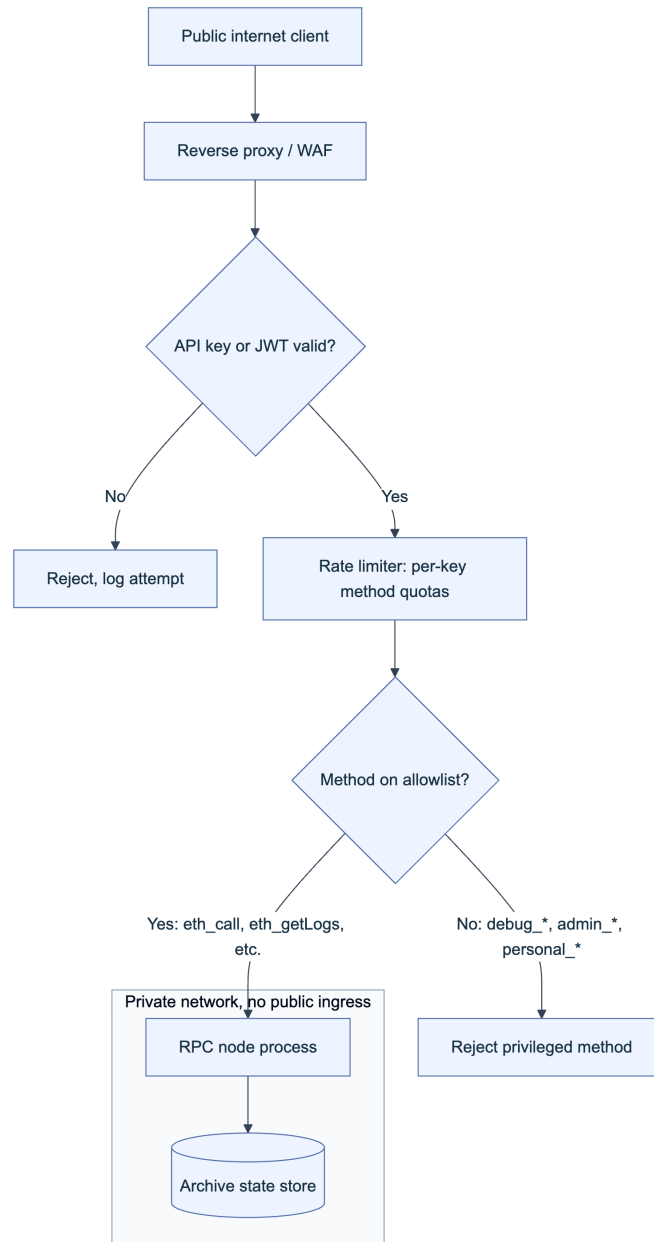


Figure 1. Layered exposure for an RPC/archive node. Nothing reaches the node process directly; every request passes through authentication, rate limiting, and a method allowlist first.

3.1.1 Access Control and Network Exposure

Never bind a node's HTTP or WebSocket RPC interface to `0.0.0.0` and expose it to the public internet without authentication. Geth's own security guidance is direct: administrative and debugging namespaces are not built to withstand hostile traffic, and exposing them routinely leads to drained accounts ([Geth security docs](#)). Restrict exposure in layers: bind RPC listeners to a private interface behind a reverse proxy or dedicated gateway, gate access with an API key or signed JSON Web Token (JWT) rather than a bare open port, and allowlist methods explicitly. A public gateway should permit `eth_call`, `eth_getBlockByNumber`, and `eth_getLogs`, and block the `debug_*`, `admin_*`, `personal_*`, and `miner_*` namespaces outright. Providers such as Chainstack

and QuickNode implement exactly this pattern as request-level access rules with per-key rate limits, worth replicating on self-hosted infrastructure ([Chainstack RPC access rules](#), [QuickNode endpoint security](#)).

```
# Geth: bind RPC to loopback only, let the reverse proxy handle public traffic
geth --http --http.addr 127.0.0.1 --http.port 8545 \
    --http.api eth,net,web3 \
    --http.vhosts "rpc.internal.example" \
    --authrpc.jwtsecret /etc/geth/jwt.hex

# nginx in front of it: reject any request missing a valid API key first
# if ($http_x_api_key = "") { return 403; }
# limit_req zone=rpc_per_key burst=20 nodelay;
```

Exposure tier	Who reaches it	Methods allowed
Public gateway	Any internet client with an API key	Read-only: <code>eth_call</code> , <code>eth_getLogs</code> , <code>eth_getBlockByNumber</code>
Internal service mesh	First-party backend services only	Read plus <code>eth_sendRawTransaction</code>
Loopback / private VLAN	The node operator's own tooling	<code>debug_*</code> , <code>admin_*</code> , <code>txpool_*</code> , Engine API

3.1.2 Patching and Hardening

Node clients are complex, actively developed software with the same vulnerability lifecycle as any network daemon. CVE-2023-42319 is representative: Geth through version 1.13.4, run with `--http --graphql`, let a remote attacker hang the daemon and exhaust memory via a crafted GraphQL query ([CVE-2023-42319 advisory](#)). The fix was a version bump, but only for operators tracking releases. Subscribe to the client's security advisory feed and treat a security release as a same-day or next-day patch, not a routine maintenance item. Since the Merge, execution and consensus clients authenticate to each other over the Engine API using a shared JWT secret rather than an open localhost port; rotate that secret if it is ever exposed in a log or container image layer. Beyond version currency, harden configuration: disable GraphQL and WebSocket unless a consumer needs them, cap `--maxpeers` to bound resource use, and run the client under a dedicated, unprivileged system account, never root.

Severity (client advisory)	Target patch window	Example
Critical (remote code execution, consensus fork risk)	Same day, out of band	Client-specific supermajority bugs
High (remote DoS, resource exhaustion)	Within 24 to 48 hours	CVE-2023-42319 (GraphQL memory exhaustion)
Medium/Low (local-only, requires existing access)	Next scheduled maintenance window	Minor dependency CVEs with no network path

3.1.3 Monitoring and Availability

A silently stalled or forked node is worse than one that is visibly down, because it keeps answering with stale or wrong data. On 11 November 2020, an unrelated Geth patch caused nodes still on an older release, including those behind Infura, to diverge onto a minority chain; exchanges reading exclusively from Infura briefly showed inconsistent balances and paused Ethereum withdrawals before the affected nodes updated ([The Block](#), [Infura outage coverage](#)). The lesson generalizes: a single RPC source, however reliable, is a single point of both availability and correctness failure. Monitor not just “is the port open” but “is this node’s view of the chain correct,” by comparing block hash and height against an independent provider and alerting on divergence. Export standard metrics (Prometheus is the common choice for Geth, Erigon, and most consensus clients) and alert on the thresholds below.

Metric	Healthy range	Alert condition
Block height lag vs. reference node	0 to 2 blocks	Sustained lag beyond 3 blocks
Peer count	Client-recommended minimum (often 25+)	Sustained drop below minimum
Disk free (archive volume)	Above 15% free	Below 10% free
RPC p95 latency	Under 500ms for read calls	Sustained breach for 5+ minutes
Chain ID / genesis hash	Matches expected value at every check	Any mismatch (possible fork or misconfiguration)

3.2 Validator and Staking Infrastructure (Where Applicable)

Where a project runs validator or staking infrastructure, for its own protocol, a liquid staking product, or a proof-of-stake network it depends on, the stakes shift from availability to slashing: signing two conflicting messages for the same slot destroys stake irreversibly, with no equivalent to a chargeback. Client diversity matters more here than almost anywhere else in this handbook. In May 2023, a bug in how the Prysm and Teku consensus clients handled old-target attestations caused Ethereum mainnet to briefly lose finality twice within 24 hours; the network recovered only because minority clients such as Lighthouse handled the same input correctly and kept producing valid blocks ([Offchain Labs postmortem](#), [ethereum.org client diversity](#)). Running the majority client because it is the default carries systemic consequences, not a neutral one.

Key custody is the other axis. A signing key on the same machine that faces the public P2P network is doubly exposed: to remote compromise, and to accidental double-signing if the same keystore is ever loaded on two machines during a migration. The industry answer is to separate the signer from the network-facing node. Consensus Web3Signer does this by holding keys in a dedicated signing service and maintaining a slashing-protection database (typically PostgreSQL) that records every block and attestation a key has signed, so a duplicate signing request is rejected before it reaches the network ([Web3Signer slashing protection](#)). Distributed Validator Technology (DVT) goes further, splitting a validator key across independent operator nodes with

threshold signing, so no single machine holds a complete key and a minority of compromised or offline operators cannot produce a slashable action alone. Obol's Charon middleware and the SSV Network are the two production DVT implementations for Ethereum staking today ([Figment, DVT and infrastructure resilience](#)).

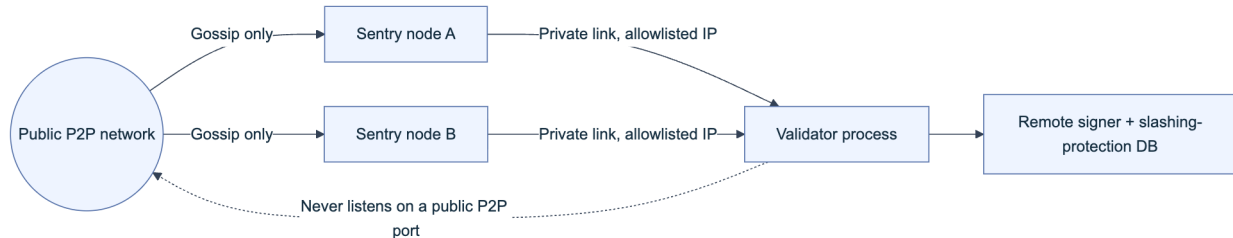


Figure 2. Sentry-node architecture. Public traffic terminates at expendable sentry nodes; the validator process itself, and the key it signs with, never faces the internet directly.

Custody model	Where the key lives	Slashing risk if a node is compromised	Operational cost
Local keystore on validator client	Same host as the validator process	High: full key exposed on one machine	Low
Remote signer (Web3Signer + HSM)	Dedicated signing service, hardware-backed	Medium: signer isolated, but still a single point	Medium
Distributed Validator Technology (DVT)	Split across N independent operators, threshold signed	Low: no single machine holds a complete key	Higher (coordination, latency)

3.3 Separation of Environments (Dev, Staging, Prod)

Development, staging, and production infrastructure protect different things and deserve different trust levels; collapsing them into one account or one network is how a staging bug reaches mainnet funds. Production holds mainnet RPC credentials, live signing keys, and real user data. Staging should run against a testnet or a forked mainnet snapshot, never a production RPC endpoint with production credentials, and development should not hold any secret capable of moving real value. Enforce the separation structurally, not by convention: separate cloud accounts or projects per environment (Section 4.2 covers the mechanics), separate IAM (Identity and Access Management) roles that cannot be assumed across the boundary, and a CI/CD pipeline requiring an explicit, logged promotion step to move an artifact from staging to production rather than a shared deploy target. The 2025 exposure of an unauthenticated MongoDB instance holding wallet data at the NCX exchange is a reminder of what happens when any environment, staging included, is reachable without authentication: the attacker does not care which label the database carried ([Cybernews, NCX exposure](#)).

Dimension	Development	Staging	Production
Chain endpoint	Local devnet or public testnet	Testnet or mainnet fork, isolated RPC key	Production mainnet RPC, dedicated key

Dimension	Development	Staging	Production
Signing keys	None, or disposable throwaway keys	Testnet-only keys, zero real value	HSM-backed or remote-signed production keys
Cloud account/project	dev account, broad developer access	staging account, restricted to release engineers	prod account, break-glass access only
Data	Synthetic or anonymized	Sanitized copy, no real PII/keys	Live user and protocol data
Network reachability	Developer VPN only	Internal CI runners only	Locked-down ingress, WAF-fronted

3.4 Node Infrastructure Security Checklist

- ☐ No RPC/WebSocket interface public without authentication and a method allowlist.
 - ☐ `debug_*`, `admin_*`, `personal_*`, `miner_*` namespaces disabled or loopback-only.
 - ☐ Client version tracked against the vendor advisory feed; same-day patching for Critical advisories.
 - ☐ Engine API JWT secret rotated on any suspected exposure.
 - ☐ Two or more independent RPC sources cross-checked for block height and hash agreement.
 - ☐ Prometheus (or equivalent) metrics and alerting on peer count, sync lag, and disk headroom.
 - ☐ Validator client diversity assessed; no single client running a supermajority of validators.
 - ☐ Signing keys held in a remote signer or HSM with slashing protection, not a bare local keystore.
 - ☐ DVT (Obol, SSV, or equivalent) evaluated for validators above an internal stake threshold.
 - ☐ Dev, staging, and production run in separate cloud accounts with non-assumable IAM boundaries.
 - ☐ Staging never holds production RPC credentials, signing keys, or unsanitized data.
-

4. Cloud Infrastructure

Most Web3 teams run on someone else's cloud, so the security model is a negotiation between what the provider guarantees and what the customer configures. This chapter covers that negotiation: provider selection, account structure, compute and storage, secrets, and enforcing configuration discipline with policy-as-code rather than manual review.

4.1 Cloud Provider Selection and Shared Responsibility

Every major cloud provider divides security duties along the same line: the provider secures the cloud (data centers, hypervisor, managed-service internals), and the customer secures what runs in the cloud (guest operating systems, IAM policy, network configuration, application logic, data). Amazon Web Services (AWS) states this as “security of the cloud” versus “security in the cloud” ([AWS shared responsibility model](#)), and the line shifts with the service model: infrastructure-as-a-service (IaaS, raw virtual machines) puts the most weight on the customer, while platform-as-a-service (PaaS) and software-as-a-service (SaaS) shift progressively more to the provider. The 2019 Capital One breach is the canonical case study of misreading that line. An attacker exploited a server-side request forgery (SSRF) vulnerability in a misconfigured web application firewall on an EC2 instance, used it to query the instance metadata service, and retrieved temporary IAM credentials granted far broader S3 access than the firewall needed; the resulting exfiltration exposed 106 million customer records ([AppSecco technical walkthrough](#)). AWS's infrastructure was never at fault; the customer-side IAM scope and the vulnerable application were. Four months later AWS shipped Instance Metadata Service v2 (IMDSv2), requiring a session token obtained via an explicit `PUT` request, which blocks the simple GET-based SSRF pattern that made the original attack trivial ([AWS Security Blog, IMDS defense in depth](#)).

Cloud Computing Shared Security Responsibility Model

Traditional On-Premise	Infrastructure as a Service	Platform as a Service	Software as a Service
Applications	Applications	Applications	Applications
Data	Data	Data	Data
Runtime	Runtime	Runtime	Runtime
Middleware	MiddleWare	Middleware	Middleware
Operating System	Operating System	Operating System	Operating System
Virtualization	Virtualization	Virtualization	Virtualization
Servers	Servers	Servers	Servers
Storage	Storage	Storage	Storage
Networking	Networking	Networking	Networking

The legend below shows the cloud shared security responsibility for both the CSPs and cloud customers, depending on the adopted cloud computing service model.

Cloud Computing Service Model Legend:

Managed by the Cloud Customer
Managed by Cloud Service Provider

Figure. The cloud shared responsibility model: the provider secures the layers of the cloud, the customer secures what they configure and run in it. The Capital One breach fell on the customer side of this line. Source: [National Security Agency Cybersecurity](#), via [Wikimedia Commons](#), CC BY-SA 4.0.

Service model	Provider secures	Customer secures
IaaS (raw VMs, block storage)	Physical hardware, hypervisor, network fabric	Guest OS patching, firewall rules, IAM policy, data
PaaS (managed database, managed Kubernetes control plane)	Above, plus the managed service's internals	Application config, access policy, data, workload identity
SaaS (managed RPC provider, managed CI/CD)	Above, plus the application itself	Account credentials, API keys, data shared with the service

4.2 Account and Project Structure

A single cloud account holding development, staging, production, and shared services together means one leaked credential or misconfigured policy can reach all of them. Structure accounts (AWS Organizations, Google Cloud projects under a resource hierarchy, or Azure subscriptions under management groups) so each environment, and ideally each major service, sits in its own account with its own billing boundary and IAM root. A workable baseline separates a management/billing account (no workloads, only governance), a security/log-archive account (centralized, write-once audit logs workload accounts cannot delete), a shared-services account (CI/CD runners, artifact registries), and separate development, staging, and production accounts per

Section 3.3. Cross-account access should flow through explicit IAM role assumption with a defined trust policy, never account-wide credentials shared across boundaries. This bounds blast radius directly: a compromised development account cannot reach production, because no standing credential or network path lets it.

4.3 Compute (VMs, Containers, Serverless)

Each compute model carries a different failure mode. Virtual machines fail through unpatched images and long-lived SSH access; containers fail through escape vulnerabilities and over-permissive runtime configuration; serverless functions fail through a platform layer the team does not operate at all. The February 2018 Tesla incident illustrates the container case: researchers found a Kubernetes administration console exposed to the internet without a password, letting attackers deploy cryptomining pods and, from inside a running pod, retrieve AWS credentials that reached a sensitive S3 bucket ([Electrek coverage](#)).

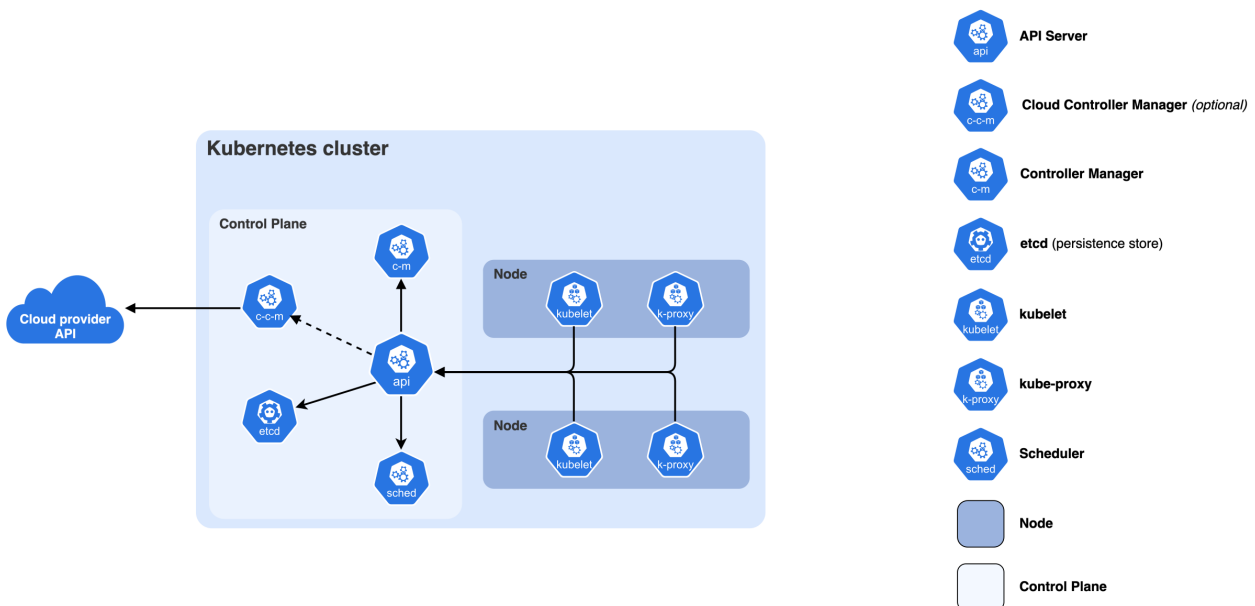


Figure. The current Kubernetes component boundary. Treat the API server as the authentication, authorization, and admission choke point; etcd as the authoritative cluster-state store; and each kubelet and container runtime as a privileged enforcement surface. The Tesla incident exposed access into this control plane, not merely an application container. Source: [Kubernetes documentation, Components](#), CC BY 4.0.

Container runtime bugs keep the escape vector alive at a lower level too: CVE-2024-21626, nicknamed “Leaky Vessels,” was a file-descriptor leak in `runc` (the runtime underneath Docker and Kubernetes) that let a malicious container image escape to the host filesystem ([Palo Alto Networks Unit 42](#)). The serverless case differs in kind: there is no host to patch, but the team inherits every failure of the platform’s control plane. In April 2026, hosting provider Vercel disclosed that an employee’s credentials were compromised through a third-party AI tool infected with infostealer malware, giving attackers access to internal systems holding customer environment variables, including API keys, database credentials, and deployment tokens for hundreds of customers; Web3 teams that stored wallet, payment processor, or chain infrastructure credentials

in those variables were disproportionately exposed, because the secrets connected directly to fund movement ([Vercel incident bulletin](#), [TechCrunch coverage](#)). No serverless configuration would have prevented that breach; the mitigation is architectural, keeping fund-moving secrets out of plaintext platform environment variables in favor of a secrets manager referenced at runtime (Section 4.5).

Compute type	Primary risk	Primary control
Virtual machines	Stale, unpatched images; standing SSH access	Golden images rebuilt on a fixed cadence, short-lived SSH via a bastion or SSM
Containers	Runtime/escape CVEs (e.g. CVE-2024-21626); over-privileged pods	Non-root containers, read-only filesystems, current <code>runc</code> / container runtime, image scanning
Serverless / PaaS functions	Platform-side compromise; secrets in plaintext env vars	Secrets manager references instead of literal env var values, least-privilege execution role

4.4 Storage and Backup Security

Object storage and database misconfiguration remain among the most common root causes of data exposure in this handbook’s incident set, because the failure is a missing default, not a sophisticated attack. In December 2025, the NCX exchange left a MongoDB instance reachable without authentication, exposing roughly five million records including wallet addresses and two-factor authentication codes; no exploit was required, only a network scan ([Cybernews](#)). Treat “no public access by default” as a platform-enforced invariant, not a per-bucket setting: AWS S3’s account-level Block Public Access, applied at the account or organization level, prevents any bucket in scope from becoming public even if an individual policy is later misconfigured. Encrypt data at rest (Section 4.5), enable versioning so an overwritten or deleted object is recoverable, and treat backups as their own security boundary: an attacker who deletes production data and its backups in the same blast radius has achieved a ransomware outcome without ransomware. Follow the 3-2-1 pattern, three copies on two storage types with one air-gapped or in a separate account the primary credentials cannot reach, and test restoration on a schedule rather than trusting a backup that has never been restored.

```
# AWS: enforce account-wide Block Public Access so no bucket, present or future, can be
# made public
aws s3control put-public-access-block \
  --account-id 123456789012 \
  --public-access-block-configuration \
    BlockPublic-
      Acls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=t
      rue
```

4.5 Secrets and Key Management in Cloud

A secret in a plaintext environment variable, a CI/CD log line, or a hardcoded constant is a secret with an unbounded number of places it can leak from, and the Vercel breach in Section 4.3 illustrates it directly: the exposed data was environment variables not marked sensitive, stored without encryption at rest. The corrective pattern is a dedicated secrets manager or key management service (KMS) the application references at runtime rather than a baked-in value. AWS Key Management Service, Google Cloud KMS, and Azure Key Vault each provide envelope encryption for data keys and centralized audit logging of every access, turning “who touched this credential” from a forensic guess into a queryable log. Classify secrets by blast radius before deciding where they live: a mainnet signing key or a withdrawal-capable exchange API key belongs in a hardware-backed KMS or HSM with mandatory approval for retrieval, while a low-sensitivity third-party key can live in a standard secrets manager entry.

4.5.1 Rotation and Automated Workflows (e.g., Vault, Cloud Secrets)

A secret stored securely but never rotated is still a long-lived credential, and long-lived credentials are exactly what an attacker who gains one-time access wants to find. Two patterns close that gap. Scheduled rotation, native to AWS Secrets Manager and Azure Key Vault, rotates a static credential on a timer via a provider-managed function, with overlapping old/new versions so in-flight connections do not break mid-rotation. Dynamic secrets, HashiCorp Vault’s model, go further by never persisting a long-lived credential at all: an application requests a database credential, Vault generates one on demand scoped to a short time-to-live (TTL), and it expires automatically with no rotation job needed ([HashiCorp, rotated vs. dynamic secrets, HashiCorp Cloud Platform auto-rotation](#)). For chain-facing credentials, prefer dynamic secrets wherever the downstream system supports them, and fall back to aggressive scheduled rotation (weeks, not years) everywhere else.

```
# HashiCorp Vault: AWS secrets engine role that issues short-lived, scoped credentials on demand
resource "vault_aws_secret_backend_role" "node_operator" {
  backend      = "aws"
  name         = "node-operator-readonly"
  credential_type = "assumed_role"
  role_arns    = ["arn:aws:iam::123456789012:role/node-metrics-reader"]
  default_ttl  = 900    # 15 minutes
  max_ttl      = 3600   # 1 hour hard cap
}
```

4.6 Identity and Access: Least Privilege, MFA, Hardware-Backed

Identity and Access Management (IAM) policy is the control that failed at the center of the Capital One breach: the compromised role could list and read from far more S3 buckets than the firewall process attached to it ever needed. Write every IAM policy to the minimum action and resource scope a workload requires, deny by default, and review unused permissions on a schedule rather than letting them accumulate as roles get copied and extended. Layer multi-factor

authentication (MFA) on top of scoped policy, favoring phishing-resistant methods over one-time codes. NIST Special Publication 800-63B ties this to its Authenticator Assurance Levels: AAL3, the highest level, requires a cryptographic authenticator with a non-exportable private key and explicit phishing resistance ([NIST SP 800-63B](#)). FIDO2/WebAuthn hardware security keys meet that bar because the credential is cryptographically bound to the registering origin, so a look-alike phishing site cannot complete authentication even with a stolen password ([Yubico, phishing-resistant MFA](#)). Require hardware-backed MFA for every human account with production access, without exception for accounts that touch signing keys, treasury wallets, or IAM policy itself.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["cloudwatch:GetMetricData", "cloudwatch:ListMetrics"],
    "Resource": "*",
    "Condition": {
      "Bool": { "aws:MultiFactorAuthPresent": "true" },
      "NumericLessThan": { "aws:MultiFactorAuthAge": "3600" }
    }
  }]
}
```

4.7 Configuration and Posture Management

Manual configuration review does not scale past a handful of resources, and it catches nothing between review cycles. The controls in this section replace manual review with code-defined baselines, automated policy checks in the deployment pipeline, and continuous scanning of what is actually running against what was approved.

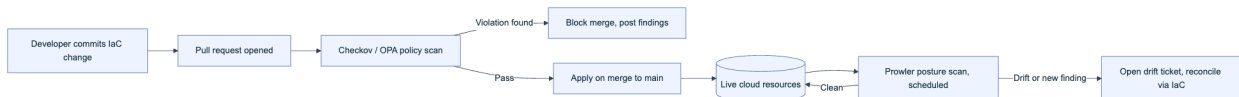


Figure 3. Policy-as-code gates a change twice: once before merge, against the proposed configuration, and again on a schedule, against what is actually deployed.

4.7.1 Infrastructure as Code and Approved Baselines

Infrastructure as Code (IaC), most commonly Terraform in this space, turns infrastructure configuration into a reviewable, versioned artifact instead of a set of clicks in a console that leave no diff. Build approved baseline modules for each resource type a project provisions repeatedly (a hardened VM, a locked-down S3 bucket, a properly scoped IAM role), publish them to an internal module registry, and require teams to compose from those modules rather than writing raw provider resources for anything security-sensitive. Pair the module with a golden image or golden container base built from a CIS-hardened baseline (Section 5.1) so that every VM or container inherits the baseline hardening automatically rather than depending on a runbook step someone might skip.

```
# Approved baseline module call, not a raw aws_instance resource
module "node_operator_vm" {
  source      = "git::https://git.internal.example/modules/hardened-vm.git?ref=v3.2.0"
  instance_type = "m6i.2xlarge"
  cis_profile   = "level1"
  allow_ssh_cidrs = ["10.20.0.0/24"] # bastion subnet only
}
```

4.7.2 Policy-as-Code (OPA, Checkov, Prowler) in CI

Policy-as-code encodes security and compliance rules as executable checks that run in the CI/CD pipeline, rejecting a non-compliant change before merge rather than flagging it after deployment. Open Policy Agent (OPA) is a general-purpose policy engine using the Rego language, applicable to IaC, Kubernetes admission control, and API authorization alike. Checkov is a static analyzer purpose-built for IaC (Terraform, CloudFormation, Kubernetes manifests) with a large built-in check library mapped to CIS, NIST, and other frameworks, usable without writing custom policy. Prowler complements both by scanning already-deployed accounts (AWS, Azure, GCP, Kubernetes) against the CIS Foundations Benchmarks and frameworks including NIST, PCI-DSS, and SOC2, validating what Checkov approved actually got deployed correctly ([Open Policy Agent](#), [Checkov](#), [Prowler](#)). Run Checkov (or OPA with a Terraform-aware policy set) as a required pull request check, and run Prowler on a recurring schedule against every live account, feeding failures into the same alerting path as Section 4.7.3's drift detection.

```
# GitHub Actions: block merge on IaC policy violations, then scan live posture nightly
jobs:
  iac-policy-scan:
    steps:
      - uses: bridgecrewio/checkov-action@v12
        with:
          directory: infra/
          framework: terraform
          soft_fail: false # any failed check blocks the merge
  nightly-posture-scan:
    if: github.event_name == 'schedule'
    steps:
      - run: prowler aws --compliance cis_4.0_aws
```

4.7.3 Configuration Monitoring and Drift Detection

Approved baselines and pre-merge scanning only protect the moment of deployment; configuration drifts afterward through manual console changes, out-of-band emergency fixes, and provider-side default changes. AWS Config, and its equivalents on other providers, continuously records each resource's actual configuration and evaluates it against defined rules, flagging any deviation from the last known-good, IaC-defined state. Treat a drift alert as an incident-adjacent event, not routine noise: a resource changed outside the IaC pipeline either reflects an emergency fix that needs retroactive codification, or unauthorized modification that needs investigation.

Reconcile drift by re-applying the IaC definition, which reverts the manual change, or, if it was legitimate, by updating the IaC source and re-running the policy-as-code checks from Section 4.7.2 before it is accepted as the new baseline.

4.8 Cloud-Specific Checklist

- ☐ Provider selection documented against the shared responsibility model, customer obligations assigned to a team.
 - ☐ IMDSv2 (or provider equivalent) enforced account-wide; IMDSv1 disabled.
 - ☐ Separate accounts/projects for management, security/log-archive, shared services, and each environment.
 - ☐ Golden images/containers rebuilt on a fixed cadence from a CIS-hardened baseline.
 - ☐ Container runtime kept current against escape CVEs; containers run non-root with read-only filesystems.
 - ☐ No fund-moving secret stored as a plaintext env var; secrets referenced from a KMS/secrets manager.
 - ☐ Account-wide storage public-access block enabled; backups follow 3-2-1 with an air-gapped copy.
 - ☐ Dynamic secrets (Vault) or scheduled rotation applied to every credential, no static long-lived secrets.
 - ☐ Hardware-backed, phishing-resistant MFA (FIDO2/WebAuthn) required for every production account.
 - ☐ IaC required for security-sensitive resources; ad hoc console changes are the exception.
 - ☐ Checkov/OPA a required blocking pull request check; Prowler scheduled against every live account.
 - ☐ Drift detection active, alerts routed to a team that reconciles within a defined SLA.
-

5. Operating System Security

Every VM, bare-metal node, and container base image in this handbook eventually rests on an operating system, and OS-level hardening determines whether an exploited application vulnerability stays contained or escalates to full host compromise.

5.1 OS Hardening Guidelines

Start from a published, versioned hardening standard rather than an internal, unwritten convention. The Center for Internet Security (CIS) Benchmarks are the most widely adopted baseline, published per distribution and major version (current benchmarks cover, among others, Red Hat Enterprise Linux 10 and Ubuntu 24.04 LTS), with two profile levels: Level 1, broadly applicable hardening with minimal operational impact, and Level 2, stricter controls for higher-sensitivity environments that may affect compatibility ([CIS Benchmarks](#)). Distribution vendors increasingly ship first-party automation for these baselines; Canonical, for instance, provides CIS-benchmark hardening automation directly for Ubuntu 24.04 LTS rather than leaving teams to implement it by hand ([Ubuntu CIS hardening automation](#)). Bake the chosen profile into the golden image pipeline from Section 4.7.1 so every instance inherits it at boot, rather than applying hardening as a post-provisioning script that can silently fail or be skipped.

```
# Representative CIS Level 1 controls, applied at image build time
# Disable core dumps for setuid programs
echo "* hard core 0" >> /etc/security/limits.conf
# Enforce SSH protocol and key-only authentication
sed -i 's/^#PasswordAuthentication.*/PasswordAuthentication no/' /etc/ssh/sshd_config
sed -i 's/^#PermitRootLogin.*/PermitRootLogin no/' /etc/ssh/sshd_config
# Enable ASLR
echo "kernel.randomize_va_space = 2" >> /etc/sysctl.d/60-hardening.conf
```

Profile	Scope	Typical use
CIS Level 1	Baseline hardening, minimal compatibility impact	Default for all production instances
CIS Level 2	Stricter controls, may affect legacy compatibility	Signing infrastructure, validator hosts, other high-sensitivity workloads

5.2 Patch and Configuration Management

An OS patching program needs two distinct tracks: routine patches on a predictable cadence, and emergency patches for actively exploited vulnerabilities that cannot wait for the next window. For the routine track, automate patch application with `unattended-upgrades` on Debian/Ubuntu or `dnf-automatic` on RHEL-family systems, scoped to security updates so automation does not silently pull in unreviewed feature changes. For the emergency track, maintain a defined SLA

measured in hours, not the days a routine window allows, and rehearse the out-of-band deployment path (the same golden-image and IaC pipeline from Chapter 4, run on demand) so it is not improvised for the first time during an actual incident. Configuration management tooling should enforce that the patch baseline matches policy continuously, the same drift-detection principle from Section 4.7.3 applied at the OS layer instead of the cloud-resource layer.

```
# /etc/apt/apt.conf.d/50unattended-upgrades (Debian/Ubuntu)
Unattended-Upgrade::Allowed-Origins {
    "${distro_id}:${distro_codename}-security";
};
Unattended-Upgrade::Remove-Unused-Dependencies "true";
Unattended-Upgrade::Automatic-Reboot "false"; // reboot on a controlled schedule, not
        silently
```

5.3 Minimal Services and Least Privilege

Every running service, listening port, and installed package is attack surface whether or not it is ever used, and the fastest reduction in a host's exposure is turning off what does not need to run. Audit each image against its actual required function and remove or disable the rest: mail transfer agents, unused language runtimes, package managers left open to unauthenticated installation, and default services (print daemons, legacy remote-access protocols) that ship enabled but serve no purpose on a headless server. Apply least privilege at the process and account level to match: no service should run as root unless its function requires root, `sudo` access should be scoped per-command via `sudoers` entries rather than granted wholesale, and Secure Shell (SSH) access should route through a bastion or session-manager tool with per-session logging rather than direct key-based login to every host.

- ☐ `systemctl list-unit-files --state=enabled` reviewed and every entry justified against the host's function.
- ☐ No service process runs as root unless root privilege is functionally required.
- ☐ `sudo` grants scoped to specific commands, not blanket `NOPASSWD:ALL` entries.
- ☐ Direct SSH key login to production hosts disabled in favor of a bastion or session-manager with logging.
- ☐ Package manager configured to block installs outside an approved repository mirror.

5.4 Logging and Audit Trails

An OS without audit logging cannot answer “what happened” after an incident, only “what state are we in now,” and the two questions need different data. The Linux Audit System (`auditd`) provides kernel-level visibility that application and service logs miss: process execution, file access to sensitive paths, authentication attempts, and privilege escalation, each tied to a specific user and process ID in a tamper-resistant log (Red Hat, [auditing the system](#)). `journald`, the `systemd` logging component, complements this with application and service-level operational logs and is the layer most teams already query day to day. Configure specific `auditd` rules for what matters most on chain-facing infrastructure: modification of SSH authorized-keys files, changes to sudoers, loading of unexpected kernel modules, and execution from a world-writable directory. Neither tool is useful sitting only on local disk; ship both to a centralized destination (a Security Information and Event Management system, or SIEM, or an equivalent log-aggregation pipeline) in near real time, since an attacker with root can otherwise delete local evidence, and align retention with the incident response requirements in the Incident Response Handbook (06).

```
# /etc/audit/rules.d/node-infra.rules
-w /root/.ssh/authorized_keys -p wa -k ssh_key_change
-w /etc/sudoers -p wa -k sudoers_change
-w /etc/sudoers.d/ -p wa -k sudoers_change
-a always,exit -F arch=b64 -S execve -F path=/tmp -k exec_from_tmp
```

Key controls for Part 2

- Bind RPC/archive interfaces to private networks; authenticate, rate-limit, and allowlist methods for everything that reaches them.
- Track client security advisories, patch Critical issues same-day, and rotate the Engine API JWT secret on any suspected exposure.
- Separate validator signing keys from network-facing infrastructure via a remote signer with slashing protection, or DVT for high-value stake.
- Enforce dev/staging/prod separation with distinct cloud accounts and non-assumable IAM boundaries.
- Scope IAM to least privilege, require hardware-backed MFA for production access, and prefer dynamic secrets over static ones.
- Gate infrastructure changes with policy-as-code (Checkov/OPA) before merge and continuous posture scanning (Prowler) after deployment.
- Build every host from a CIS-hardened golden image, disable unused services, and ship `auditd`/`journald` logs to a centralized destination.

Part III: Network and Availability

Part 3 moves from the servers and cloud accounts covered in Part 2 to the network paths connecting them and the availability guarantees users depend on. It builds the segmentation, ingress and egress filtering, transport encryption, and monitoring that keep a network defensible under normal load, works through a distributed denial-of-service (DDoS) threat model specific to remote procedure call (RPC) endpoints, front ends, and application programming interfaces (APIs), and closes on zero-trust design, the operating model that assumes any network segment, including your own, may already be hostile. Each chapter narrows the trust boundary the previous one drew: segmentation defines zones, DDoS mitigation defends the edge of those zones under attack, and zero trust removes the assumption that being inside a zone means anything at all.

6. Network Security

This chapter covers the network-layer controls that separate a Web3 project's tiers from each other and from the open internet: segmentation, filtering, transport encryption, and the monitoring that turns a breach attempt into a detected event rather than a silent one.

6.1 Network Segmentation and Firewalling

Segmentation divides a network into zones so that compromising one zone does not automatically grant reach into the next. NIST's guidelines on firewalls frame the goal precisely: a firewall enforces an access control policy between network segments of differing trust, not merely a boundary at the internet edge ([NIST SP 800-41 Rev. 1](#)). For a Web3 project, the tiers to segment map onto the asset categories from Part 1's inventory (Section 2.1): a public edge that terminates connections from the open internet, an application tier running API and RPC services, a data tier holding databases and indexers, and a signing tier holding validator keys, deployer wallets, or hardware security modules (HSMs), which should sit in its own zone reachable from nowhere except a tightly scoped management path.

The mistake that recurs across incidents in this space is flat networking: a validator node, an internal admin dashboard, and a public-facing RPC endpoint sharing the same virtual private cloud (VPC) subnet with permissive rules between them, because segmenting later felt like it could wait. It shouldn't. Retrofitting segmentation onto a running production network means finding every implicit dependency the flat topology hid, usually under time pressure during an incident.

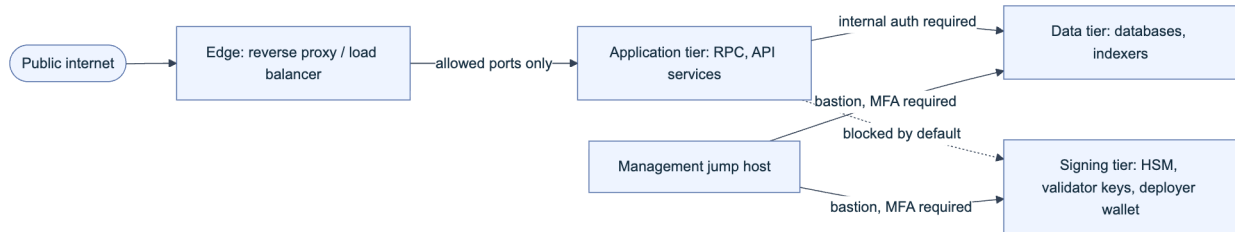


Figure 1. Zone boundaries by sensitivity tier: The application tier has no default path to the signing tier; only a dedicated, MFA-gated management path reaches it.

Zone	Example workload	Inbound source	Baseline control
Edge	Load balancer, reverse proxy	0.0.0.0/0 on 443 only	TLS termination, WAF rules
Application	RPC gateway, API services	Edge zone only	Security group + host firewall
Data	Database, indexer	Application zone only	No direct internet route
Signing	HSM, validator key, deployer wallet	Management jump host only	Default-deny, hardware MFA, logged access

Enforce the zone boundaries with security groups or network access control lists (ACLs) in cloud environments, and with host-based firewalls inside each zone as a second layer, so a single misconfigured cloud rule is not the only thing standing between the internet and a signing key.

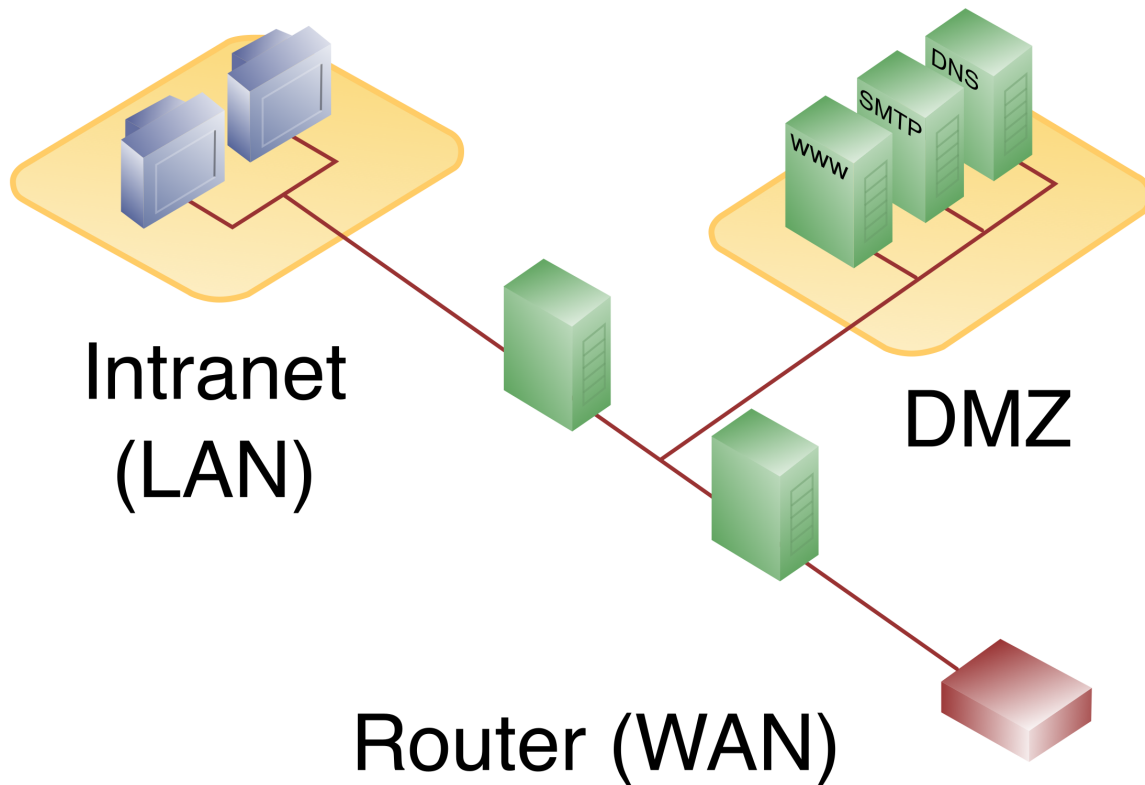


Figure. A classic two-firewall demilitarized zone (DMZ) topology: public-facing services sit in a buffer zone between two firewalls, and the internal network is never directly reachable from the internet. The same pattern generalizes to the edge, application, data, and signing zones in Figure 1 above. Source: [Wikimedia Commons](#), public domain.

6.2 Access Control and Ingress/Egress Filtering

Ingress filtering answers one question for every port on every host: does this specific source need to reach this specific port, and if not, block it by default. The 2018 discovery that attackers had drained more than \$20 million in ether and hundreds of token types from Ethereum clients that exposed their JSON-RPC interface on port 8545 to the open internet is the canonical failure mode. The Geth client's RPC interface grants access to sensitive functions including account unlock and fund transfer, and node operators who bound it to `0.0.0.0` instead of `localhost` handed that capability to anyone scanning the internet for the open port ([BleepingComputer, June 2018](#); [Go Ethereum security guidance](#)). The campaign ran for roughly two years before it was reported, which is the ingress-filtering failure mode at its worst: nothing alerted, because nothing was watching for a port that should never have been open.

Egress filtering is the less-practiced half of the same discipline and matters just as much. A compromised application server with unrestricted outbound access can exfiltrate data or reach command-and-control infrastructure without tripping any ingress rule, because the traffic looks like ordinary outbound activity. Default-deny egress, allow-listing only the specific destinations a service needs, turns a compromised host from a stepping stone into a dead end.

```
# nftables: default-deny both directions, explicit allow only
table inet filter {
    chain input {
        type filter hook input priority 0; policy drop;
        ct state established,related accept
        iif "lo" accept
        tcp dport 443 ip saddr $LB_CIDR accept      # app tier reachable only from LB
        tcp dport 8545 drop                        # JSON-RPC never exposed directly
    }
    chain output {
        type filter hook output priority 0; policy drop;
        ct state established,related accept
        ip daddr $CHAIN_PEERS tcp dport 30303 accept # egress to known chain peers only
        ip daddr $PKG_REGISTRY tcp dport 443 accept
    }
}
```

Pair the rule set with a periodic audit that greps every security group and firewall config for `0.0.0.0/0` on a non-edge port; that single pattern would have caught the exposed-RPC campaign above before it started.

6.3 Encryption in Transit

Every hop in a Web3 project's network, not only the browser-facing one Handbook 01 covers, should assume the wire is hostile. Transport Layer Security (TLS) 1.3, standardized in [RFC 8446](#), removes the negotiation weaknesses and legacy cipher suites that made downgrade attacks against TLS 1.2 and earlier practical, and NIST's server TLS guidance sets 1.3 as the preferred version with 1.2 as the practical floor for anything still supporting it ([NIST SP 800-52 Rev. 2](#)). Terminate TLS 1.3 at every external edge, enforce HTTP Strict Transport Security (HSTS) so a client never silently downgrades to plaintext, and disable TLS 1.0 and 1.1 outright; both are formally deprecated and neither belongs on infrastructure connected to a signing path.

Internal, service-to-service traffic deserves the same standard, not an exception because "it's inside the VPC." A VPC boundary is a network-layer control, not a cryptographic one, and an attacker who reaches the internal network through any other foothold reads plaintext internal traffic for free. Mutual TLS (mTLS), where both sides of a connection present and verify a certificate rather than only the server doing so, closes this gap; Section 8.4 builds it into a full service-mesh identity model.

```
server {
    listen 443 ssl;
    ssl_protocols TLSv1.3;
    ssl_prefer_server_ciphers off;
    add_header Strict-Transport-Security "max-age=63072000; includeSubDomains; preload"
    always;
    ssl_certificate      /etc/ssl/certs/app.pem;
    ssl_certificate_key  /etc/ssl/private/app.key;
}
```

6.4 Monitoring and IDS/IPS Considerations

An intrusion detection system (IDS) observes traffic and raises an alert; an intrusion prevention system (IPS) sits inline and can drop offending traffic automatically. Both matter, and where each sits is a tradeoff between response speed and the risk of a false positive silently dropping legitimate traffic on a path connected to user funds. CIS Controls Safeguard 13, Network Monitoring and Defense, sets the baseline: centralize security event logging across the network, deploy a network intrusion detection capability, and retain logs long enough to support investigation, not just real-time alerting ([CIS Controls](#)).

Two open-source tools cover most of this ground without a commercial license. [Suricata](#), maintained by the Open Information Security Foundation, is a signature-based engine that runs in either IDS or inline IPS mode and inspects protocols up through the application layer. [Zeek](#) takes a passive, behavioral approach instead: rather than matching signatures, it builds rich transaction logs of every connection, DNS lookup, and file transfer, which is what makes it useful for retrospective threat hunting after an alert fires elsewhere. Mature security operations run both side by side, Suricata for real-time signature alerts, Zeek for the behavioral telemetry an investigator pulls up next.

```
# Suricata rule: flag any external attempt to reach a JSON-RPC port directly
alert tcp any any -> $RPC_NODES 8545 (msg:"Unexpected external JSON-RPC access"; \
  flow:to_server; classtype:policy-violation; sid:1000001; rev:1;)
```

Signal source	Catches	Feeds
Suricata (IDS/IPS)	Known-signature exploit traffic, protocol anomalies	Alerting, SIEM
Zeek	Connection, DNS, and TLS transaction logs, behavioral baselines	Threat hunting, retrospective IR
Cloud flow logs (VPC Flow Logs, GuardDuty)	East-west traffic, anomalous API calls, known-bad IP contact	Cloud-native alerting
Host-based endpoint detection	Process, file, and syscall telemetry on the host	SIEM, endpoint IR

7. DDoS Protection

This chapter builds a DDoS threat model specific to the three surfaces a Web3 project exposes publicly, RPC, front end, and API, then works through mitigation, rate limiting, and the incident response playbook for when an attack succeeds despite the defenses.

7.1 Threat Model for DDoS in Web3 (RPC, Front-End, APIs)

A DDoS attack floods a target with more traffic or more requests than it, or the network path to it, can absorb, degrading or blocking service for legitimate users. MITRE ATT&CK catalogs this as [Network Denial of Service \(T1498\)](#), with two sub-techniques that map directly onto Web3 infrastructure: direct network floods (T1498.001), raw volumetric traffic aimed at a target's bandwidth, and reflection amplification (T1498.002), where an attacker spoofs the victim's address in requests to third-party servers that reply with a far larger response. MITRE's [AADAPT framework](#), launched in July 2025 specifically to catalog adversary tactics against digital asset systems, extends this model to blockchain-native impact: an availability attack against a chain's infrastructure can degrade consensus timeliness even when it never touches contract logic or private keys.

Three surfaces carry this risk for a typical Web3 project, and each fails differently. The RPC surface, the endpoints wallets and dApps use to read and broadcast against a chain, is the highest-value target because it sits directly between users and the ability to transact; Manta Network's layer-2 blockchain absorbed more than 135 million fake RPC requests on January 18, 2024, timed to coincide with its token's exchange listing, an attack aimed squarely at denying access during the moment of highest user interest ([Cointelegraph](#)). The front-end surface is the conventional web layer, subject to the same volumetric and application-layer floods any web service faces. The API surface, indexers, analytics endpoints, and any backend a dApp calls outside the chain itself, is frequently the least defended because teams invest budget in the RPC and front end first.

Scale sets the bar any defense must clear. Solana disclosed in December 2025 that it had absorbed a sustained attack peaking near 6 terabits per second (Tbps), the fourth-largest DDoS attack recorded against any distributed system at the time, without downtime: layered defenses including the QUIC transport protocol, stake-weighted quality of service, and local fee markets filtered spam and prioritized legitimate traffic through the flood. Sui, targeted around the same period, experienced block-production delays and degraded performance instead, illustrating that protocol design, not bandwidth alone, determines the outcome of the same class of attack ([CCN](#)).

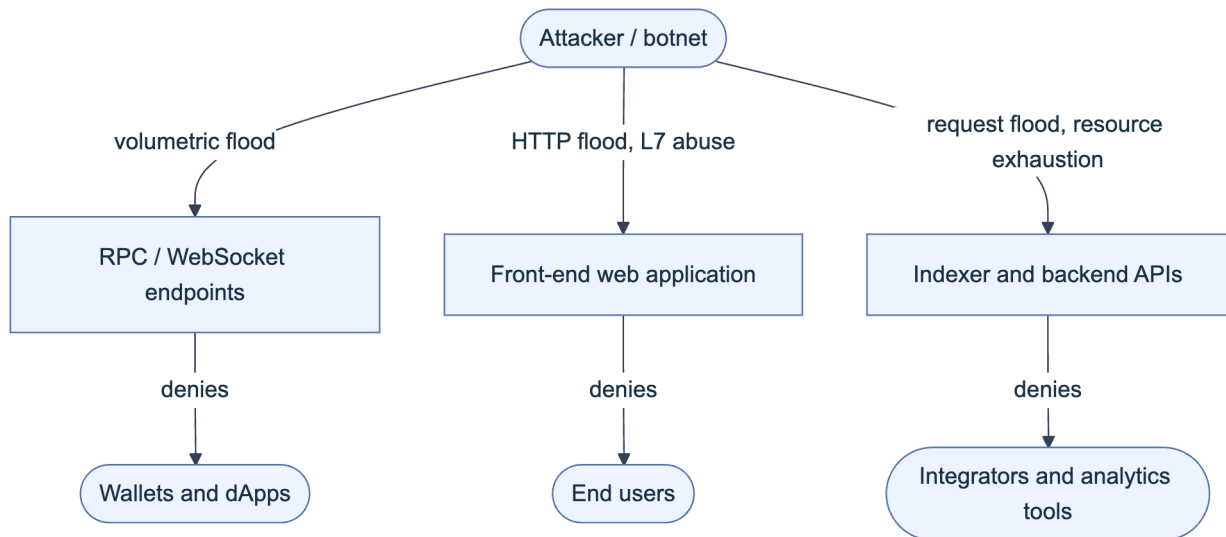


Figure 2. The three surfaces a Web3 project exposes publicly, each with a distinct attack vector and a distinct victim population.

7.2 Mitigation Strategies and Provider Options

Mitigating a volumetric attack requires absorbing more traffic than any single origin can hold, which is why the standard architecture routes traffic through an anycast network of scrubbing nodes before it reaches the origin at all. Cloudflare’s fourth-quarter 2025 DDoS report recorded a 31.4 Tbps attack, the largest publicly disclosed to date, sustained for just 35 seconds, part of a year in which the total volume of attacks the network mitigated more than doubled to 47.1 million ([Cloudflare Q4 2025 DDoS threat report](#)). Earlier in the same year Cloudflare had already blocked a 7.3 Tbps attack lasting 45 seconds, and the growth of Internet of Things (IoT) botnets, the Aisuru botnet chief among them, drove much of the escalation ([Cloudflare Q2 2025 report](#); [Cloudflare Q3 2025 report](#)). No origin network, however large, absorbs that scale directly; the defense has to happen upstream.

Two provider models dominate. Content delivery network (CDN)-integrated protection covers the front-end surface with no separate contract. Network-layer, Border Gateway Protocol (BGP)-based protection, Cloudflare Magic Transit, Akamai Prolexic, or Amazon Web Services (AWS) Shield Advanced, extends the same scrubbing to non-HTTP infrastructure: RPC nodes, validator peer-to-peer ports, and anything that is not a website, by announcing your IP prefixes through the provider’s network and filtering before traffic reaches your origin, an anycast scrubbing path that never lets the flood reach the workloads Figure 2 shows under attack. Shield Advanced adds cost protection against attack-driven autoscaling bills and integrates tightly with AWS’s web application firewall (WAF) and CloudFront; Magic Transit is cloud-agnostic and protects on-premises and multi-cloud infrastructure under one account. The single-versus-multi-provider tradeoff from Part 1 (Section 1.2) applies directly here: a sole scrubbing provider is one more concentration point, so critical-tier assets should have a tested secondary path.

Provider / model	Layer protected	Notes
CDN HTTP DDoS protection	Front end, API (L7)	Included on any proxied site
Cloudflare Magic Transit	Network (L3/L4), BGP-announced prefixes	Cloud-agnostic, enterprise tier, minimum /24 prefix
AWS Shield Advanced	AWS-hosted infrastructure	Cost protection, DDoS Response Team escalation
Akamai Prolexic	Network and application layer	Dedicated scrubbing-center model

7.3 Rate Limiting and Throttling

Rate limiting stops the class of DDoS that pure network-layer scrubbing cannot: requests that are individually well-formed and authenticated but collectively exceed what a backend should serve any single caller. The [OWASP API Security Top 10's API4:2023, Unrestricted Resource Consumption](#), names this directly: an API that does not bound how often, how much, or how expensively a client can call it is vulnerable regardless of how well it authenticates that client. For an RPC endpoint, an expensive call, `eth_getLogs` over a huge block range, or a full archive query, can each cost orders of magnitude more compute than a simple balance check, so a rate limiter counting requests alone still leaves an amplification gap.

Two algorithms cover most implementations. The token bucket adds tokens to a per-client bucket at a fixed rate up to a capacity; each request consumes a token, and the bucket empties under sustained abuse while still tolerating a legitimate short burst. The leaky bucket instead processes requests at a constant rate regardless of arrival pattern, smoothing bursty traffic into steady output at the cost of queuing delay for legitimate spikes. Token bucket suits most per-API-key RPC and API limiting, where occasional bursts from a legitimate integrator are expected; leaky bucket suits protecting a fixed-capacity downstream resource that cannot absorb any burst at all.

```
http {
    limit_req_zone $binary_remote_addr zone=rpc_ip:10m rate=20r/s;
    limit_req_zone $http_x_api_key zone=rpc_key:10m rate=100r/s;

    server {
        location /rpc {
            limit_req zone=rpc_ip burst=40 nodelay;
            limit_req zone=rpc_key burst=200 nodelay;
            proxy_pass http://rpc_backend;
        }
    }
}
```

Algorithm	Behavior	Best fit
Token bucket	Fixed refill rate, allows bursts up to capacity	Per-API-key RPC and API limits
Leaky bucket	Constant output rate, queues excess	Protecting a fixed-capacity downstream resource

Algorithm	Behavior	Best fit
Fixed or sliding window	Counts requests per time window	Simple, coarse edge limits

Layer the limits: a coarse network-layer limit at the edge, a per-API-key limit tied to authentication, and a per-method cost weighting so a cheap call and an expensive archive query do not share the same budget. Tie the API-key limit to the identity model in Section 8.2 rather than treating rate limiting as a network-only control; the two work as one system.

7.4 Incident Response for DDoS

A DDoS runbook has to exist before an attack starts, because the minutes spent deciding who has authority to fail traffic over to a scrubbing provider are minutes the attack keeps running unmitigated. The [Incident Response Handbook \(06\)](#) owns the general response process; this section is the DDoS-specific extension of it.

Detection should trigger on the same telemetry Section 6.4 established: a sudden deviation in request rate, connection count, or upstream bandwidth utilization against a baseline, correlated across the RPC, front-end, and API surfaces so a coordinated multi-surface attack is recognized as one incident rather than three unrelated alerts. MITRE's [detection strategy for T1498](#) recommends monitoring network traffic content and flow for volume and pattern anomalies across platforms, which is the same signal a flow-log pipeline already produces if Section 6.4's monitoring is in place.

DDoS Incident Response Checklist

- ☐ Detect: alert fires from flow-log/IDS baseline deviation, correlated across RPC/front-end/API
- ☐ Triage: confirm attack vs. legitimate traffic spike (launch, listing, viral event)
- ☐ Activate: engage scrubbing provider (BGP failover or DNS-based redirect) per pre-authorized runbook
- ☐ Communicate: status page update, on-call paging per Incident Response Handbook (06)
- ☐ Rate-limit: tighten per-key/per-IP limits at the edge (Section 7.3) as a secondary control
- ☐ Preserve: capture flow logs and IDS alerts for post-incident analysis before rotation/expiry
- ☐ Review: post-incident retro; update baseline thresholds and runbook gaps found under real load

Practice the runbook, not just write it. A scrubbing-provider failover that has never been rehearsed under simulated load is the same untested-hypothesis failure mode Part 1 (Section 1.2.1) flagged for multi-provider redundancy generally: the first real test of a DDoS response plan should not be the first real attack.

8. Zero-Trust Principles

Zero trust reframes every control in this Part around one assumption: the network itself, internal or external, is never a trust boundary on its own. This chapter works through NIST's and the Cybersecurity and Infrastructure Security Agency's (CISA) zero-trust models and applies each tenet to Web3 infrastructure specifically.

8.1 Assume Breach and Verify Explicitly

Zero trust starts from an assumption traditional perimeter security explicitly rejects: that an attacker may already be inside the network, and every access decision should be made as if that were true. NIST codified this model in [Special Publication 800-207, Zero Trust Architecture](#), published in August 2020, around seven tenets: treat every data source and computing service as a resource; secure all communication regardless of network location; grant access to individual resources on a per-session basis; base access decisions on dynamic policy, including client identity, device posture, and behavioral signals; continuously monitor the integrity and security posture of every asset; enforce authentication and authorization dynamically before every access rather than once at login; and collect telemetry about asset state, network traffic, and access requests to continuously improve the policy governing them.

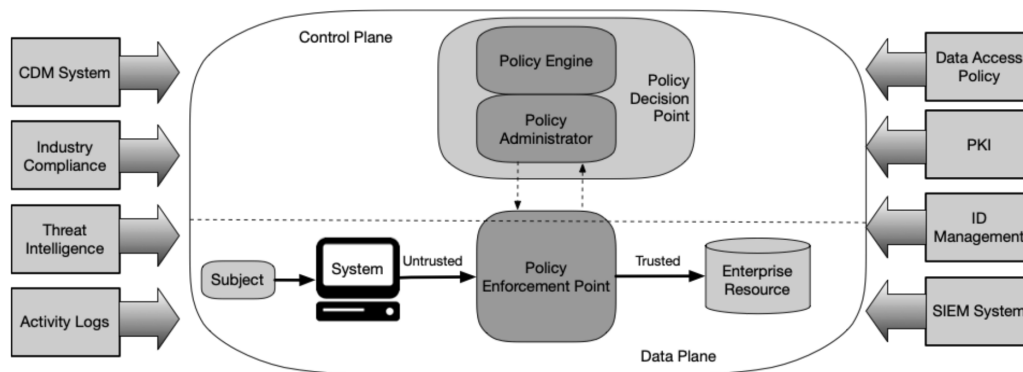


Figure 2: Core Zero Trust Logical Components

Figure. NIST SP 800-207's Core Zero Trust Logical Components diagram. Every access request passes through the policy enforcement point, which only allows traffic once the policy engine and policy administrator, informed by external signals, approve it. Source: [NIST SP 800-207](#), U.S. government work, public domain.

For Web3 infrastructure, “assume breach” carries a sharper edge than in a generic enterprise network, because the asset a breach ultimately threatens is a signing key, not a file share. A perimeter model asks “is this request coming from inside the VPC,” a question a compromised app server inside that VPC answers “yes” by definition. A zero-trust model asks “does this specific

identity, on this specific device, in this specific context, have standing authorization for this specific action,” a question that stays hard for an attacker to answer correctly even after gaining a foothold.

#	NIST SP 800-207 tenet (condensed)	Implemented in
1	Every data source and service is a resource	Asset inventory (Part 1, Chapter 2)
2	All communication is secured regardless of location	Section 8.4, mTLS
3	Access is granted per session	Sections 8.2, 8.6
4	Access decided by dynamic policy (identity, device, behavior)	Section 8.2
5	Integrity and posture of every asset is monitored	Section 8.7
6	Authentication and authorization enforced dynamically, every time	Section 8.6
7	Telemetry drives continuous policy improvement	Section 8.7

NIST describes several deployment models for placing the policy enforcement point (PEP). They are alternatives, not maturity levels: choose the model that most tightly binds policy to the resource without creating an unobservable bypass path.

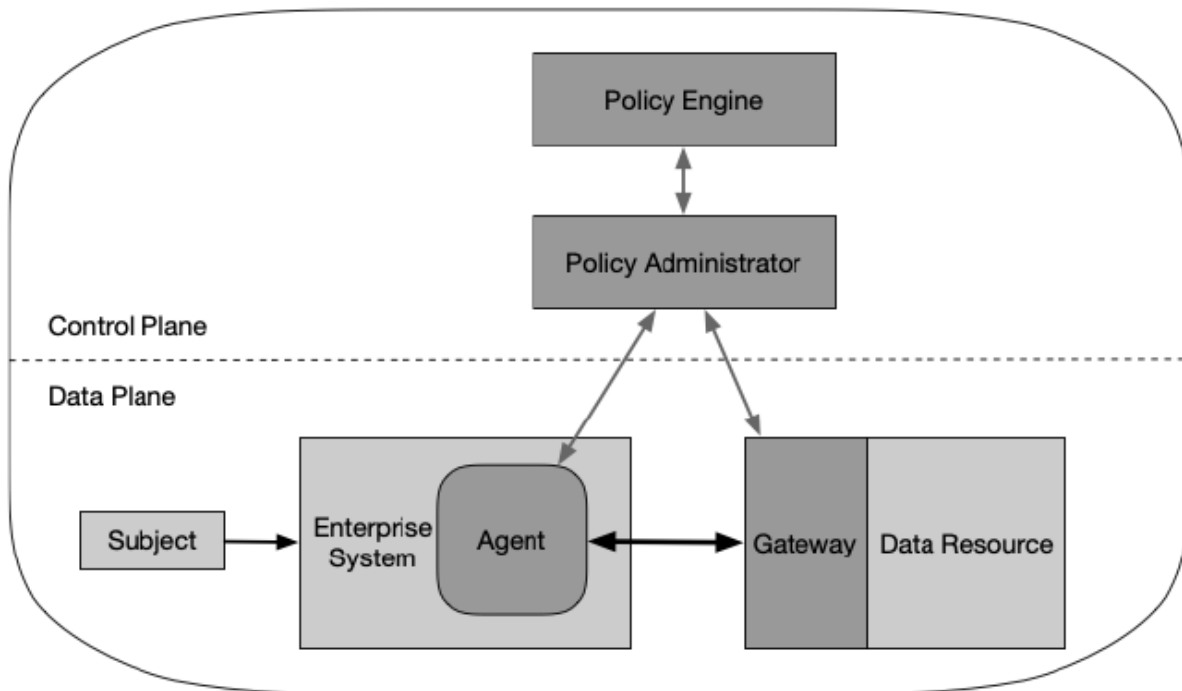


Figure 3: Device Agent/Gateway Model

Figure. Device-agent/gateway model. An endpoint agent establishes the data-plane channel to a gateway protecting the resource; this is a strong fit for managed administrator workstations reaching private RPC, cloud, or signing services. Source: [NIST SP 800-207](#), U.S. government work, public domain.

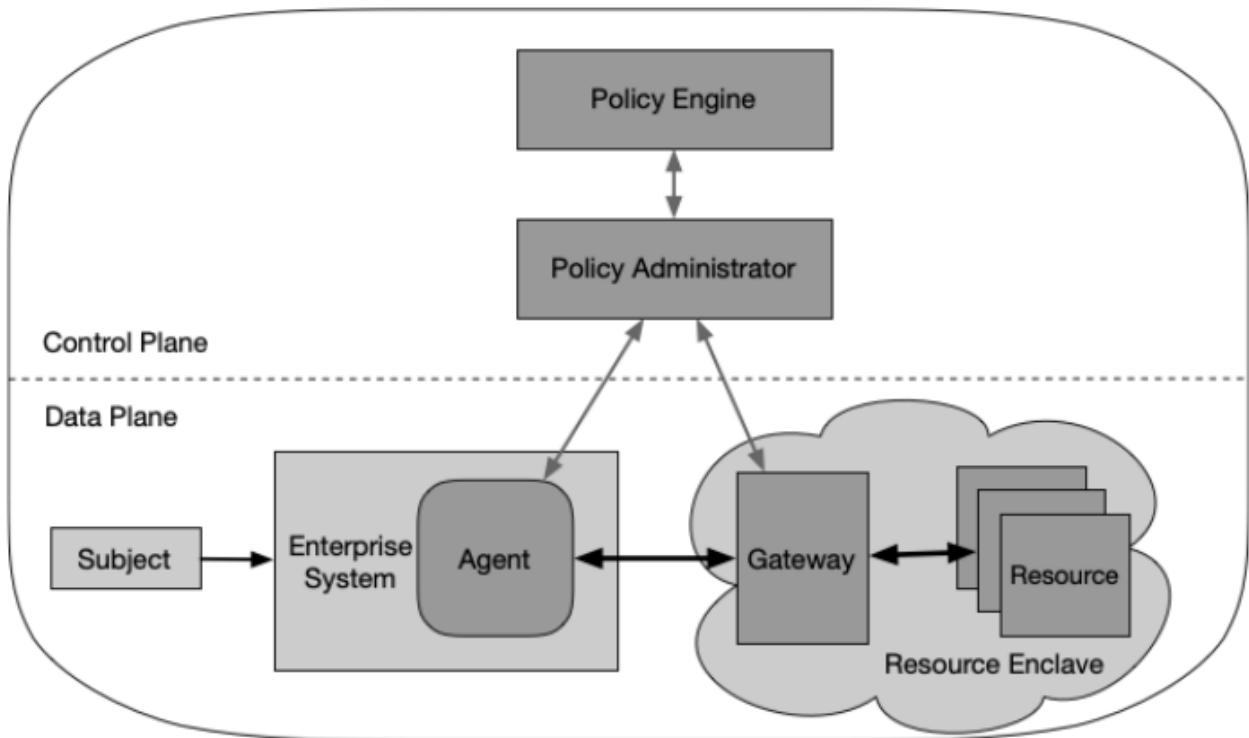


Figure 4: Enclave Gateway Model

Figure. Enclave-gateway model. One PEP protects a resource group, useful for a tightly controlled validator or node subnet, but compromise inside the enclave can still enable lateral movement unless workload-level controls are added. Source: [NIST SP 800-207](#), U.S. government work, public domain.

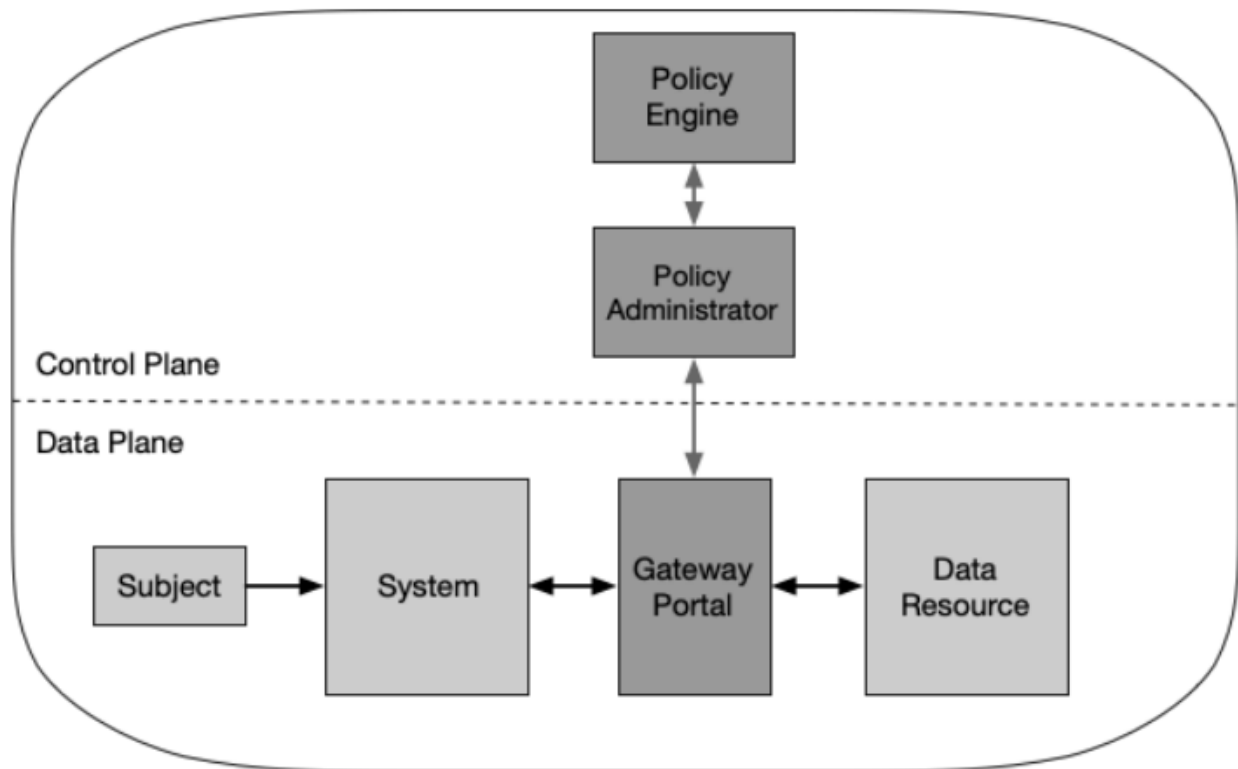


Figure 5: Resource Portal Model

Figure. Resource-portal model. A brokered portal is practical for browser-based administration and unmanaged clients, provided the origin cannot be reached directly and the portal continuously re-evaluates identity, device, and session risk. Source: [NIST SP 800-207](#), U.S. government work, public domain.

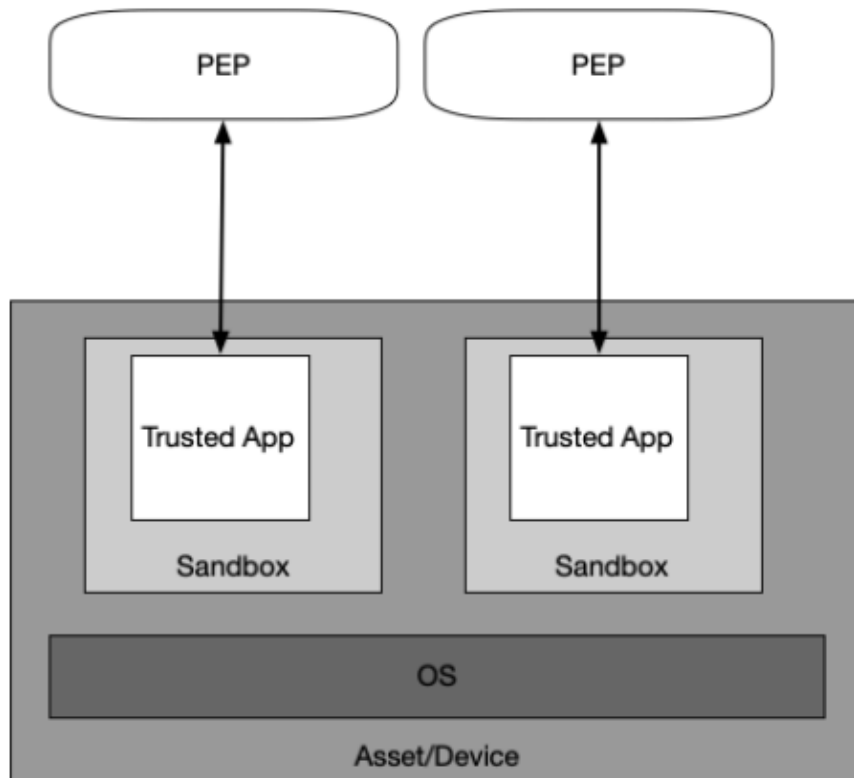


Figure 6: Application Sandboxes

Figure. Application-sandbox model. Per-application isolation narrows the trust boundary on bring-your-own-device or high-risk endpoints; a compromised application does not automatically inherit every connection available to the device. Source: [NIST SP 800-207](#), U.S. government work, public domain.

8.2 Identity and Device Verification

CISA's Zero Trust Maturity Model (ZTMM) 2.0 organizes the discipline into five pillars, Identity, Devices, Networks, Applications and Workloads, and Data, with three cross-cutting capabilities: visibility and analytics, automation and orchestration, and governance ([CISA ZTMM v2.0, April 2023](#)). Identity and device are the first two pillars for a reason: almost every other control in a zero-trust architecture consumes their output as an input to a policy decision.

Identity verification means every human and every service account authenticates with phishing-resistant, hardware-backed multi-factor authentication (MFA), the same standard Part 1's critical-asset tier requires (Section 2.2), using FIDO2/WebAuthn security keys rather than SMS or a software authenticator that a social-engineering call can talk a support desk into resetting. Device verification adds a second, independent signal: is this specific device enrolled, patched, and running expected security tooling, checked continuously rather than once at enrollment. A stolen but valid session token from an unmanaged, unpatched personal laptop should fail a policy check that a token from a managed, current, enrolled device passes, even though both present the identical credential.

The two signals compose into the policy decision that replaces “are you on the VPN”: is this identity authenticated to the required assurance level, on a device meeting the required posture, requesting an action within its authorized scope, right now. Any one of the three failing denies access, regardless of the other two.

8.3 Micro-Segmentation and Least Privilege in Network

Section 6.1 segmented the network into coarse zones; micro-segmentation carries the same logic down to individual workloads, so that even two services inside the same “application tier” zone cannot reach each other unless a policy explicitly allows it. The distinction matters because coarse zoning still leaves lateral movement available to an attacker who compromises any single service inside a zone: if every workload in an application tier can reach every other workload by default, the zone boundary from Section 6.1 protected the perimeter and nothing else.

In a Kubernetes environment, `NetworkPolicy` resources implement micro-segmentation natively, default-denying pod-to-pod traffic and allowing only the specific flows a workload actually needs.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: rpc-services
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-rpc-to-indexer
  namespace: rpc-services
spec:
  podSelector:
    matchLabels: {app: indexer}
  ingress:
    - from:
      - podSelector: {matchLabels: {app: rpc-gateway}}
      ports: [{protocol: TCP, port: 5432}]
    policyTypes: [Ingress]
```

Outside Kubernetes, the same principle applies through host-based firewall rules scoped per service, extending the `nftables` approach from Section 6.2 to intra-zone traffic, or through a service mesh’s authorization policies, covered next. The test for whether micro-segmentation is actually in place, rather than assumed: pick any two workloads in the same zone and ask whether compromising one grants network reachability to the other. If the honest answer is yes without a documented business reason, the segmentation is not fine-grained enough.

8.4 Service Mesh and mTLS for Service-to-Service

Section 6.3 established mTLS as the standard for internal traffic; a service mesh is the infrastructure that issues, rotates, and enforces the certificates that make mTLS operationally sustainable at more than a handful of services. [SPIFFE](#) (Secure Production Identity Framework for Everyone) defines an open standard for workload identity, and [SPIRE](#) is its reference implementation: SPIRE performs node and workload attestation, verifying what a workload actually is (which container, in which namespace, running which binary) before issuing it a SPIFFE Verifiable Identity Document (SVID), an X.509 certificate or JSON Web Token bearing a URI-formatted identity such as `spiffe://prod.example/ns/rpc/sa/gateway`.

Service meshes such as Istio consume this identity model directly: every workload gets an identity derived from its platform context, and the mesh's sidecar proxies use that identity to negotiate mTLS on every connection, transparently to the application code. Configuring the mesh's peer authentication in strict mode means a workload without a valid mesh-issued identity cannot establish a connection at all, closing the gap Section 6.1's zone boundaries leave open by default: two services in the same zone, on the same subnet, still cannot talk to each other without a verified identity on both ends.

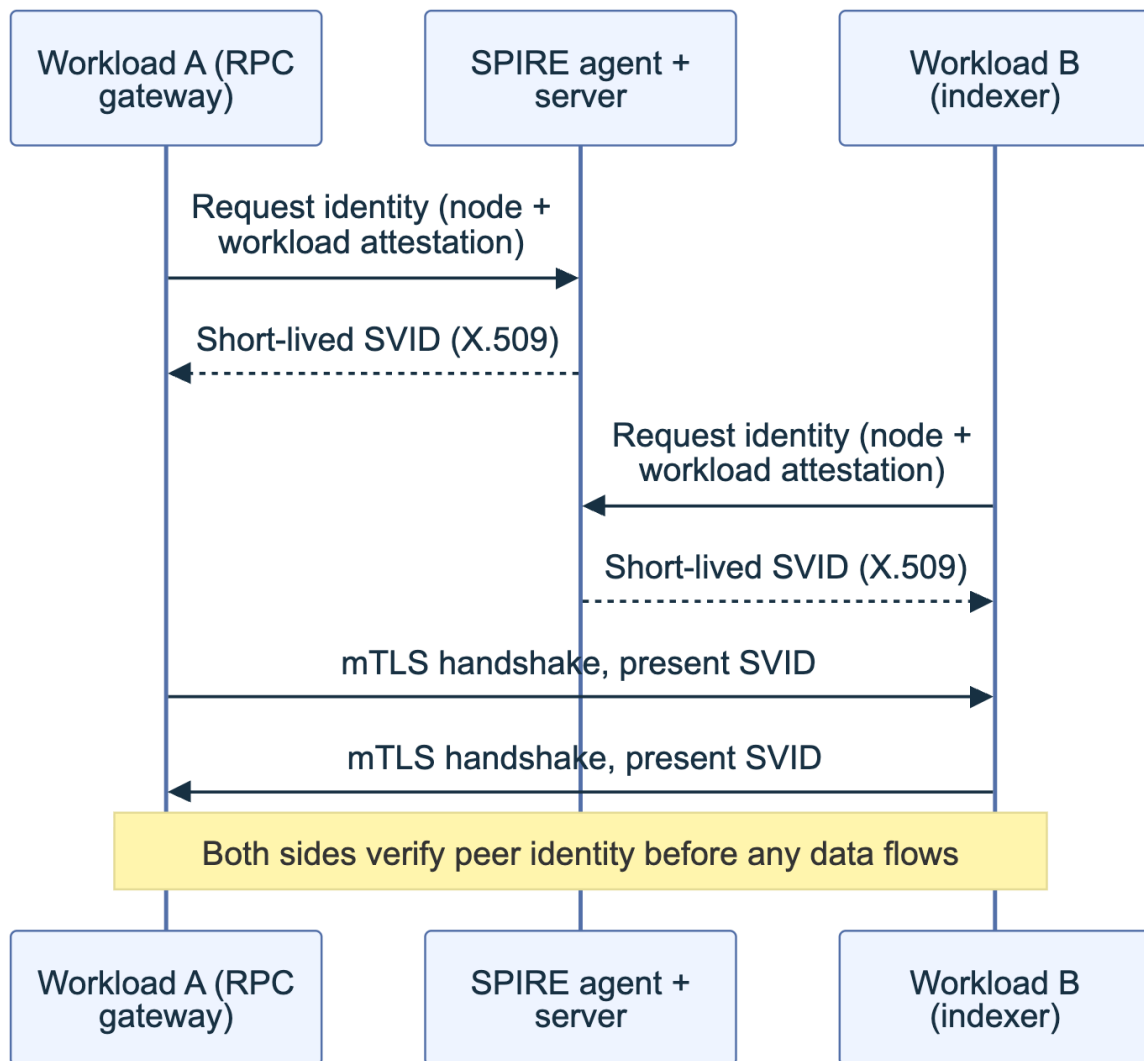


Figure 3. Workload identity issuance and mutual authentication. Neither side trusts the connection until both certificates verify.

The operational payoff is rotation. A certificate authority issuing short-lived SVIDs, commonly valid for under an hour, means a leaked certificate is worthless within the hour without any human running a manual rotation, and a compromised workload's identity can be revoked by simply declining to renew it.

8.5 ZTNA and Secure Tunnels for Remote Access

Remote administrative access is where zero-trust principles most directly replace a legacy control: the corporate or site-to-site virtual private network (VPN) that, once connected, places a remote engineer's laptop on the same broadcast domain as production infrastructure. A VPN authenticates once, at connection time, and then trusts the tunnel for its duration; a stolen VPN session or a compromised laptop inherits full network reach for as long as the tunnel stays up, precisely the flat-trust model Section 8.1 argues against.

Zero Trust Network Access (ZTNA) replaces the network-level tunnel with an application-level, identity-brokered connection: a user authenticates to a policy engine, the policy engine checks identity, device posture, and the specific resource requested, and only then does traffic flow, scoped to that one resource rather than the whole network. Cloudflare Access and comparable platforms implement this at the application layer, while [Tailscale](#), built on the [WireGuard](#) protocol, implements a mesh-network variant: every device gets a WireGuard keypair, and access between any two devices is authorized per-connection through the coordination plane's policy rather than granted by network topology.

Whichever model a team picks, the operational win is the same: revoking one engineer's access means revoking their identity from the policy engine, not rotating a shared VPN pre-shared key that every other remote user also depends on. Bastion hosts and jump servers (Section 6.1's diagram) should themselves sit behind this model rather than behind a static VPN, so the management path into the signing tier inherits the same per-session verification as every other resource.

8.6 Just-in-Time, Ephemeral Credentials

A standing credential, an API key that works today, tomorrow, and every day until someone remembers to rotate it, is a liability the instant it is issued, because its blast radius is bounded only by how long it takes someone to notice it leaked. Just-in-time (JIT) access replaces the standing credential with one generated on demand, scoped to a specific task, and expiring automatically on a short time-to-live (TTL), so a credential that leaks is frequently already worthless by the time an attacker tries to use it.

HashiCorp Vault's dynamic secrets engines implement this directly: rather than storing a static database password or cloud key, Vault generates a unique credential at request time with a built-in TTL and revokes it automatically on expiry, without requiring the requester to remember to clean up. AWS's Security Token Service (STS) provides the same pattern natively for cloud access: `AssumeRole` issues temporary credentials, typically valid for an hour, instead of handing out a long-lived Identity and Access Management (IAM) access key.

```
# AWS: request a 1-hour scoped credential instead of using a static IAM key
aws sts assume-role \
  --role-arn arn:aws:iam::123456789012:role/rpc-deploy-role \
  --role-session-name deploy-$(date +%s) \
  --duration-seconds 3600

# Vault: read a dynamic, short-TTL database credential
vault read database/creds/rpc-readonly
```

Reserve standing credentials for the narrow case a JIT model cannot cover: emergency break-glass access when the identity provider or Vault itself is unreachable. Treat that break-glass path with the critical-asset controls from Part 1 (Section 2.2): sealed, logged, alerting on use, and

rotated immediately after. Section 8.7's continuous monitoring should specifically watch for JIT credentials issued outside expected patterns, because a working JIT system with no one watching its issuance log is only half the control.

8.7 Continuous Verification and Real-Time Monitoring

Zero trust is not a one-time architecture migration; NIST's fifth and seventh tenets (Section 8.1) require continuous monitoring of every asset's integrity and posture, and continuous telemetry collection to keep the policy engine's decisions current. A device that passed a posture check this morning and got compromised by lunch should lose access before the end of the day, not at the next scheduled review, which means the policy engine has to consume live signal rather than a periodic snapshot.

This closes the loop the rest of the Part has been building toward. Section 6.4's IDS and IPS telemetry, Section 8.6's credential-issuance logs, and cloud-native posture signals all feed a security information and event management (SIEM) platform or a user and entity behavior analytics (UEBA) layer that looks for deviation from an established baseline, not just matches against a known-bad signature. An engineer whose credentials always request access from one geography at business hours, then suddenly request a break-glass credential at 3 a.m. from a new network origin, is a UEBA-catchable anomaly even though no individual event in the sequence violated a hard rule.

The final connection runs back to Part 1's asset inventory (Section 2.4): continuous verification is only as good as the record of what should be normal for a given asset, and the inventory's reconciliation loop, declared state against what active discovery finds, is itself a zero-trust control. An unexpected asset appearing on the network is exactly the signal tenet five asks the enterprise to monitor for. Zero trust, in this sense, is less a discrete product to buy than the discipline of wiring every control this Part built, segmentation, filtering, encryption, monitoring, DDoS defense, identity, and ephemeral credentials, into one continuously re-evaluated policy rather than a set of independent, static gates.

Key controls for Part 3

- Segment the network into zones (edge, application, data, signing) with defense-in-depth firewalling at both the cloud security-group and host level, and never expose an RPC or admin interface to `0.0.0.0/0` (Sections 6.1, 6.2).
- Terminate every external and internal connection with TLS 1.3 and enforce mTLS for service-to-service traffic (Sections 6.3, 8.4); disable TLS 1.0/1.1 outright.
- Build a DDoS threat model covering RPC, front-end, and API surfaces separately, and pre-authorize a scrubbing-provider failover runbook before an attack, not during one (Chapter 7).

- Layer rate limiting (edge IP limit, per-key limit, per-method cost weighting) to stop the well-formed abuse that network-layer DDoS scrubbing cannot see (Section 7.3).
- Replace standing VPN access and static credentials with ZTNA, service-mesh mTLS, and just-in-time ephemeral credentials, so every access decision is per-session and continuously re-evaluated rather than granted once and trusted indefinitely (Chapter 8).

Part IV: DNS and Domain Security

A smart contract cannot be phished, but the domain name that points a browser at its front end can be, and DNS or registrar access has now displaced smart contract bugs as the fastest path to draining a well-audited Web3 project. This part covers who is allowed to change where a domain resolves, the cryptographic records (DNSSEC, CAA, SSHFP) that make DNS answers verifiable rather than merely trusted, the email authentication stack (SPF, DKIM, DMARC) that protects the registrar and user notification channel, and the attack patterns that recur across every domain-hijack incident in this handbook's incident record.

9. DNS and Domain Registration

This chapter covers who controls a domain, how that control is locked down against theft, and how it survives the loss of any single provider.

9.1 Registrar Security and Lockdown

The domain registrar account is the root of trust for everything downstream of it: DNS records, TLS issuance, and every subdomain a project runs. Whoever controls that account controls where users land when they type the project's name, which makes the registrar account a critical-tier asset under the sensitivity tiering built in Part I, Section 2.2, deserving the same hardware-backed multi-factor authentication (MFA) and break-glass process as a deployer wallet, not the password-and-SMS treatment typical of a marketing tool signup.

Two incidents define the threat model. In November 2020, attackers vished (voice-phished) GoDaddy support staff into transferring control of at least six domains, including the cryptocurrency platforms Liquid.com and NiceHash, giving the attackers the ability to rewrite DNS records and intercept internal email ([Krebs on Security](#)). In August 2022, Curve Finance's front end was hijacked when the nameserver belonging to Curve's registrar, `iwantmyname`, was compromised directly, redirecting `curve.fi` to a phishing clone and costing users roughly \$570,000 before the

team could point traffic at the unaffected `curve.exchange` domain, hosted through a separate provider ([Decrypt](#); [The Block](#)). Neither attack touched the registrant's own credentials; both defeated the registrar's own operational security.

Lock the account down with hardware-backed MFA (WebAuthn or FIDO2, never SMS-only), a dedicated account not shared with unrelated domains, WHOIS privacy redaction (standard since ICANN's GDPR-driven Temporary Specification), and the EPP transfer authorization code stored offline. Then add the transfer-prevention status codes and, for the domain that fronts user funds, registry lock.

Control	Layer	What it blocks	Bypassed by
<code>clientTransferProhibited</code>	Registrar account	Unauthorized inter-registrar transfer	Compromised registrar account credentials
<code>clientUpdateProhibited</code> / <code>clientDeleteProhibited</code>	Registrar account	Nameserver or contact changes, accidental deletion	Compromised registrar account credentials
Registry lock (e.g., Verisign Registry Lock, CSC, MarkMonitor)	Registry, above the registrar	All of the above, even with valid registrar credentials	Only an out-of-band verification call to the registry, by design

Registrar-level status codes stop an outsider who has not compromised the account; they do nothing once the account itself is owned, which is exactly what happened at GoDaddy and `iwantmyname`. Registry lock closes that gap by moving the final approval outside the registrar's own control plane, requiring a phone call or signed request the registry verifies independently before any nameserver change takes effect ([ICANNWiki](#), [Domain Locking](#)).

9.2 Nameserver and Zone Management

Once the registrar account is locked down, the next control surface is the DNS provider account that hosts the actual zone: the file mapping the domain to its A, CNAME, MX, and TXT records. This account deserves identical treatment to the registrar (hardware MFA, dedicated ownership, logged access) because it can redirect traffic just as effectively without ever touching the registrar at all.

Two operational practices matter beyond account security. First, restrict zone transfers (AXFR), the mechanism a secondary nameserver uses to pull a full copy of the zone. An unrestricted AXFR hands your entire DNS layout, every subdomain, every internal-facing hostname, to anyone who asks, which is reconnaissance a targeted attacker will use before a phishing or takeover attempt. Restrict transfers to known secondary IPs and authenticate them with Transaction Signature (TSIG), a shared-secret HMAC defined in [RFC 8945](#):

```
# Unrestricted: anyone can dump the zone
dig axfr example.com @ns1.example.com
# ;; ANSWER SECTION: (returns every record in the zone)

# BIND fix: require TSIG and restrict by IP
key "transfer-key" {
    algorithm hmac-sha256;
    secret "base64-encoded-shared-secret";
};
zone "example.com" {
    type master;
    allow-transfer { key transfer-key; 203.0.113.10; };
};
```

Second, manage zone records the same way Part I, Section 2.3 recommends managing every other infrastructure resource: as code, reviewed in a pull request, applied through a pipeline rather than a console click. A Terraform-managed zone makes every record change an auditable diff and prevents the silent, undocumented DNS drift that both hides misconfigurations and slows incident response when a responder needs to know what the record looked like before an attacker changed it.

```
resource "aws_route53_record" "app_a" {
    zone_id = aws_route53_zone.primary.zone_id
    name    = "app.example.com"
    type    = "A"
    ttl     = 300
    records = ["203.0.113.50"]
}
```

9.3 Redundancy and Failover

A domain with a single authoritative nameserver, or several nameservers hosted by a single provider, inherits that provider's availability and integrity risk in full, the same consolidated-versus-diversified tradeoff Part I, Section 1.2 established for cloud accounts and CDNs, applied here to the record that everything else resolves through. If that one DNS provider suffers an outage, a BGP leak, or an account compromise, every service relying on the domain goes down or gets hijacked with it.

Diversify authoritative nameservers across two providers on different networks (different autonomous systems, different upstream transit) so that a single provider's incident degrades rather than eliminates resolution. A hidden-primary architecture, where the authoritative source of truth is not itself publicly listed as a nameserver, works well here: the primary pushes zone updates to two independent secondary providers, each publicly authoritative and neither dependent on the other's infrastructure.

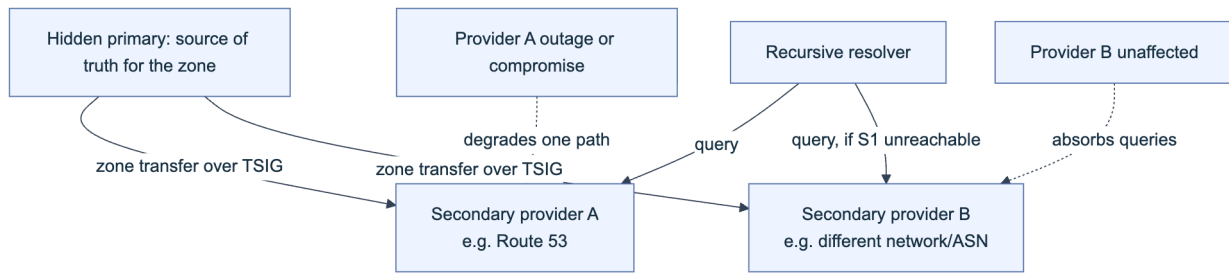


Figure 1. Authoritative nameserver redundancy. Neither secondary depends on the other, so an outage or account compromise at one provider does not remove the domain from resolution.

Set TTLs deliberately: low enough (300 to 900 seconds) that a legitimate emergency record change, redirecting to a status page during an incident, propagates quickly, but not so low that recursive resolvers hammer the authoritative servers or that a transient blip triggers unnecessary failover. Test the failover path before relying on it; an untested secondary is, per Part I's framing, a hypothesis rather than a control. Redundancy at this layer also complements the distributed denial-of-service (DDoS) protections covered in Part III, Chapter 7, since a DNS-layer outage produces the same user-facing unavailability as a network-layer attack.

10. DNS Security (DNSSEC, CAA, etc.)

This chapter covers the cryptographic and policy records that convert a DNS answer, and the certificate issued against it, from something a resolver merely accepts into something it can verify.

10.1 DNSSEC Implementation

Plain DNS has no authentication: a resolver accepts whatever answer arrives on the expected port with a matching query ID, which is precisely the weakness Dan Kaminsky demonstrated at scale in 2008 and which underlies cache poisoning attacks to this day. Domain Name System Security Extensions (DNSSEC), defined across [RFC 4033](#), [RFC 4034](#), and [RFC 4035](#), fixes this by having each zone sign its own records and chaining that signature up to the DNS root, so a validating resolver can cryptographically confirm an answer was not forged or altered in transit. The National Institute of Standards and Technology's [Secure Domain Name System Deployment Guide, SP 800-81 Revision 3](#) (finalized March 2026) treats DNSSEC as a baseline control within a zero-trust and defense-in-depth DNS architecture.



Figure 2. The DNSSEC chain of trust. A resolver validates each delegation by hashing the child zone's DNSKEY and comparing it against the DS record published in the parent, up to a root it trusts implicitly.

Adoption remains uneven and worth knowing before treating DNSSEC as a default. As of mid-2026, roughly 94% of top-level domains (TLDs) in the root are signed, but only about 4.3% of second-level domains across a 240-million-domain sample actually deploy it, and only around 11.5% of DNS queries are handled by resolvers that validate signatures at all ([DNSChkr](#), [DNSSEC Adoption 2026](#); [technologychecker.io](#)). DNSSEC protects your zone's integrity against every validating resolver on the path, even though most traffic today still passes through resolvers that skip validation entirely, so it is a defense-in-depth control worth deploying regardless of universal adoption, not a control you should skip because coverage is partial.

10.1.1 Key Management and Signing

DNSSEC uses two key pairs per zone. The Key-Signing Key (KSK) signs the zone's DNSKEY record set and anchors the chain of trust to the parent zone through the DS record; the Zone-Signing Key (ZSK) signs the actual resource record sets and rotates more frequently since it carries none of the parent-zone dependency. [RFC 8624](#) sets ECDSAP256SHA256 as the recommended algorithm for new deployments, offering equivalent strength to RSASHA256 with a materially shorter signature, which matters for UDP packet size and DNS amplification resistance.

Most managed DNS providers (Route 53, Cloudflare, Google Cloud DNS) automate key generation, signing, and rollover behind a single toggle, which is the right default for most teams: manual key management with `dnssec-keygen` and `dnssec-signzone` under BIND is more error-prone and is warranted mainly for teams running their own authoritative infrastructure.

```
# Generate a KSK and ZSK for a self-managed BIND zone
dnssec-keygen -a ECDSAP256SHA256 -f KSK example.com
dnssec-keygen -a ECDSAP256SHA256 example.com
# Sign the zone, producing RRSIG, NSEC, and DNSKEY records
dnssec-signzone -A -3 $(head -c 16 /dev/urandom | xxd -p) -N INCREMENT -o example.com
-t example.com.zone
```

Key rollover follows the pre-publish method described in [RFC 6781](#): publish the new key alongside the old one, wait out the TTL so every cached copy of the old key set expires, switch signing to the new key, then remove the old one. Skipping the wait period breaks validation for any resolver still holding a cached, now-stale key. Use NSEC3 rather than NSEC for the denial-of-existence records unless zone enumeration resistance is not a concern, since plain NSEC lets an attacker walk the entire zone by following its ordered chain of “next domain name” pointers.

10.1.2 DS Records and Registrar Support

Signing a zone accomplishes nothing until the parent zone (the TLD) publishes a Delegation Signer (DS) record pointing to it, a hash of your zone's KSK, defined in [RFC 4034](#) and extended to SHA-256 digests by [RFC 4509](#). That DS record is submitted through the registrar, which relays it to the registry, which is why DNSSEC support varies by registrar and belongs on the selection checklist in [Section 9.1](#) rather than assumed. Some registrars still lack a DS-submission workflow or API entirely, which quietly rules out DNSSEC for any domain registered there.

The failure mode unique to this step is severe: if the DS record at the registry does not match the DNSKEY currently active in your zone, whether from a botched rollover or a registrar that dropped the submission, every validating resolver refuses the domain outright rather than falling back to unsigned answers. This turns a signing mistake into a full outage for a meaningful slice of users, not a silent downgrade. Verify the chain after every change:

```
# Confirm the DS record the registry publishes matches your zone's active KSK
dig DS example.com +short
dig DNSKEY example.com +short
# Full chain validation, reports BOGUS if DS and DNSKEY disagree
delv example.com
```

10.2 CAA and Certificate Control

The Certification Authority Authorization (CAA) record, standardized in [RFC 8659](#) (November 2019, obsoleting RFC 6844), lets a domain owner declare in DNS which certificate authorities (CAs) may legitimately issue TLS certificates for it. Since the CA/Browser Forum's Ballot 187 took effect in September 2017, every publicly trusted CA must check for a CAA record before issuing and refuse issuance if the record names a different authority, which converts CAA from an advisory hint into a binding, browser-relevant control.

```
example.com. IN CAA 0 issue "letsencrypt.org"
example.com. IN CAA 0 issuewild ";"
example.com. IN CAA 0 iodef "mailto:security@example.com"
```

The three tags above cover the common case for a Web3 front end: authorize exactly one CA for standard certificates, forbid wildcard issuance entirely (";" means no CA is authorized) since a stolen wildcard certificate is far more dangerous than a single-name one, and route any violation report to a monitored inbox. CAA prevents future mis-issuance but says nothing about certificates already issued before the record existed, which is where Certificate Transparency (CT) logs, mandated for all publicly trusted certificates and formalized in [RFC 9162](#), complete the picture: monitor [crt.sh](#) or a CT-log alerting service for any certificate issued for your domain, and treat an unexpected entry as an incident regardless of whether CAA was in place at the time. CAA and DNSSEC compound well together, since an attacker who can forge unsigned DNS answers can also forge a fraudulent CAA record to permit their own issuance; DNSSEC is what makes the CAA record itself trustworthy to a checking CA.

10.3 SSHFP and Other Record Types Where Relevant

Secure Shell Fingerprint (SSHFP) records, defined in [RFC 4255](#) and extended with ECDSA and SHA-256 support in [RFC 6594](#) and Ed25519 support in [RFC 7479](#), publish a host's SSH public key fingerprint in DNS so a connecting client can verify it automatically rather than relying on trust-on-first-use (TOFU), the "yes, I trust this fingerprint" prompt most operators click through without actually checking. For a Web3 infrastructure footprint, this matters most on the bastion

and jump hosts that front RPC and validator nodes covered in Part II: an operator SSHing into a node for the first time from a new laptop has no independent way to verify the host key without SSHFP, and TOFU acceptance is exactly the gap an attacker performing an adversary-in-the-middle attack on that first connection relies on.

```
# Generate SSHFP records from a host's existing key material
ssh-keygen -r bastion.example.com -f /etc/ssh/ssh_host_ed25519_key
# bastion.example.com IN SSHFP 4 2 a1b2c3...

# Client-side: instruct ssh to verify against DNS (requires DNSSEC)
# In ~/.ssh/config:
#   Host bastion.example.com
#     VerifyHostKeyDNS yes
```

SSHFP is only as trustworthy as the DNSSEC signature protecting it; without DNSSEC, an attacker capable of spoofing DNS answers can forge the SSHFP record along with the fake host they want a client to trust, so deploy the two together. The same principle extends to TLSA records under DNS-Based Authentication of Named Entities (DANE, [RFC 6698](#)), which pin a TLS certificate or CA directly in DNS; DANE sees limited adoption in general-purpose web browsers but is worth considering for internal service-to-service TLS where you control both ends and want certificate pinning independent of the public CA ecosystem.

11. Email and Communication Security (SPF, DKIM, DMARC)

This chapter secures the email channel a Web3 team relies on for registrar and cloud-provider recovery, incident notifications, and every security alert sent to users.

11.1 Preventing Spoofing and Phishing

Standard email carries no authentication of the visible “From” address, which is why anyone can send a message that appears to come from your domain’s support address. Three complementary records close that gap, and understanding what each one actually checks matters because none of them alone stops a determined spoofer.

Record	RFC	What it authenticates	Key limitation
Sender Policy Framework (SPF)	RFC 7208	The sending IP is authorized for the envelope-from domain	Does not check the visible From: header a user sees; breaks on mail forwarding
DomainKeys Identified Mail (DKIM)	RFC 6376	A cryptographic signature ties the message to the signing domain, survives forwarding	Says nothing about whether the visible From: header matches the signing domain

Record	RFC	What it authenticates	Key limitation
Domain-based Message Authentication, Reporting and Conformance (DMARC)	RFC 7489	Requires SPF or DKIM to pass <i>and</i> align with the visible From: domain; defines the enforcement policy and reporting	Only as strong as the policy configured; <code>p=none</code> reports but blocks nothing

The attack this stack defeats is straightforward and common in the incident record this handbook cites: an attacker sends a message that looks like it came from `support@yourproject.com`, asking a registrar’s staff to authorize a transfer, or asking a user to “reconfirm” a wallet connection. Without DMARC alignment enforced, a spoofed message with no valid SPF or DKIM for your domain still lands in an inbox looking legitimate. With `p=reject` enforced, receiving mail servers discard it before delivery.

11.2 Configuration and Monitoring

Publish all three records as DNS TXT entries, and roll the DMARC policy out gradually: start at `p=none` to observe traffic through aggregate reports without affecting delivery, move to `p=quarantine` once legitimate senders are fully accounted for, then to `p=reject`. CISA’s Binding Operational Directive 18-01 codified exactly this progression for U.S. federal agencies, mandating `p=none` with reporting within 90 days and `p=reject` within one year, a timeline worth adopting even outside a federal mandate because it is what production experience showed prevents self-inflicted outages from a hasty jump to full enforcement ([CISA BOD 18-01](#)).

```
example.com.      IN TXT "v=spf1 include:_spf.google.com include:sendgrid.net -all"
selector1._domainkey.example.com. IN TXT "v=DKIM1; k=rsa; p=MIGfMA0GCSq..."
_dmarc.example.com. IN TXT "v=DMARC1; p=quarantine; pct=100; rua=mailto:dmarc-reports@example.com; ruf=mailto:dmarc-forensics@example.com"
```

Watch two operational traps. SPF evaluation fails closed once a lookup chain exceeds 10 DNS lookups (RFC 7208 §4.6.4), a limit that a growing stack of third-party senders (transactional email, marketing tools, help desk software) can quietly exceed, producing a `permerror` that some receivers treat as an outright fail; audit the SPF chain whenever a new sending service is added. Second, review DMARC aggregate reports (the `rua` address) on a fixed schedule, not only after an incident: they are the only visibility into who is sending mail claiming your domain, and a spike in failing sources is often the first signal of a phishing campaign impersonating the project before any user reports it.

11.3 Relevance to Web3 (Notifications, Alerts, Admin Contact)

The admin contact email tied to a domain registration is frequently the master recovery key for an entire Web3 project’s operational stack, not just the domain itself. Registrar account recovery, cloud console password resets, GitHub organization recovery, and social media account recovery all commonly route through email, which means a compromised admin inbox cascades into every

downstream system that trusts it, often faster than the team can react. Host that mailbox on infrastructure independent of the domain it recovers, protect it with hardware MFA, and reserve it for recovery flows rather than day-to-day correspondence that widens its exposure to phishing.

The stack in Sections 11.1 and 11.2 also protects the other direction: alerts a project sends to users. Since February 2024, Google and Yahoo have required bulk senders (roughly 5,000 or more messages to personal accounts in 24 hours) to authenticate with SPF and DKIM and maintain a DMARC record at minimum `p=none`, rejecting or spam-folding noncompliant mail outright by late 2025 ([PowerDMARC](#); [Mailgun](#)). A project that skips this configuration risks its own security notices, a suspicious withdrawal alert, a domain-change confirmation, a breach disclosure, landing in spam or being silently rejected at exactly the moment a user needs to see it. The [Incident Response Handbook \(06\)](#) assumes these notification channels function reliably; this section is the precondition that makes that assumption safe.

12. DNS and Hosting Attack Playbooks (Overview)

This chapter closes Part IV by naming the attack patterns DNS and domain infrastructure face in production Web3 systems and pointing to where the full incident-response procedure lives.

12.1 Common Attack Vectors (Hijack, Poisoning, Abuse)

Every DNS-layer incident in this handbook's record reduces to one of five patterns, and recognizing which one is unfolding determines the correct first response.

Registrar account takeover through social engineering or phishing of support staff, as in the November 2020 GoDaddy vishing campaign that compromised NiceHash, Liquid.com, and Escrow.com ([Krebs on Security](#)). **Registrar or registry infrastructure compromise**, where the attacker never touches the registrant's credentials at all, as in Curve Finance's August 2022 incident against its registrar [iwantmyname](#) ([Decrypt](#)). **Provider-wide DNS-hosting compromise**, illustrated by the July 2024 wave that hit Compound Finance and Celer Network through DNS records hosted on Squarespace, exploiting accounts without MFA and putting over 120 additional DeFi protocols at risk on the same platform ([Decrypt](#); [Cybernews](#)). **Border Gateway Protocol (BGP) hijacking**, which forges the internet-routing announcements DNS resolution depends on rather than DNS itself, as in the April 2018 hijack of Amazon Route 53 that redirected MyEtherWallet users ([ThousandEyes](#)) and the February 2022 hijack of KakaoTalk's JavaScript SDK infrastructure that cost KLAYswap users \$1.9 million despite KLAYswap's own systems being untouched ([CertiK](#)). **Subdomain takeover**, where a dangling CNAME still points at a deprovisioned cloud resource an attacker can re-claim, the unowned-asset failure mode Part I, Section 2.2 flags directly.

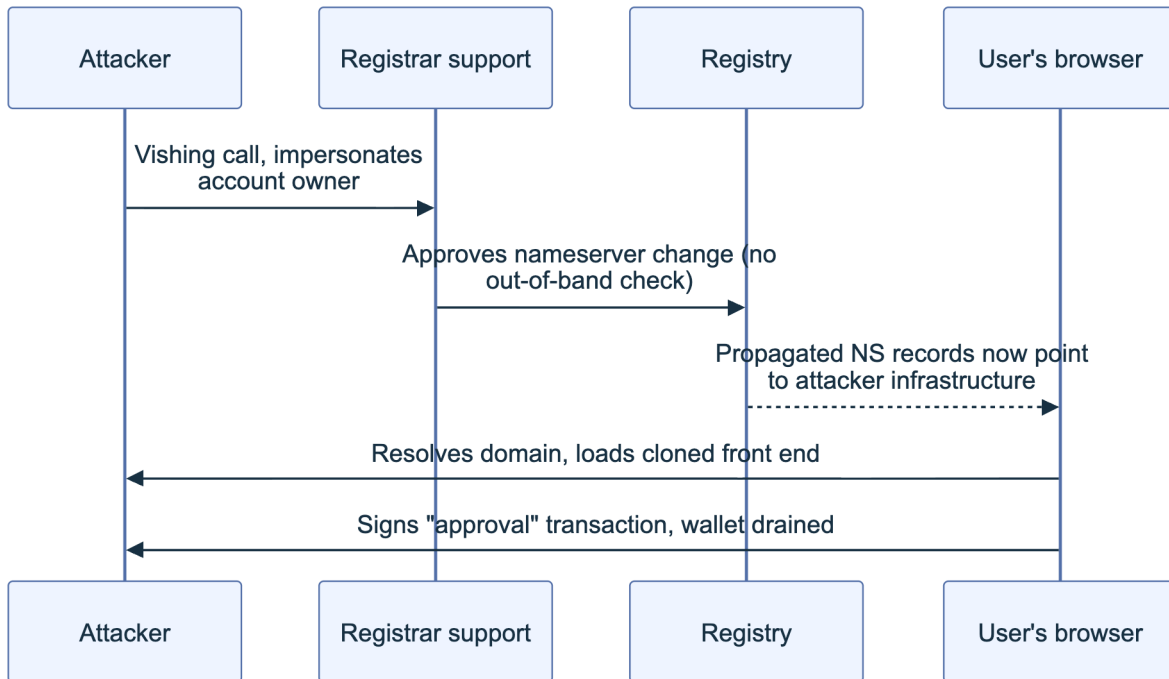


Figure 3. The generic registrar-hijack sequence behind the GoDaddy 2020 and Curve 2022 incidents. Registry lock (Section 9.1) breaks this chain at the second step by requiring verification the registrar's own support process cannot bypass.

12.2 Detection and Response Steps

Detection has to run continuously, because every incident above was discovered by monitoring, not by the affected team noticing on its own. Maintain a known-good baseline of NS, A, CNAME, MX, and CAA records and diff against it on a short interval; watch Certificate Transparency logs (Section 10.2) for issuance you did not request; monitor domain expiration dates with enough lead time that a lapsed renewal never becomes an opportunistic re-registration; and run periodic WHOIS/RDAP checks for registrant or nameserver changes outside a known maintenance window. Commercial and open-source tooling (SecurityTrails, Microsoft Defender External Attack Surface Management, `dnstwist` for typosquat variants of your own domain) automate most of this.

On suspected hijack, the response sequence is time-critical:

1. **Verify out-of-band**, through a channel independent of the potentially compromised domain and email, before taking any action based on an alert alone.
2. **Contact the registrar or registry through a pre-established emergency channel**, invoking registry lock override procedures if in place, to freeze further changes.
3. **Rotate every credential** that touched the compromised path: registrar account, DNS provider account, and any API tokens with zone-edit permission.
4. **Revoke and reissue TLS certificates** if origin certificate material may have been exposed, and check CT logs for unauthorized issuance during the incident window.

5. **Publish a warning through a verified, unaffected channel**, following the precedent Curve and Celer set by redirecting users to a secondary domain or a pinned social media account the attacker does not control.
6. **Preserve evidence**, including DNS record history, registrar access logs, and timestamps, for the post-incident review.

Every step above is a summary pointer, not the executable procedure; the [Incident Response Handbook \(06\)](#) owns the detailed runbook, including communication templates and legal or law-enforcement escalation paths.

12.3 References to Detailed Playbooks

This chapter deliberately stops at the overview level. The full, step-by-step operational playbooks live in three places built for that purpose: the consolidated node and DNS checklist in Part VI, Section 15, the printable [DNS and Domain Security Checklist in Appendix C](#), and the incident-type-specific runbooks in the Incident Response Handbook (06), which cover communication, legal, and forensic steps this handbook does not duplicate. Treat this Part as the design and detection reference you consult before an incident, and the Incident Response Handbook as the procedure you execute during one; keeping that separation, rather than merging a response runbook into every domain that touches infrastructure, is what keeps both documents usable under pressure.

Key controls for Part 4

- Put hardware-backed MFA and registry lock (not just registrar-level `clientTransferProhibited`) on any domain that fronts user funds; registrar-level locks did not stop the GoDaddy or Curve incidents because the registrar account itself was the compromised layer.
- Diversify authoritative nameservers across two providers on independent networks, and restrict zone transfers to authenticated secondaries with TSIG.
- Deploy DNSSEC with ECDSAP256SHA256, verify the DS record at the registry matches the active DNSKEY after every change, and pair it with CAA records and Certificate Transparency monitoring.
- Roll out DMARC from `p=none` to `p=reject` deliberately, and treat the domain's admin recovery mailbox as a critical-tier asset independent of the domain it recovers.
- Monitor DNS records, certificate issuance, and domain expiration continuously; every hijack in this handbook's record was caught by monitoring or a user report, never by the registrant noticing first.

Part V: Identity and Access Management

Every control in Parts II through IV, a hardened node, a segmented network, a locked-down DNS registrar, still depends on a single upstream question: who is allowed to touch it, and how do you know it is really them? Identity and access management (IAM) is the layer that answers that question for the infrastructure itself, as distinct from the application-level authentication a dApp's front end offers its end users. This part covers role design, privileged and emergency access, the machine identities that now outnumber human ones on most infrastructure teams, hardware-backed authentication, and the offboarding discipline that keeps yesterday's employee from being today's attacker.

13. Identity and Access Management for Infrastructure

This chapter works through the five IAM decisions that recur on every infrastructure team: how roles are modeled, how privileged and emergency access is granted without leaving standing risk, how non-human identities are secured, how strong an authenticator must be, and how access is torn down cleanly when it should no longer exist.

13.1 Role-Based Access Control (RBAC)

Role-based access control (RBAC) assigns permissions to a role tied to a job function, cloud region operator, RPC node administrator, DNS record editor, and then assigns humans and services to that role rather than granting permissions to an identity directly. NIST SP 800-53's AC-6 (Least Privilege) control makes the underlying requirement explicit: an organization must employ the principle of least privilege, allowing only the access necessary to accomplish an assigned task, for both user accounts and the processes acting on their behalf (NIST SP 800-53 r5, AC-6). RBAC is the standard mechanism for operationalizing that requirement at scale: define the role once, tie its permission set to the narrowest set that job function needs, and every subsequent grant or revoke is a role assignment change rather than a bespoke policy edit.

RBAC alone breaks down when access should depend on conditions that a static role cannot express, an operator can restart archive nodes, but only ones tagged for the network they are certified on, only from a managed device, only with a fresh multi-factor authentication (MFA) assertion. Attribute-based access control (ABAC) extends RBAC by evaluating attributes of the subject, the resource, and the environment against policy at request time rather than encoding every combination as a separate role, and NIST SP 800-162 is the reference definition of the model ([NIST SP 800-162, Guide to ABAC](#)). In practice, mature infrastructure teams combine both: RBAC supplies the coarse role (`archive-rpc-operator`), and ABAC conditions narrow that role's grant to the specific resources and context it should apply to. The policy below extends the tagged `rpc_node_archive_01` resource introduced in Part 1's asset inventory example, granting the operator role only against resources carrying the matching `Purpose` and `Sensitivity` tags, and only when a recent MFA assertion is present.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ArchiveRpcOperatorScopedAccess",
      "Effect": "Allow",
      "Action": ["ec2:RebootInstances", "ec2:DescribeInstances", "ssm:StartSession"],
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "aws:ResourceTag/Purpose": "ethereum-archive-rpc",
          "aws:ResourceTag/Sensitivity": "critical"
        },
        "Bool": { "aws:MultiFactorAuthPresent": "true" },
        "NumericLessThan": { "aws:MultiFactorAuthAge": "3600" }
      }
    }
  ]
}
```

Dimension	RBAC	ABAC
Grant unit	Role, assigned to a subject	Policy, evaluated against subject/resource/environment attributes
Best fit	Stable job functions with a bounded permission set	Context-dependent access (tags, device posture, time, MFA freshness)
Audit question	"What role is this identity in?"	"What attributes made this request pass policy?"
Common failure	Role explosion (a role per exception)	Attribute sprawl (tags that drift out of date)
Web3 infra example	<code>dns-registrar-admin</code> , <code>archive-rpc-operator</code>	Restrict a role to resources tagged <code>Sensitivity=critical</code> only

Role sprawl is the most common failure in practice: teams start with a handful of clean roles and, under deadline pressure, add one-off exceptions until every account effectively has a unique permission set, which is RBAC in name only. Review role definitions on a fixed cadence (Section 14.2's checklist gives a starting interval) and collapse any role that exists for a single person back into an attribute condition on a shared role wherever possible.

13.2 Privileged Access and Break-Glass

Privileged access, the set of grants that can change infrastructure state, alter security controls, or reach signing keys, deserves controls beyond standard RBAC because its abuse or compromise has outsized blast radius. The dominant modern pattern is just-in-time (JIT) access: instead of a standing grant that sits active whether or not it is in use, an identity requests a specific privilege for a specific, time-boxed window, and the grant expires automatically. Vendors in this space describe the strongest form of this posture as zero standing privileges (ZSP), no persistent privileged account exists to steal in the first place, because privilege is provisioned on demand and removed the moment the task completes ([CyberArk, what is just-in-time access](#); [BeyondTrust, guide to JIT PAM](#)). This is the direct infrastructure-layer expression of the continuous-verification principle in NIST's Zero Trust Architecture (SP 800-207): no request, privileged or otherwise, is trusted by default on the basis of a prior grant ([NIST SP 800-207](#)).

JIT access assumes the normal path (an identity provider, a JIT broker, a working network) is available. A break-glass account is the deliberately separate emergency path for when it is not: an identity-provider outage, a lockout during active incident response, or a control-plane misconfiguration that needs immediate reversal. A break-glass credential is defined by what it is not: not a routine admin account, not something used for convenience, and not exempt from monitoring. It should be vaulted with restricted, logged access; invoked only under dual control so no single person can exercise it alone; monitored with session recording and immediate post-use review; and rotated or reissued after every use so a stale emergency credential never becomes a standing one ([NHI Management Group, break-glass account glossary](#)).

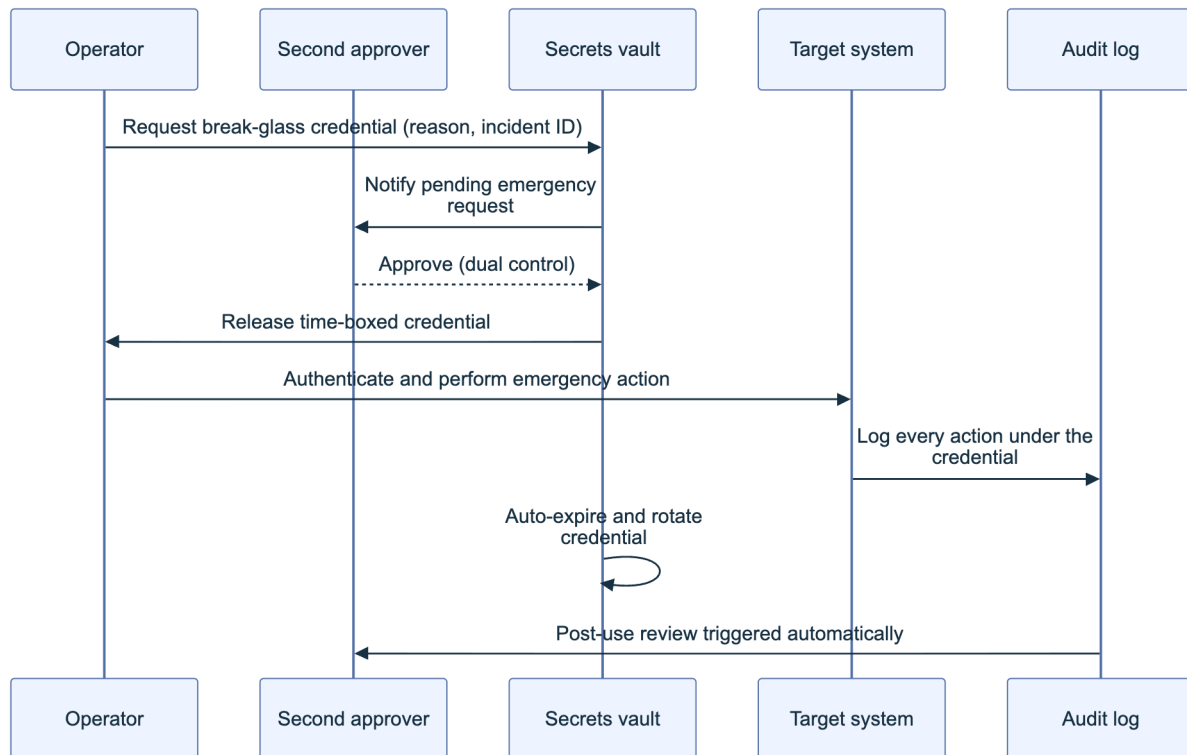


Figure 1. A break-glass request cycle. Dual control gates issuance, the credential is time-boxed and single-use, and rotation plus a mandatory post-use review close the loop so the emergency path never becomes a quiet permanent one.

Test the break-glass path on a schedule, not only when an incident forces its first real use. An emergency account nobody has exercised since it was created is a hypothesis about how the process will behave under pressure, not a verified control, the same logic Part 1 applied to untested failover providers.

13.3 Service Accounts and Machine Identity

Machine identities, service accounts, API tokens, CI/CD runners, and increasingly autonomous agents, now vastly outnumber human accounts on a typical infrastructure estate. CyberArk's 2025 State of Machine Identity Security report, drawn from 2,600 security decision-makers, found non-human identities outnumbering human ones by roughly 82 to 1, with nearly half carrying sensitive or privileged access, and 68% of respondents reporting no identity security controls covering their AI-driven identities at all ([CyberArk, 2025 State of Machine Identity Security Report](#)). Every RPC node's monitoring agent, every Terraform CI job, every webhook relay is a credential-bearing identity, and most organizations track a fraction of them as rigorously as they track a human employee's account.

Okta's own October 2023 support-system breach is the reference incident for how service accounts fail in practice, and the root cause was mundane rather than exotic: an employee signed into a personal Google account on a browser also used for work, the browser's saved-credential feature retained a service account's login for Okta's support case system, and that personal

account was compromised, exposing the service account. The attacker used it to read support cases for 134 customers over roughly three weeks before detection, including session tokens later used to hijack five customers' own sessions ([Okta Security, root cause of the October 2023 breach](#)). Okta's remediation is the checklist: disable the account, block personal-profile sign-in on managed browsers entirely, add network-location binding so a session used from a new network forces re-authentication, and treat the incident as evidence that a service account with an interactive login path is a service account with a human's attack surface attached to it.

The structural fix is to remove long-lived, storable credentials from machine identities wherever the workload's host platform supports it. AWS, GCP, and Azure all support workload identity federation: a CI/CD job authenticates to its own platform's OpenID Connect (OIDC) provider, presents that token to the cloud provider, and receives a short-lived, narrowly scoped credential with nothing to leak once the job ends ([AWS IAM security best practices](#); [Latacora, OIDC workload identity on AWS](#)).

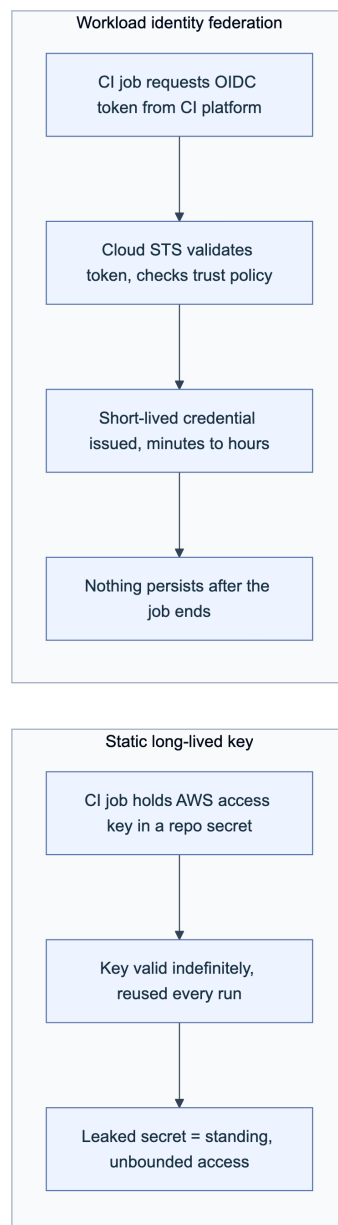


Figure 2. A stored static key is a standing liability the moment it is leaked; a federated identity issues a credential that is worthless once the job that requested it finishes.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Federated": "arn:aws:iam::123456789012:oidc-provider/token.actions.githubusercontent.com" },
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": { "token.actions.githubusercontent.com:aud": "sts.amazonaws.com" },
      "StringLike": { "token.actions.githubusercontent.com:sub": "repo:org/infra-repo:ref:refs/heads/main" }
    }
  }]
}
```

Where federation is not an option, the fallback discipline still holds: one credential per service, never shared across two workloads; secrets stored in a vault (HashiCorp Vault, AWS Secrets Manager, or equivalent) rather than in configuration files or environment variables checked into a repository; automated rotation on a defined interval; and an inventory (Part 1, Chapter 2) that tracks every service account's owner and purpose with the same rigor applied to human accounts, because an orphaned service account is exactly the unowned-asset finding Part 1 warned about.

13.4 Multi-Factor and Hardware-Backed Auth for Infra

Not all multi-factor authentication (MFA) provides the same protection, and the infrastructure control plane is exactly where that difference matters most. NIST SP 800-63B defines three Authenticator Assurance Levels (AAL); AAL3, the level appropriate for infrastructure administrators and anyone with signing-key adjacent access, requires proof of possession of a hardware-based authenticator plus resistance to verifier impersonation, meaning the authenticator itself refuses to complete an authentication ceremony against a phishing site impersonating the real one ([NIST SP 800-63B](#)). SMS one-time codes and time-based one-time password (TOTP) apps satisfy only AAL2 at best, and both remain phishable and, in the SMS case, vulnerable to SIM-swap takeover of the phone number itself.

FIDO2 and its browser-facing WebAuthn API are the practical implementation of AAL3-grade authentication: a hardware security key or platform authenticator holds a private key that never leaves the device, and the cryptographic challenge-response is bound to the exact origin requesting it, which is what makes it phishing-resistant where a code typed into any page is not ([FIDO Alliance, FIDO2](#); [W3C Web Authentication Level 3](#)). Uber's September 2022 breach is the clearest illustration of why phishable MFA fails under determined pressure: an attacker linked to the Lapsus\$ group compromised a contractor's password, then triggered a flood of MFA push notifications for over an hour until the contractor, exhausted and contacted over WhatsApp by someone posing as Uber IT, approved one, handing over VPN and internal Slack access within minutes

([Dark Reading](#), [Uber MFA bombing attack](#)). A push-approval prompt is defeatable by persistence and social pressure; a hardware key bound to the wrong origin simply does not produce a valid assertion, regardless of how many times an attacker asks.



Figure. A FIDO2 USB security key. The private key is generated on and never leaves this device, and the origin-bound challenge-response is what a push notification or a typed one-time code cannot replicate. Source: [Wikimedia Commons](#), CC BY-SA 4.0.

AAL	Authenticator examples	Phishing resistance	Recommended for
AAL1	Password only	None	Never, for infrastructure access
AAL2	Password + TOTP or SMS code	Low (code is phishable, SIM-swap risk)	Low-sensitivity internal tools only
AAL2 (push)	Password + push approval	Moderate (defeatable via MFA fatigue)	Not for privileged infra roles
AAL3	Hardware security key (FIDO2/WebAuthn), platform authenticator with hardware-backed key storage	High (origin-bound, unphishable)	Cloud console root/admin, DNS registrar, node SSH, break-glass

Enrolling and verifying a hardware authenticator is a routine operational task, not a one-time setup. With a Yubico security key, for example:

```
# Confirm the key is FIDO2-capable and check enrolled credential count
ykman fido info

# List discoverable (resident) credentials bound to this key
ykman fido credentials list

# Require a PIN before the key will produce an assertion, so a
# physically stolen key alone is insufficient
ykman fido access change-pin
```

Require AAL3 for every identity in the Critical tier from Part 1's sensitivity classification, cloud root and IAM administrators, DNS registrar accounts, and anyone who can reach a node's SSH or signing surface, and register a second physical key per person as a recovery path so a lost key does not force a break-glass event of its own.

13.5 Offboarding and Access Revocation

Access granted is easy to track; access revoked is where most IAM programs quietly fail. NIST SP 800-53's PS-4 (Personnel Termination) control requires an organization to disable system access within an organization-defined time period of a termination, and its supplemental guidance singles out for-cause terminations as needing the fastest possible execution, because the interval between notice and revocation is exactly the window a departing insider can act in ([NIST SP 800-53 r5, PS-4](#)). The US Cybersecurity and Infrastructure Security Agency frames the same requirement from the insider-risk side: departing personnel, including those leaving on good terms, are a recurring and preventable category of exposure precisely because offboarding is treated as an HR formality rather than a security control ([CISA, insider threat mitigation](#)).

Coinbase's May 2025 breach shows the offboarding problem extending past direct employees to contracted third parties. Attackers bribed customer-support agents at an outsourced vendor to exfiltrate personal data on roughly 70,000 customers, then demanded a \$20 million ransom. Coinbase publicly refused to pay; the company's response included terminating the implicated support contracts and, per subsequent litigation, cutting off hundreds of accounts at one vendor facility once the scale of insider collusion became clear ([Coinbase, protecting our customers](#); [CNBC, Coinbase bribed staff breach](#)). A vendor's staff churn is not visible to the org that granted them access unless offboarding is wired to that vendor's own personnel changes, not just your own HR system, which is exactly the kind of dependency the Employee Lifecycle Security Handbook (03) treats as first-class.

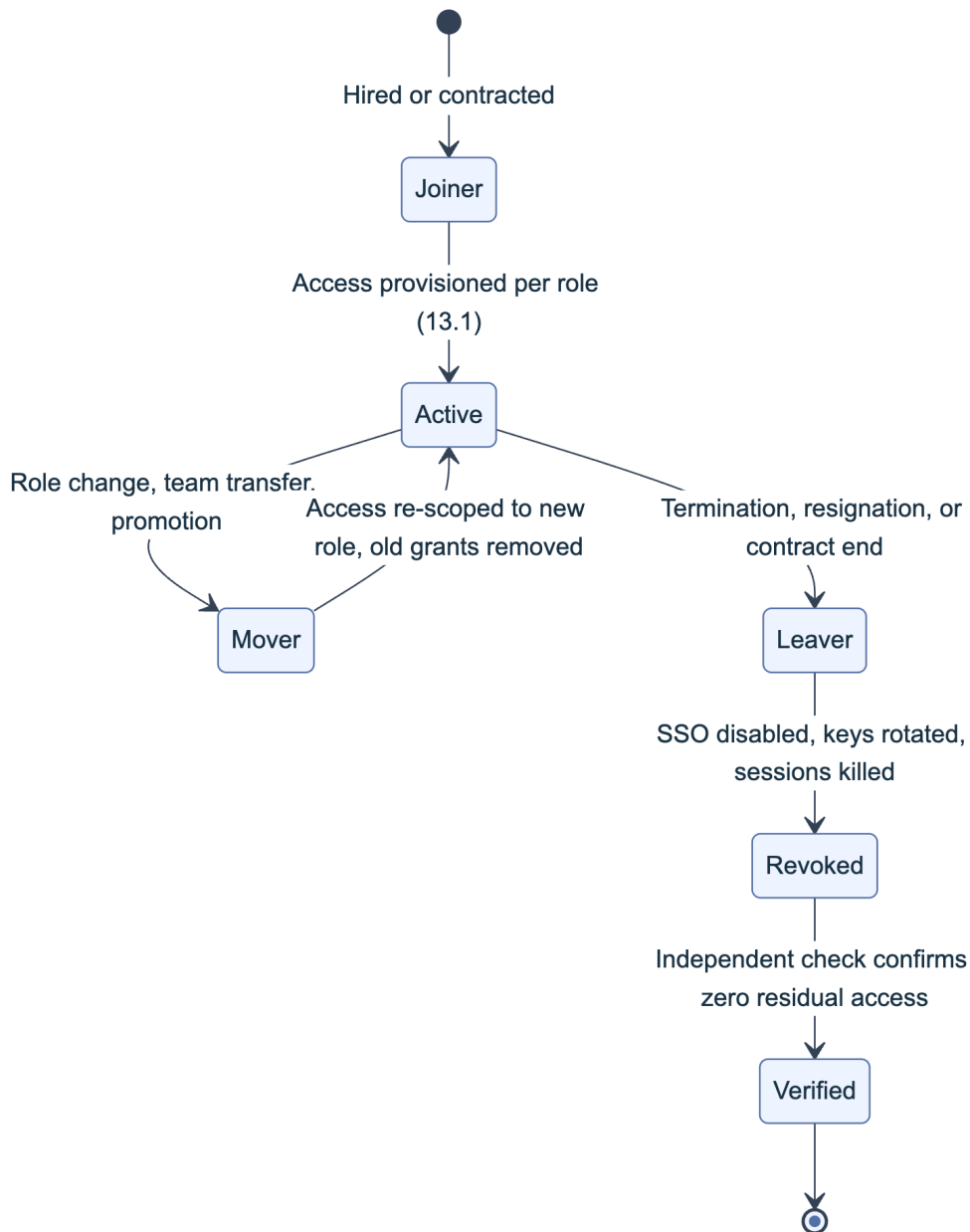


Figure 3. The joiner-mover-leaver lifecycle. A Mover event that skips re-scoping (old grants left in place alongside new ones) is as dangerous as a Leaver event that skips revocation.

Build revocation as an automated, verified workflow rather than a manual checklist alone: disabling the identity provider account should cascade to every downstream system that trusts single sign-on (SSO), but cloud IAM users, SSH keys, API tokens, and any local accounts created outside SSO's reach need their own explicit teardown step. A same-day revocation service-level agreement (SLA) for Critical-tier access (Part 1, Section 2.2), independently verified rather than self-reported by the offboarding system, closes the gap where a departed identity's access is assumed disabled but never confirmed.

Offboarding checklist (Critical and Sensitive tier assets):

- ☐ SSO account disabled and all active sessions revoked, not merely the password reset

- ☐ Cloud IAM users and roles tied to the identity removed or reassigned to a named owner
 - ☐ SSH keys, API tokens, and any hardware MFA registrations tied to the identity revoked
 - ☐ Break-glass and shared-account passwords rotated if the departing identity had knowledge of them
 - ☐ Vendor and contractor offboarding triggers the same workflow when a third party's staff changes
 - ☐ Independent verification (not the offboarding system's own log) confirms zero residual access within the SLA window
 - ☐ Offboarding event recorded against the asset inventory (Part 1, Section 2.4) so ownership records stay current
-

14. Integration with IAM Framework

This chapter draws the boundary between the infrastructure-facing controls in Chapter 13 and the organization's broader identity program, so neither is mistaken for covering the other.

14.1 Cross-Reference to Dedicated IAM Guidance

Chapter 13 covers IAM as it applies to infrastructure specifically: roles scoped to cloud resources and nodes, break-glass paths into production systems, machine identities, and the hardware authentication and revocation mechanics that protect them. It does not cover the organizational program that decides who should be hired, what their role entitles them to, or how a termination decision gets made in the first place, that is a human-resources and governance process, not an infrastructure control, and it belongs to the [Employee Lifecycle Security Handbook \(03\)](#), which owns hiring, onboarding, role-change governance, and termination policy end to end. Treat this part as the technical enforcement layer that a Handbook 03 policy decision triggers: when Handbook 03's process determines someone is leaving, Section 13.5 here is what actually disables their access.

Two further references round out the picture. The [OWASP Access Control Cheat Sheet](#) and [Authentication Cheat Sheet](#) give application-layer guidance for the identity and access decisions inside a dApp or admin panel's own code, distinct from the infrastructure-layer decisions covered here. The full NIST SP 800-63 Digital Identity Guidelines suite (63A for identity proofing, 63B for authentication referenced in Section 13.4, 63C for federation) is the canonical normative reference if your organization is building or procuring an identity provider rather than only configuring one.

Topic	Owned by
Hiring, onboarding, role-change policy, termination decisions	Employee Lifecycle Security Handbook (03)
Cloud/node RBAC, break-glass, service accounts, hardware MFA, technical revocation	This handbook, Part 5
Application-layer access control and authentication inside your own code	OWASP Access Control / Authentication Cheat Sheets
Identity provider architecture, identity proofing, federation protocols	NIST SP 800-63 A/B/C
DNS registrar and domain-account access specifically	DNS and Hosting Security Handbook (02)
Response process once an IAM control fails (compromised admin, leaked key)	Incident Response Handbook (06)

14.2 Infrastructure-Specific IAM Checklist

The consolidated checklist in Part VI folds this section into the handbook's full operational checklist set; the table below is the infrastructure-IAM subset on its own, sized for a standalone quarterly review.

Control	Verification method	Target cadence
Every infra role maps to a documented job function, no per-person roles	Export role list, flag any role with exactly one assignee	Quarterly
ABAC conditions (resource tags, MFA freshness) attached to Critical-tier role grants	Policy review against Part 1's asset sensitivity tiers	Quarterly
No standing privileged grants outside JIT/PAM workflow	Query IAM for permanent admin-equivalent assignments	Monthly
Break-glass credential vaulted, dual-control gated, tested	Run a live break-glass drill and confirm auto-rotation fires	Quarterly
Service account inventory matches actual credential-bearing identities	Reconcile IaC state and cloud API against Part 1's inventory	Monthly
Workload identity federation used wherever the platform supports it	Grep CI/CD configs for static cloud credentials	Quarterly
AAL3 (hardware-backed, phishing-resistant) MFA enforced for Critical-tier identities	Pull MFA device type report from identity provider	Quarterly
Offboarding SLA met with independent verification, including vendor staff	Sample terminated identities, confirm zero residual access	Monthly

Key controls for Part 5

- Model access with RBAC for stable job functions and layer ABAC conditions (resource tags, MFA freshness) on top for context-sensitive grants; review role definitions on a fixed cadence to catch role sprawl.
- Replace standing privileged access with just-in-time grants and zero standing privileges; reserve a vaulted, dual-control, auto-rotating break-glass path for when the normal access path itself is down, and drill it before an incident forces first use.
- Inventory every machine identity with the same rigor as a human account; prefer workload identity federation (OIDC to short-lived cloud credentials) over any stored long-lived key or token.
- Require AAL3 hardware-backed, phishing-resistant authentication (FIDO2/WebAuthn) for every Critical-tier identity; push-approval and SMS/TOTP MFA are defeatable by fatigue attacks and SIM swaps respectively.

- Treat offboarding as an automated, independently verified workflow with a same-day SLA for Critical and Sensitive tiers, extended to vendor and contractor staff changes, not a checklist that runs on trust.

Part VI: Checklists and Operations

Part 6 turns the domain-by-domain controls spread across Parts II through V into the artifacts a team actually runs day to day: a lifecycle-ordered checklist covering pre-deployment through incident readiness, a hardened infrastructure as code (IaC) pipeline, and a monitoring and logging program that spans the cloud layer and the chain itself. Earlier parts explain why a control matters; this part specifies when to apply it, what tool or process enforces it, and what evidence proves it happened. Read the three chapters that follow as the last mile between the standards cited throughout this handbook and a runbook an on-call engineer can execute without guessing.

15. Node Infrastructure Security Checklist (Consolidated)

This chapter pulls the hardening controls scattered across Part II (node and compute infrastructure), Part III (network and availability), Part IV (DNS and domain security), and Part V (identity and access management) into a single lifecycle: what must already be true before an asset takes production traffic, what must stay true while it runs, and what must be pre-staged before something goes wrong.

15.1 Pre-Deployment

A pre-deployment gate exists because “we will fix that after launch” is how every category of control in this handbook quietly never gets applied. The gate is a fixed list of conditions that must already be true, verified with evidence rather than a verbal confirmation, before a remote procedure call (RPC) node, an archive node, a cloud account, or any supporting service is allowed to receive real user or protocol traffic. This mirrors the discipline behind [CIS Control 4, Secure Configuration of Enterprise Assets and Software](#), which requires a documented secure baseline applied before, not after, a system joins the fleet. In a Web3 context the gate has to span infrastructure and chain-facing surfaces at once: a node with a hardened operating system (OS) sitting behind an open admin port is still an open node, and a domain with a perfect firewall but no [DNS Security Extensions \(DNSSEC\)](#) is still hijackable, as the [Curve Finance DNS hijack of August 2022](#) (Part I, §1.1) showed. Treat every row below as blocking: the asset does not go live, and does not receive a DNS cutover, until every applicable row has evidence attached in the asset inventory record built in Part I, §2.

Domain	Pre-deployment control	Confirms
Compute / OS	Hardened baseline image applied, unused services and ports disabled	No default-open attack surface (Part II, §5)
Node-specific	RPC/archive admin and debug interfaces bound to a private network only, rate limiting configured	Node cannot be abused as an open relay (Part II, §3)
Network	Default-deny security groups and firewall rules, node tier segmented from management tier	No flat network an attacker can pivot across (Part III, §6)
Availability	Distributed denial-of-service (DDoS) protection or scrubbing enabled before DNS cutover	Launch traffic spike cannot be weaponized (Part III, §7)
DNS	Registrar transfer lock, DNSSEC signed, Certification Authority Authorization (CAA) record set	Domain is not hijackable at launch (Part IV, §9-10)
Email	SPF, DKIM, and DMARC published before the domain sends its first message	Domain cannot be spoofed for phishing (Part IV, §11)

Domain	Pre-deployment control	Confirms
Identity and access management (IAM)	Least-privilege deploy role, hardware-backed multi-factor authentication (MFA) on every human account, no long-lived static keys	No single stolen credential grants broad access (Part V, §13)
Secrets	No plaintext secret in source, IaC state, or environment variables; secret manager wired in	Secrets cannot leak through the deploy path (§16.3)
Monitoring	Log shipping and on-chain monitor rules active before the asset takes real traffic	Nothing runs blind, even for hour one (§17)
Inventory	Asset registered with owner, purpose, and sensitivity tier	Incident response can find and triage it in seconds (Part I, §2.2)

15.2 Ongoing Operations

Passing the pre-deployment gate proves a point-in-time state, not a durable one. Configurations drift, dependencies age, credentials outlive their need, and a control that was correct on day one silently stops being correct as the system around it changes; that decay is exactly what the reconciliation step in Part I, §2.3 exists to catch at the inventory level, and this checklist applies the same discipline to the running fleet. Patch cadence should be tied to a concrete external signal rather than a vague “regularly”: the [CISA Known Exploited Vulnerabilities \(KEV\) catalog](#), created under [Binding Operational Directive 22-01](#) (November 2021), lists vulnerabilities under active exploitation with a mandated remediation deadline for federal agencies; adopting the same catalog as a private patch-priority signal, with a committed internal SLA (for example, 72 hours for a KEV-listed vulnerability on a critical-tier asset), converts “patch when convenient” into a measurable control.

Activity	Cadence	Owning process
KEV-listed and critical CVE patching	Within committed SLA of publication (e.g. 72 hours, critical tier)	On-call / platform
Credential and API key rotation	90 days, or immediately on role change	IAM (Part V, §13)
Access review (who can reach what)	Quarterly	IAM (Part V, §14)
Node client and dependency updates	Monthly, or on upstream security release	Protocol / infra engineering
Backup and restore test (not just backup)	Quarterly	SRE
DDoS and failover drill	Semiannually	SRE / network
Configuration drift check	Continuous (CI-triggered) or daily	Platform (§16.2)
Log and alert review for false-negative gaps	Monthly	Security

The failure mode this checklist exists to prevent is a control that was true once being assumed true forever. A quarterly access review that finds zero stale grants three quarters running is worth re-scoping, not skipping; a review that has never been run is a finding in itself.

15.3 Incident Readiness

Readiness is what separates a contained incident from a prolonged one, and the record in this handbook is unambiguous about the stakes: the [BadgerDAO front-end compromise of December 2021](#) (Part I, §1.1) ran for roughly ten hours before it was fully contained, and in an irreversible-transaction environment, response speed during that window determines the loss far more than the initial vulnerability does. Readiness means the actions a responder needs are written down, rehearsed, and pre-authorized before the incident starts, not improvised during it. A runbook drafted while an attacker is actively draining wallets is a runbook drafted too late.

Pre-stage the following, and validate each with a tabletop exercise at least twice a year:

- **Break-glass access:** a documented, logged, time-boxed emergency access path to the DNS registrar, the cloud root account, and the CDN, usable when the normal on-call path is itself compromised.
 - **Pre-authorized emergency actions:** explicit, board- or lead-approved authority for specific responders to pause a contract, rotate a DNS record, revoke an API key, or fail over to a secondary RPC provider, without waiting for a synchronous approval chain during the event.
 - **Communication tree:** a named, redundant escalation path (not a single Slack channel) covering engineering, legal, communications, and executive leadership, with phone numbers that do not depend on the compromised system to reach.
 - **Configuration backups:** an out-of-band, offline copy of DNS zone files, IAM policies, and firewall rules, so recovery does not depend on reading configuration from the very system that may be compromised.
 - **Cross-handbook trigger:** a documented handoff point into the [Incident Response Handbook \(06\)](#), so the person who first notices an anomaly knows exactly which playbook activates and who owns it from that moment on.
-

16. Infrastructure as Code (IaC) Security

Provisioning cloud and node infrastructure through code, rather than console clicks, turns every infrastructure change into something that can be reviewed, tested, and rolled back like application code. It also turns the repository, the pipeline, and the state file into a new, high-value attack surface that a purely “click-ops” shop never had to defend.

16.1 Secure IaC Practices

Terraform, Pulumi, and AWS CloudFormation share a common risk profile: the code that describes infrastructure carries the same authority as the infrastructure itself, so a compromised pipeline or a careless module reference can provision (or destroy) production resources without ever touching a cloud console. Pin every module and provider to an exact, reviewed version rather than a floating reference; a `terraform init` that silently resolves to a newer, unreviewed module version reintroduces the exact “trust whatever the source currently serves” failure mode covered for CDN content in Handbook 01. Source modules only from a vetted registry or an internal module library, never from an unpinned Git branch.

Grant the CI/CD runner that applies changes the narrowest role that can do its job, and issue that role through short-lived, federated credentials, [OpenID Connect \(OIDC\) federation into the cloud provider](#), rather than a long-lived static access key stored as a pipeline secret. Gate every `apply` behind an automated policy-as-code check in addition to human review: [Checkov](#) (Bridgecrew/Prisma Cloud) and [Trivy’s IaC scanning](#), which absorbed the [tfsec check library in 2024](#), cover Terraform, CloudFormation, and Kubernetes manifests out of the box, while [Open Policy Agent \(OPA\)](#) with [Conftest](#), [HashiCorp Sentinel](#), and [Pulumi CrossGuard](#) let a team encode organization-specific rules a generic scanner cannot know, such as “no security group may allow ingress from `0.0.0.0/0` on the node’s peer-to-peer port.”

```
# Fail the pipeline on a HIGH or CRITICAL misconfiguration before any apply runs
trivy config --severity HIGH,CRITICAL --exit-code 1 ./infra/

# Same intent, custom organizational policy expressed in Rego for Conftest
conftest test --policy ./policy/no-open-ingress.rego ./infra/plan.json
```

```
# policy/no-open-ingress.rego
package main

deny[msg] {
  resource := input.resource_changes[_]
  resource.type == "aws_security_group_rule"
  resource.change.after.cidr_blocks[_] == "0.0.0.0/0"
  resource.change.after.from_port <= 30303
  resource.change.after.to_port >= 30303
  msg := sprintf("security group rule %s opens the node P2P port to the internet",
    [resource.address])
}
```

16.2 Review and Drift Detection

Review catches a bad change before it ships; drift detection catches a change that never went through review at all. Drift is any gap between the infrastructure a team's IaC declares and the infrastructure actually running, and it accumulates from console fixes applied during an incident, a second automation tool touching the same resources, or a manual change nobody backported into the repository. Left undetected, drift means the code in version control, the artifact everyone reviews and audits, stops describing reality, and every subsequent `plan` operates on a false premise.

The stakes of unmanaged configuration inconsistency across a fleet are not hypothetical, even outside Web3. On August 1, 2012, [Knight Capital Group deployed new trading code to seven of eight production servers and left obsolete code active on the eighth](#); a repurposed order flag that was safe against the new code triggered the dormant legacy function on that one inconsistent server, generating four million erroneous executions across 154 stocks in roughly 45 minutes and a pretax loss of \$440 million. No single server was misconfigured in isolation; the fleet was inconsistent with itself, and nothing detected the gap before it was live. Drift detection is the automated version of the consistency check Knight's deployment process lacked, running continuously rather than once at deploy time.

Enforce review with a mandatory pull request gate that attaches the `plan` output (the exact diff an `apply` will make) for human sign-off before merge, never allowing `apply` to run against production from an unreviewed branch. For drift, [driftctl](#), maintained by Snyk and now in maintenance mode (contributions are no longer actively reviewed, and the maintainers direct users to fork the project for new work), pioneered a purpose-built comparison between live cloud state and Terraform state; Terraform Cloud and Enterprise ship the equivalent as a built-in scheduled health check, and cloud-native inventories such as AWS Config, GCP Cloud Asset Inventory, and Azure Resource Graph (Part I, §2.3) can be queried on the same schedule as a lower-maintenance alternative. Run whichever tool fits on a fixed cadence, alert on any unexplained delta, and require every delta to resolve either by updating the code to match reality or by reverting the live resource, never by silently accepting the gap.

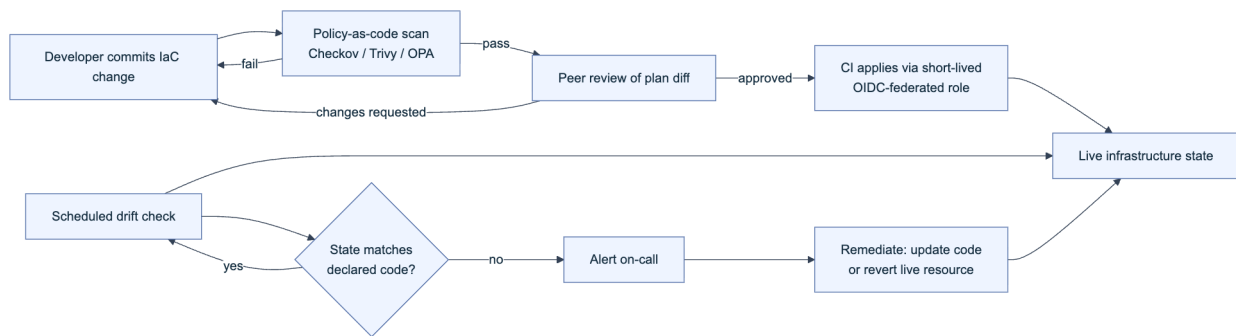


Figure 1. The IaC review and drift loop. A change only reaches production through the reviewed path on the left; the scheduled loop on the right catches anything that reached production another way.

16.3 Secrets in IaC (What to Avoid)

Two properties make secrets inside IaC uniquely dangerous compared to secrets inside application code: the values are often the highest-privilege credentials an organization holds (cloud root keys, database master passwords, node signing material), and the state file that Terraform and similar tools generate can capture those values in plaintext even when the source code never references them directly, because the state file is a snapshot of every resource attribute, sensitive or not. A `.tfstate` committed to version control, left in an improperly secured storage bucket, or exposed through verbose CI logging is a plaintext credential dump regardless of how carefully the `.tf` source itself was written.

Two real incidents show both ends of this failure. In the [Uber breach disclosed in November 2017](#), attackers obtained access to a third-party cloud storage service holding data for 57 million users and 600,000 US driver's license numbers; the credentials that enabled that access had been left reachable in a private code repository, the same pattern that turns a committed `.tf` file or state snapshot into a breach vector today. In the [Codecov Bash Uploader compromise \(January 31 to April 1, 2021\)](#), an attacker who obtained one Google Cloud Storage credential modified a widely used CI script to exfiltrate every environment variable, tokens, keys, and credentials among them, from more than 23,000 customers' build pipelines. Neither incident involved Terraform specifically, but both are the exact failure mode this section exists to prevent: a credential that a pipeline had no reason to hold in cleartext, or held for longer than it needed to, was harvested wholesale.

```
# AVOID: literal secret in source, and it also lands in state in cleartext
resource "aws_db_instance" "rpc_index" {
  password = "Sup3rSecretDBPass!"
}

# PREFER: reference a secret manager; the value never appears in .tf source,
# and access to it is independently logged and revocable
data "aws_secretsmanager_secret_version" "db_pass" {
  secret_id = "prod/rpc-index/db-password"
}
resource "aws_db_instance" "rpc_index" {
  password = data.aws_secretsmanager_secret_version.db_pass.secret_string
}
```

Encrypt remote state at rest, restrict its read access as tightly as the secrets it may contain (a state reader can read cleartext secrets even without touching the secret manager directly), and run a secret scanner such as [Gitleaks](#) or [TruffleHog](#) in the same CI gate as the policy-as-code checks from §16.1. The [OWASP Top 10 CI/CD Security Risks](#) names this pattern directly as CICD-SEC-6, Insufficient Credential Hygiene, and pairs it with CICD-SEC-7, Insecure System Configuration, both of which a pinned, scanned, least-privilege IaC pipeline is built to close.

17. Monitoring and Logging

A control that is not observed is a control you are hoping works. This chapter covers what to capture, how to watch the chain itself in addition to the infrastructure underneath it, and how to route what is captured to a human fast enough to matter.

17.1 What to Log and Retain

Logging strategy for a Web3 project spans two categories that rarely appear in the same framework: conventional infrastructure telemetry, and the on-chain signal covered separately in §17.2. For the infrastructure half, prioritize sources by their value to a hunt or an investigation rather than by how easy they are to collect: cloud control-plane logs (AWS CloudTrail, Azure Activity Log, GCP Audit Logs) record every API call against the account itself; identity and credential events record every authentication, MFA challenge, and privilege change; network flow logs and DNS query logs record what talked to what; and CI/CD pipeline logs, named explicitly as a gap by [OWASP CICD-SEC-10, Insufficient Logging and Visibility](#), record who deployed what and when.

[NIST SP 800-92, Guide to Computer Security Log Management](#), sets the baseline architecture: centralize collection, protect log integrity, and size retention to the investigative window an incident realistically needs, not to storage cost convenience. [CIS Control 8, Audit Log Management](#), operationalizes the same principle as a named control in the CIS Controls v8.1 framework. For a concrete retention benchmark, [OMB Memorandum M-21-31](#) (August 2021) assigns US fed-

eral agencies a four-tier logging maturity model, ELO through EL3, and sets a floor of 12 months in active storage plus 18 months in cold storage for its highest-value (“Criticality Level 0”) log categories, which include identity and credential management events and DNS query logs. That floor is a useful benchmark to borrow even outside federal compliance scope: align retention length to the sensitivity tier built in Part I, §2.2 rather than applying one blanket window to every log source.

Log source	Capture	Suggested retention (critical-tier asset)
Cloud control-plane / API calls	Every action, actor, source IP, timestamp	12 months active, 18 months cold
IAM and credential events	Auth attempts, MFA challenges, privilege changes	12 months active, 18 months cold
DNS query and zone-transfer logs	Query, response, source, zone changes	12 months active, 18 months cold
Network flow / firewall / web application firewall (WAF)	Connections, blocks, rule hits	90 days active, 1 year cold
RPC/node access and error logs	Request rate, error codes, peer connections	90 days active, 1 year cold
CI/CD pipeline logs	Deploy actor, artifact hash, approvals	90 days active, 1 year cold
On-chain monitor alerts (§17.2)	Alert payload, decoded event, response taken	Indefinite (append-only)

17.2 On-Chain Monitoring Objectives (Large Transfers, Ownership, Anomalies)

Infrastructure logging answers “did anything unusual touch our servers and accounts.” It cannot answer “did anything unusual happen to our funds,” because the chain is a second, independent surface with its own signal that infrastructure telemetry never sees. Watch for large or unusual value transfers against a rolling baseline rather than a single hardcoded threshold; ownership and access-control events, `OwnershipTransferred`, `RoleGranted`, and any call into an admin-only function, which should never fire outside a known, scheduled maintenance window; pause and upgrade events, since a proxy implementation change is functionally a code deployment and deserves the same review evidence as one; and anomaly signals such as oracle price deviation beyond a sane band or gas usage that spikes far outside a contract’s historical pattern.

Purpose-built tooling exists for exactly this surface. [Forta](#) runs a network of detection bots as an on-chain firewall, screening and blocking malicious transactions with a stated recall above 99% and a false-positive rate below 0.0002% by combining static rules with transaction simulation. [OpenZeppelin Defender](#) provided this capability through a hosted platform for several years, but new sign-ups closed on June 30, 2025, and OpenZeppelin is migrating the platform’s monitoring functionality to the open-source [OpenZeppelin Monitor](#) and its transaction-relay functionality to

[OpenZeppelin Relayer](#), with the hosted service remaining operational through mid-2026 to give teams a migration window. [Tenderly's Alerting and Monitor products](#) cover the same event classes, decoded transactions, state changes, token transfers, and custom invariants such as health-factor or oracle-deviation thresholds, and route them into Slack, PagerDuty, Discord, and webhooks within seconds of the triggering block.

```
// Illustrative monitor rule (Forta/OpenZeppelin Monitor style): alert on any
// ownership transfer of a critical-tier contract outside the change window
function handleTransaction(txEvent) {
  const transferEvents = txEvent.filterLog(OWNERSHIP_TRANSFERRED_ABI, CRITICAL_CONTRACT_ADDRESS);
  if (transferEvents.length > 0 && !isWithinApprovedChangeWindow()) {
    return Finding.fromObject({
      name: "Unscheduled ownership transfer",
      severity: FindingSeverity.Critical,
      metadata: { contract: CRITICAL_CONTRACT_ADDRESS, newOwner: transferEvents[0].args.newOwner },
    });
  }
}
```

Map monitor coverage directly to the asset inventory's sensitivity tiers from Part I, §2.2: every critical-tier contract's admin functions get an explicit rule, not blanket "monitor everything" coverage that drowns the signal a responder actually needs in noise.

17.3 Alert Thresholds and Multiple Channels (Email, SMS, Messaging)

A threshold set too loosely misses the incident; a threshold set too tightly trains the on-call rotation to ignore alerts, which is functionally the same failure by a slower path. Prefer a baseline-relative threshold (alert on a transfer three standard deviations above the asset's 30-day rolling average) over a single static number wherever the underlying activity is naturally variable, and reserve static thresholds for events that should never occur at all, an unscheduled ownership transfer, a debug port opening, an admin login from a new country.

Severity should determine channel redundancy, not just urgency of tone. Email alone is inadequate for anything time-critical: it has no guaranteed delivery latency, no acknowledgment mechanism, and no escalation path if the recipient is asleep or offline. Route Critical and High severity through a paging tool such as PagerDuty or Opsgenie that supports acknowledgment and automatic re-escalation, in parallel with a team-visible channel (Slack, Discord, or Telegram) so the wider team has situational awareness without depending on one person's phone.

Severity	Primary channel	Response SLA	Escalation if unacknowledged
Critical (funds at risk)	Phone/SMS page + team chat	5 minutes	Auto-escalate to secondary on-call at 10 minutes, then lead at 20

Severity	Primary channel	Response SLA	Escalation if unacknowledged
High (control failure, no confirmed loss)	Page + team chat	15 minutes	Auto-escalate at 30 minutes
Medium (degraded, contained)	Team chat + ticket	4 hours	Escalate at next business-day standup
Low / informational	Ticket / dashboard only	Next business day	None

17.4 Centralized Logging and Alerting

Individually, each log source and each on-chain monitor answers a narrow question. Centralizing them into one queryable store is what lets a responder ask the question that actually matters during an incident: what changed across every layer, cloud, network, DNS, and chain, in the same ten-minute window. A security information and event management (SIEM) platform, or a lighter aggregation stack (Grafana Loki with Prometheus, an OpenSearch or Elastic cluster, or a managed option such as Datadog or Google Security Operations), is the mechanism; the discipline that matters is routing every source into it rather than leaving any one system as an island a responder has to remember to check separately.

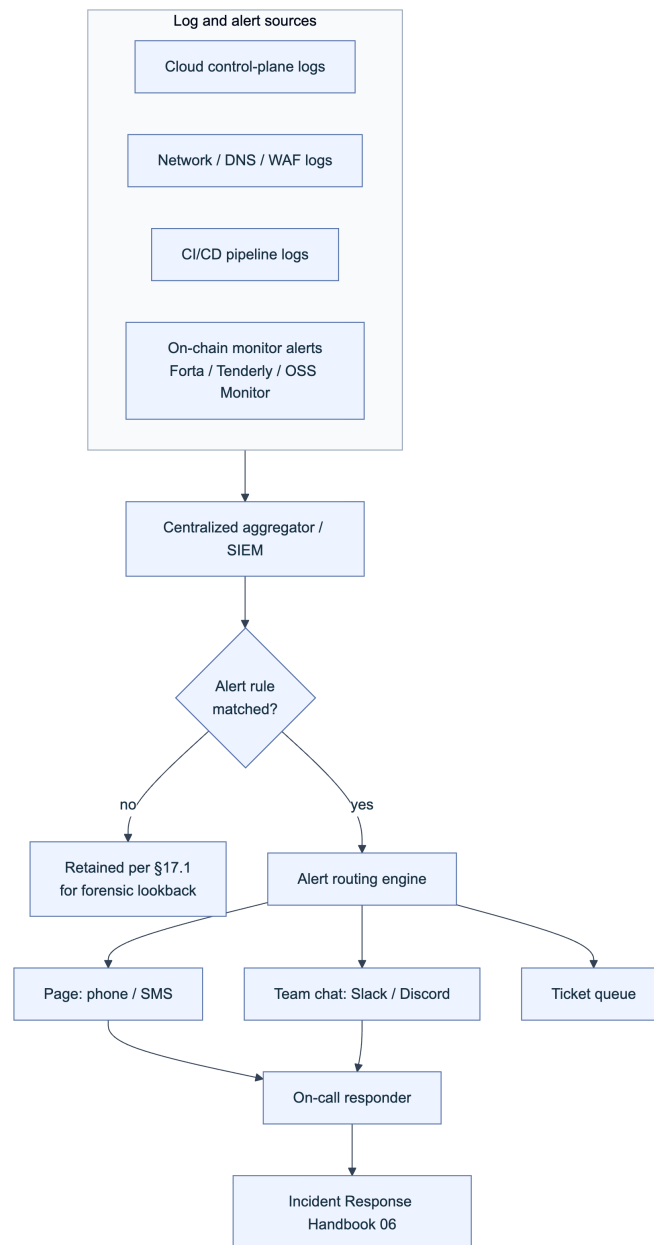


Figure 2. Centralized logging and alerting architecture. Every source, infrastructure and on-chain alike, feeds one aggregator; so triage never depends on a responder remembering which of five dashboards to open first.

Had the [BadgerDAO](#) front-end script injection been correlated automatically against wallet-approval spikes at the moment it started, rather than discovered downstream, the ten-hour exposure window (Part I, §1.1) had material room to shrink. Centralization does not by itself guarantee faster detection, but a fragmented log estate guarantees slower detection, because it requires a human to already suspect where to look before they can confirm anything.

17.5 Integration with Incident Response

Monitoring that does not page into a defined process is monitoring that produces evidence after the fact instead of a response during the event. Every alert severity tier from §17.3 should map to a specific playbook entry point in the [Incident Response Handbook \(06\)](#), so the person who re-

ceives the first page already knows which runbook applies rather than improvising a triage process live. On-call ownership should trace directly back to the named owner field on the asset inventory record (Part I, §2.2 and §2.4): the person paged for an anomaly on a given RPC node or contract should be the same person, or team, accountable for that asset's day-to-day operation, not a generic rotation with no context on what the asset does.

After the event, the centralized log store from §17.4 becomes the primary source for timeline reconstruction, which is why retention windows must be set to outlast the realistic detection-to-investigation lag, a log rotated out before a forensic team requests it is equivalent to a log that was never captured. Two operational habits keep this integration real rather than aspirational: freeze retention (suspend any automatic deletion) on logs relevant to an active investigation the moment it opens, and feed every post-incident finding back into §15.1's pre-deployment checklist and §17.2's monitor coverage, so a gap that caused one incident cannot cause an identical one a second time.

Key controls for Part 6

- Treat the pre-deployment checklist (§15.1) as a hard gate: an asset does not receive production traffic or a DNS cutover until every applicable row has evidence attached in the inventory record.
- Tie ongoing-operations cadences to external signals, not vague schedules: patch KEV-listed vulnerabilities within a committed SLA, rotate credentials on a fixed cycle, and test failover before an outage forces the test.
- Pre-authorize emergency actions (pause, DNS rotation, key revocation) and rehearse them; a runbook written during an active incident is a runbook written too late.
- Pin every IaC module and provider version, gate every `apply` behind policy-as-code and human review, and run scheduled drift detection so the code in version control never silently stops describing reality.
- Never let a secret reach `.tf` source or plaintext state; reference a secret manager, encrypt state at rest, and scan every commit for accidental exposure.
- Centralize infrastructure logs and on-chain monitor alerts into one aggregator, route by severity across multiple channels with real escalation, and wire every alert into the Incident Response Handbook (06) so detection converts into action without a manual handoff.

Part VII: Appendices

This part collects the reference material the rest of the handbook assumes is close at hand. Appendix A fixes the vocabulary so that “drift” or “break-glass” means the same thing whether you read it in the cloud chapter or the identity chapter. Appendix B turns the node guidance from Part II and Part VI into a printable checklist, and Appendix C does the same for the domain and DNS guidance in Part IV, both ready to paste into a runbook without re-deriving them from prose. Appendix D names representative tools and providers by category, and Appendix E anchors the two standards this handbook leans on hardest before pointing to the full bibliography.

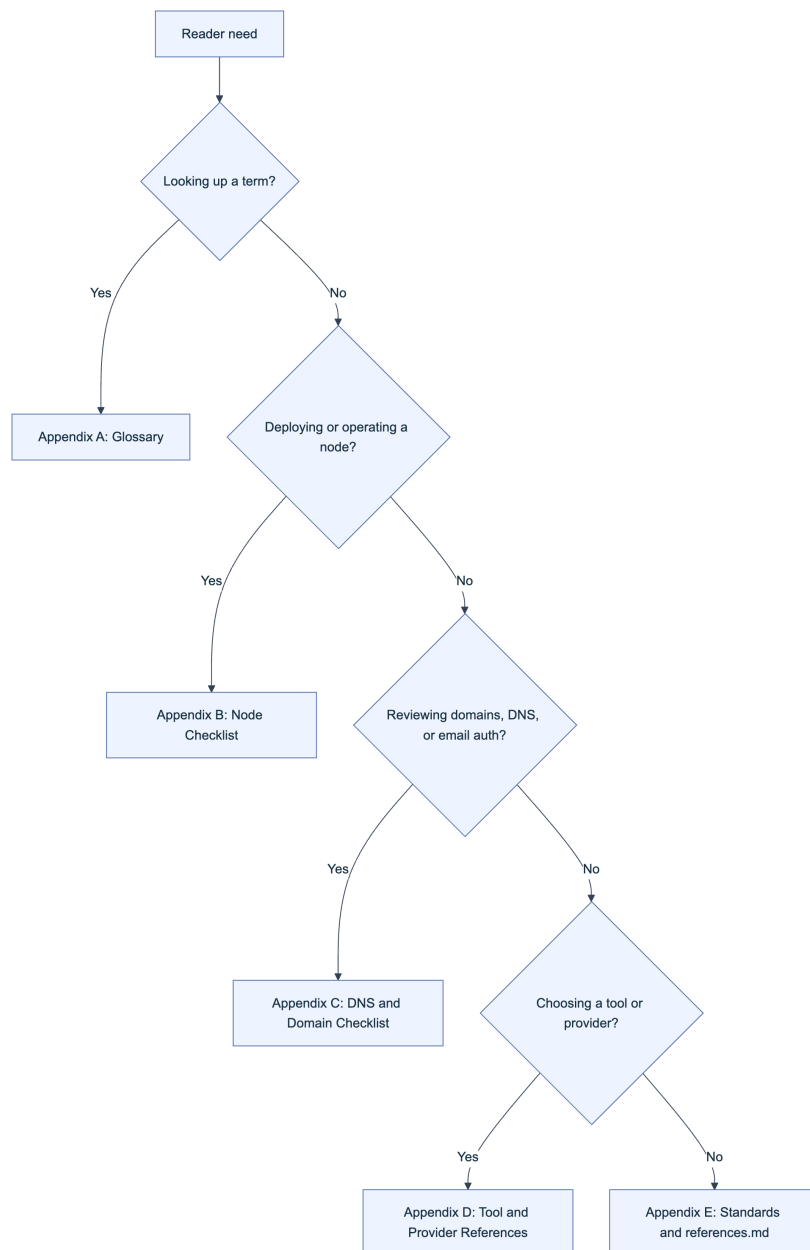


Figure 1. Appendix navigator. A single release or review often touches more than one branch: a validator deployment typically pulls B, then D for the secrets manager, then A for any unfamiliar term.

18. Appendix A: Glossary

Entries are grouped by the area of the handbook where they carry the most weight, not strict alphabetical order, because a reader troubleshooting a DNS incident and a reader hardening a validator reach for different clusters of vocabulary. Within each group, entries run alphabetically, acronyms are expanded on first appearance, and the relevant chapter is named so you can jump to the fuller treatment. Where an external standard or a documented incident defines the term, the entry cites that source directly rather than paraphrasing from memory.

18.1 Node, Compute, and Cloud Infrastructure Terms

These terms describe the machines and cloud accounts that run a protocol's own infrastructure, covered in Chapters 3 through 5.

- **Archive node:** a full node retaining a chain's complete historical state, not only recent blocks, needed for indexers and any query reaching past the client's default pruning window. See Chapter 3.1.
- **CMDB (Configuration Management Database):** a system of record listing every managed asset, its owner, and its configuration state, used to keep an asset inventory current. See Chapter 2.3.
- **Configuration drift:** gradual divergence of a running system's actual configuration from its approved, version-controlled baseline, typically from manual out-of-band changes. See Chapter 4.7.3.
- **CSPM (Cloud Security Posture Management):** automated tooling that continuously evaluates cloud account configuration against a policy baseline and flags deviations, the category [Prowler](#) belongs to. See Chapter 4.7.
- **IaC (Infrastructure as Code):** defining infrastructure through versioned, declarative configuration (Terraform, OpenTofu, CloudFormation, Pulumi) instead of manual console changes, so every change is reviewable and reproducible. See Chapter 4.7.1 and Chapter 16.
- **Least privilege:** granting an identity, human or machine, only the permissions its specific task requires, never a broader role "for convenience." See Chapter 4.6 and Chapter 13.
- **Policy as code:** expressing configuration and access rules as machine-evaluable policy, most commonly [Rego](#) for [Open Policy Agent \(OPA\)](#), so CI rejects a non-compliant change before merge. See Chapter 4.7.2.
- **RPC (Remote Procedure Call) node:** a node exposing an interface, typically JSON-RPC, letting applications query chain state and submit transactions without running their own node; public endpoints are a common single point of trust and DDoS target. See Chapter 3.1.
- **Secrets manager:** a dedicated system, a cloud provider's native service or a self-hosted tool such as [OpenBao](#) (the Linux Foundation-governed Vault fork created after Vault's 2023 move away from an OSI-approved license), that stores, versions, and rotates credentials and validator signing material outside source control. See Chapter 4.5.
- **Shared responsibility model:** the division of duties between a cloud provider, which secures the infrastructure "of" the cloud, and the customer, who secures what is configured "in" it, including identity and data. See Chapter 4.1.
- **Validator infrastructure:** nodes and signing setup participating in a proof-of-stake chain's consensus, distinguished from ordinary nodes by slashable stake and, usually, a hot signing key or remote signer. See Chapter 3.2.

18.2 Network, DDoS, and Zero-Trust Terms

These terms describe how traffic is segmented, filtered, and continuously re-verified, the subject of Chapters 6 through 8.

- **Assume breach:** the zero-trust posture that treats an attacker as already present, so every request is authenticated and authorized on its own merits rather than trusted for originating inside a perimeter. Formalized in [NIST SP 800-207](#). See Chapter 8.1.
- **DDoS (Distributed Denial of Service):** overwhelming a target, an RPC endpoint, a front end, or an API, with traffic from many sources so legitimate requests cannot get through. See Chapter 7.
- **IDS/IPS (Intrusion Detection System / Intrusion Prevention System):** network or host monitoring that detects (IDS) or detects and actively blocks (IPS) traffic matching known-malicious or anomalous patterns. See Chapter 6.4.
- **JIT (Just-in-Time) access:** granting elevated or ephemeral credentials only for a task's duration, then auto-revoking them, instead of issuing standing long-lived access. See Chapter 8.6.
- **Micro-segmentation:** subdividing a network into small, individually policed zones so lateral movement from a compromised workload must defeat a fresh set of controls at every boundary. See Chapter 8.3.
- **mTLS (mutual TLS):** a TLS handshake in which both client and server present certificates, so a service verifies its caller's identity and not only the reverse, commonly enforced by a service mesh. See Chapter 8.4.
- **Rate limiting:** capping requests a client may make in a time window, a first-line mitigation for abusive traffic and organic DDoS alike. See Chapter 7.3.
- **Service mesh:** an infrastructure layer, typically a sidecar proxy pattern, handling service-to-service encryption, identity, and policy so individual services do not each reimplement mTLS. See Chapter 8.4.
- **ZTA (Zero Trust Architecture):** an approach removing implicit trust from network location, authenticating and authorizing every request against policy, defined in [NIST SP 800-207](#) (August 2020) with implementation guidance in [NIST SP 1800-35](#) (final, June 2025). See Chapter 8 and Appendix E.1.1.
- **ZTNA (Zero Trust Network Access):** a product category replacing a traditional VPN with per-application, identity-verified access, commonly used to reach admin panels and node management interfaces without exposing them publicly. See Chapter 8.5.

18.3 DNS, Domain, and Email Security Terms

These terms describe the domain and mail layer covered in Chapters 9 through 12, a surface with an unusually direct history of Web3 fund loss.

- **BGP (Border Gateway Protocol) hijacking:** announcing false routes for a network block you do not own, redirecting traffic meant for the legitimate owner. In April 2018, attackers hijacked routes to blocks serving Amazon's Route 53 DNS service for roughly two hours, forging records for myetherwallet.com and running a phishing clone that cost users about \$152,000 in ether ([CoinDesk, April 24, 2018](#)). See Chapter 12.1.
- **CAA (Certification Authority Authorization):** a DNS record type, defined in [RFC 8659](#), restricting which certificate authorities may issue TLS certificates for a domain. See Chapter 10.2.
- **DKIM (DomainKeys Identified Mail):** a mechanism, standardized in [RFC 6376](#) (September 2011), attaching a cryptographic signature to outgoing mail headers so a recipient can verify the message and the signing domain's endorsement. See Chapter 11.
- **DMARC (Domain-based Message Authentication, Reporting, and Conformance):** a policy layer telling receiving mail servers what to do when SPF or DKIM fails and requesting failure reports, originally [RFC 7489](#) (March 2015), now superseded by the three-document specification led by [RFC 9989](#). See Chapter 11.
- **DNS hijacking:** gaining unauthorized control over a domain's resolution, typically through a compromised registrar or nameserver account, and repointing it to attacker infrastructure. On August 9, 2022, attackers compromised Curve Finance's DNS registrar and redirected curve.fi to a cloned front end harvesting token approvals, costing roughly \$575,000 while the audited smart contracts were never touched ([rekt.news](#)). See Chapter 12.1.
- **DNSSEC (Domain Name System Security Extensions):** IETF extensions, originating in [RFC 4033](#), cryptographically signing DNS zone data so a validating resolver can detect tampered or spoofed responses. See Chapter 10.1.
- **DS (Delegation Signer) record:** a parent-zone record anchoring trust to a child zone's DNSSEC signing key, completing the chain of trust from the root down to your domain. See Chapter 10.1.2.
- **Glue record:** an A or AAAA record held by the parent zone that resolves a nameserver's own hostname when that name sits inside the domain it serves, avoiding a resolution chicken-and-egg problem. See Chapter 9.2.
- **Registrar (registry) lock:** a status blocking changes to a domain's nameservers, contacts, or registrar-of-record without an out-of-band verification step, the primary defense against the account-takeover pattern behind the Curve incident. See Chapter 9.1.
- **SPF (Sender Policy Framework):** a DNS TXT record, defined in [RFC 7208](#) (April 2014), listing which mail servers are authorized to send for a domain. See Chapter 11.

- **SSHFP**: a DNS record type, defined in [RFC 4255](#), publishing an SSH host key fingerprint so a DNSSEC-validating client can verify a server's identity beyond trust-on-first-use. See Chapter 10.3.
- **Zone transfer (AXFR)**: a bulk mechanism for replicating a DNS zone from a primary to a secondary nameserver; left open to arbitrary requesters, it hands an attacker a full map of your subdomains. See Chapter 9.2.

18.4 Identity, Access, and Governance Terms

These terms describe who and what can act on infrastructure, and how that authority is granted and withdrawn, covered in Chapters 13 and 14.

- **Break-glass access**: a pre-provisioned, tightly audited emergency access path used when normal access controls are unavailable or too slow for an active incident, exercised rarely and reviewed after every use. See Chapter 13.2.
- **Hardware-backed authentication**: authentication bound to a physical security key or a device's secure enclave (a FIDO2/WebAuthn key, a platform authenticator) so the credential cannot be phished or copied off the device; NIST's [SP 800-63B](#) ranks it as the most phishing-resistant authenticator type. See Chapter 13.4.
- **MFA (Multi-Factor Authentication)**: requiring two or more independent proofs of identity before granting access. See Chapter 13.4.
- **Offboarding**: revoking every credential, key, and session tied to a departing operator or vendor, treated fully in the Employee Lifecycle Security Handbook (03) and cross-referenced here for its infrastructure-specific checklist. See Chapter 13.5 and Chapter 14.2.
- **RBAC (Role-Based Access Control)**: assigning permissions to named roles rather than to individuals directly, so access can be reasoned about, audited, and revoked at the role level. See Chapter 13.1.
- **Service account**: a non-human identity used by an application or pipeline, requiring the same least-privilege scoping and rotation discipline as a human account, with the added risk that its activity is easy to overlook in a human-focused access review. See Chapter 13.3.

18.5 Monitoring, Logging, and Incident Terms

These terms describe how infrastructure state becomes visibility and, when needed, an alert, the subject of Chapter 17.

- **Alert fatigue**: degradation of an operator's response quality from a high volume of low-value or duplicate alerts, the usual root cause behind a real incident missed inside a noisy channel. See Chapter 17.3.
- **Centralized logging**: aggregating logs from every infrastructure component into one searchable store rather than leaving them scattered across hosts, a precondition for timely correlation during an incident. See Chapter 17.4.

- **MTTD / MTTR (Mean Time to Detect / Mean Time to Respond):** metrics tracking how long a real issue takes to be noticed and resolved, used to evaluate whether monitoring and alerting are actually working. See Chapter 17.
 - **On-chain monitoring:** automated tracking of chain-level events relevant to a protocol's own infrastructure and treasury, such as large transfers or admin-key changes, feeding the same alerting pipeline as infrastructure monitoring. See Chapter 17.2.
 - **Retention policy:** a defined period logs and monitoring data are kept before deletion, balancing storage cost against the need to investigate an incident discovered weeks after it began. See Chapter 17.1.
 - **SIEM (Security Information and Event Management):** a platform ingesting logs and events from across an environment, correlating them against detection rules and surfacing alerts. See Chapter 17.4.
-

19. Appendix B: Node Infrastructure Security Checklist (Printable)

This appendix is the copy-ready form of the node guidance in Chapter 3 and the consolidated checklist in Chapter 15, organized around the lifecycle a node or validator actually goes through: decisions before it accepts traffic, discipline while it runs, and readiness built before, not during, an incident. Paste each block into a release ticket or runbook rather than reconstructing it from memory under pressure.

19.1 Pre-Deployment

Decisions made here are expensive to unwind: network exposure, environment separation, and key custody for validator infrastructure are far easier to get right before the first block is processed than to retrofit onto a running fleet. Treat this stage as a gate, not a formality, tied to the asset inventory from Chapter 2 so every new node is registered before it goes live.

Node Pre-Deployment Checklist

- ☐ Node role classified (RPC, archive, validator), provisioned from an approved, version-controlled base image (Chapter 3.1, 5.1)
- ☐ Network exposure scoped to minimum: public RPC behind a reverse proxy and rate limiter, no admin/debug ports internet-reachable (Chapter 3.1.1, 6.2)
- ☐ Dev, staging, and production nodes separated, no shared credentials (Chapter 3.3)
- ☐ Validator signing keys generated and stored in a remote signer or HSM-backed secrets manager, never plaintext on the node host (Chapter 3.2, 4.5)
- ☐ OS hardening baseline applied (minimal services, no default credentials, current patches) before the node joins any network (Chapter 5.1, 5.3)
- ☐ Monitoring hooks (peer count, sync status, disk/memory headroom) wired up before go-live, not after the first incident (Chapter 3.1.3, 17)
- ☐ Node added to the asset inventory with owner, purpose, and sensitivity recorded (Chapter 2.1, 2.2)

19.2 Ongoing Operations

A node that passed pre-deployment review can still drift into an unsafe state through routine operation: unpatched updates, expired monitoring, a slashing-relevant misconfiguration. Run this block on a fixed cadence regardless of whether anything appears wrong, because the failures it catches do not announce themselves.

Node Ongoing Operations Checklist

- ☐ Client patched on the cadence set by Chapter 5.2, security releases applied out-of-band, not on the next scheduled window
- ☐ Client diversity maintained across the validator fleet where the network supports it, so one client bug cannot correlate a slashing event (Chapter 3.2)
- ☐ Peer count, sync status, and slashing-protection DB health actively monitored, thresholds tuned against both silence and alert fatigue (Chapter 3.1.3, 17.3)
- ☐ Access reviews confirm only currently authorized operators and service accounts retain access; offboarded access already revoked (Chapter 13.5, 14.2)
- ☐ Backups and validator key recovery material tested by an actual restore, not merely confirmed to exist (Chapter 4.4)
- ☐ Configuration checked against the approved baseline for drift; unexplained deviations investigated before being reconciled away (Chapter 4.7.3)
- ☐ DDoS protection and rate limits on public RPC endpoints reviewed against current traffic patterns (Chapter 7.2, 7.3)

19.3 Incident Readiness

Readiness work has no value if invented during the incident it is meant to support. This block operationalizes Chapter 15.3; exercise it through a tabletop or live drill on a schedule set by the Incident Response Handbook's (06) process, not for the first time while a validator is being slashed.

Node Incident Readiness Checklist

- ☐ Documented runbook exists for the most likely incidents: RPC endpoint compromise, validator downtime or double-signing risk, loss of quorum on a self-hosted fleet
 - ☐ Redundant node providers or a self-hosted fallback pre-configured, so a single provider outage does not remove chain access (Chapter 1.2)
 - ☐ Key rotation/revocation procedures for validator signing keys and RPC access tokens documented and exercised at least once outside a real incident
 - ☐ Communication plan names who is notified, in what channel, within what time window, once a node incident is confirmed (Chapter 17.5)
 - ☐ Forensic snapshot capture (disk image, logs, memory where feasible) scripted in advance so evidence is not lost restoring service
 - ☐ Post-incident review required, feeding back into the pre-deployment and ongoing-operations checklists above
-

20. Appendix C: DNS and Domain Security Checklist

Domain and DNS compromise is one of the few infrastructure failures in this handbook with a direct, repeated history of turning into user fund loss without a single line of smart contract code touched. This appendix condenses Chapters 9 through 12 into registrar controls, DNS and certificate controls, email authentication, and a detection and response trigger list you can act on immediately.

20.1 Registrar and Domain Controls

The registrar account is the root of trust for everything downstream: whoever controls it can reappoint nameservers, and DNSSEC, CAA, and every other control in this appendix become moot the moment that account is taken over. This is exactly the path used against Curve Finance's registrar in August 2022 (rekt.news).

Registrar and Domain Checklist

- ☐ Registrar (and, where offered, registry) lock enabled on every production domain (Chapter 9.1)
- ☐ Registrar account access requires hardware-backed MFA for every operator, no shared or generic login (Chapter 9.1, 13.4)
- ☐ Authorized contacts and recovery methods current, reviewed on the same cadence as privileged access reviews (Chapter 13.2)
- ☐ Domain auto-renewal enabled, payment method monitored, so a lapsed domain cannot be re-registered by an attacker
- ☐ At least two independently operated, geographically diverse nameservers configured, with a documented failover path (Chapter 9.3)
- ☐ Nameserver glue records verified correct at the parent zone (Chapter 9.2)

20.2 DNS Resolution and Certificate Controls

Once the registrar account itself is locked down, the next layer is making DNS answers themselves tamper-evident and constraining who can mint a certificate for your name.

DNS and Certificate Checklist

- ☐ DNSSEC enabled on every production zone, DS record published at the registrar so the chain of trust reaches the root (Chapter 10.1, 10.1.2)
- ☐ DNSSEC signing keys follow a documented rotation schedule, not shared across unrelated zones (Chapter 10.1.1)
- ☐ CAA records restrict certificate issuance to a named, minimal set of CAs (Chapter 10.2)
- ☐ SSHFP records published for any SSH-reachable host where DNSSEC-validating clients can use them (Chapter 10.3)
- ☐ Zone transfer (AXFR) restricted to authorized secondaries only, not open to arbitrary requesters (Chapter 9.2)
- ☐ Automated monitoring alerts on unauthorized nameserver, DNSSEC, or CAA changes, independent of the registrar's own change log (Chapter 12.2)

20.3 Email Authentication Controls

Email is the notification and account-recovery channel for almost every other control in this handbook, which makes it a target in its own right (Chapter 11.3) and not a secondary concern.

Email Authentication Checklist

- ☐ SPF record published with a hard-fail (`-all`), not soft-fail, for every mail-sending domain (Chapter 11.1)
- ☐ DKIM signing enabled for all outbound mail, signing keys rotated on a defined schedule (Chapter 11.1)
- ☐ DMARC deployed at enforcement (`p=reject` , or `p=quarantine` as a documented interim step), not left at `p=none` (Chapter 11.1, 11.2)
- ☐ DMARC aggregate and failure reports routed to a monitored mailbox, reviewed on a fixed cadence (Chapter 11.2)
- ☐ Every domain able to send mail, including dormant ones, has at least an SPF record and a restrictive DMARC policy against spoofing

20.4 Detection and Response Trigger List

Pair the checklist above with a short, memorized set of triggers: what you would see first, and what the first action is, for each of the common DNS and domain attack vectors named in Chapter 12.1.

Attack Vector	First Signal	Immediate First Action
Registrar account takeover	Login/password-reset alert from an unrecognized location; nameservers changed without a matching ticket	Freeze the account via an out-of-band registrar support channel; verify lock status
Nameserver hijack	DNS monitoring alert on a nameserver or record change; user reports of a suspicious front end	Compare current records against the last known-good snapshot; start the recovery sequence in Figure 2
Zone or cache poisoning	Unexpected resolution for a subset of resolvers; SRI or certificate mismatch reports (CDN and Front-End Supply Chain Handbook, 01)	Purge affected resolver and CDN caches; confirm authoritative records at the source
Subdomain takeover	Dangling CNAME pointing to a deprovisioned cloud resource, flagged by asset inventory review (Chapter 2.3)	Reclaim or remove the cloud resource; remove the dangling record until reclaimed
BGP route hijack	Routing anomaly from a BGP monitoring service; sudden resolver behavior change tied to one network	Notify upstream transit providers and the registrar's DNS operator; largely outside self-remediation

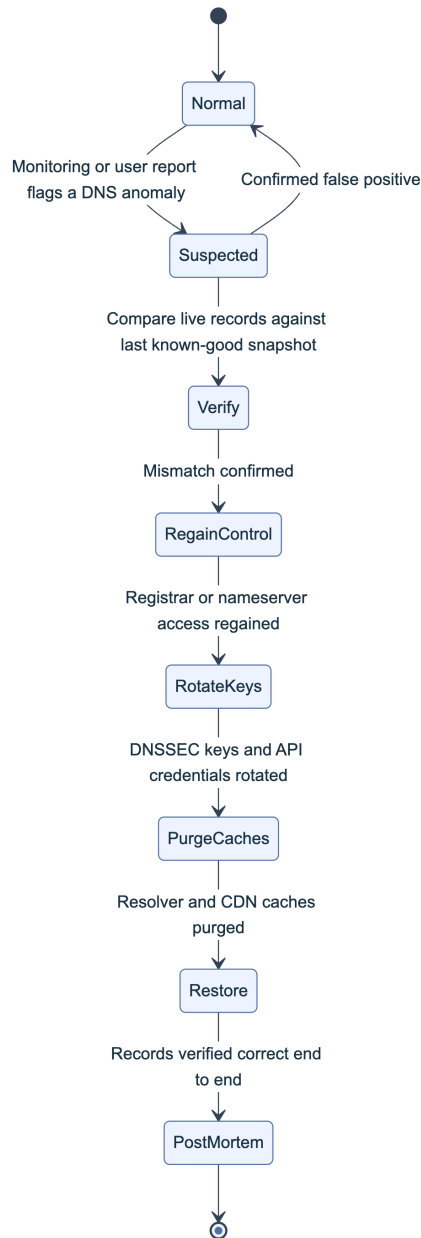


Figure 2. DNS hijack detection and recovery sequence. Verify can only be entered from a confirmed anomaly, and every state after RegainControl is forward-only: rotating keys and purging caches before declaring recovery prevents the attacker from re-entering through a credential that was compromised but never rotated.

21. Appendix D: Tool and Provider References

The tables below name representative tools and providers by category, not an endorsement of any single vendor. Where a category has both an open-source and a commercial-managed option, both are listed, since that choice is itself the single-versus-multi-provider tradeoff from Chapter 1.2. Verify current licensing before adopting any tool: several projects in this space, most visibly HashiCorp Vault's 2023 move away from an OSI-approved license, have changed terms in ways that matter for a security-critical dependency.

21.1 Node, Cloud, and Secrets Tooling

Category	Representative Tools / Providers	Purpose
Node clients	Geth, Erigon, Nethermind, Besu, Reth (execution); Lighthouse, Prysm, Teku, Nimbus (consensus)	Self-hosted node software; diversity across this list reduces correlated-bug risk (Chapter 3.2)
Managed RPC / node infrastructure	Infura, Alchemy, QuickNode, Ankr, Chainstack	Hosted RPC and archive node access without operating the infrastructure
Cloud compute	AWS, Google Cloud, Microsoft Azure	Primary compute and storage providers; each publishes its own shared-responsibility documentation (Chapter 4.1)
Secrets management	OpenBao (Linux Foundation, OSI-licensed Vault fork), HashiCorp Vault, AWS Secrets Manager, Azure Key Vault, Google Secret Manager	Storage, versioning, and rotation of credentials and signing material outside application config (Chapter 4.5)

Category	Representative Tools / Providers	Purpose
Infrastructure as Code	Terraform, OpenTofu, Pulumi, AWS CloudFormation	Declarative, versioned infrastructure provisioning (Chapter 4.7.1, 16)
Policy as code / IaC scanning	Open Policy Agent (Rego), Checkov , tfsec, Trivy	Pre-merge policy enforcement and misconfiguration scanning (Chapter 4.7.2)
CSPM (cloud posture)	Prowler , native provider posture tools (AWS Security Hub, Microsoft Defender for Cloud)	Continuous scanning of live cloud configuration against a policy baseline (Chapter 4.7)

21.2 Network, DNS, and Monitoring Tooling

Category	Representative Tools / Providers	Purpose
DDoS mitigation	Cloudflare, AWS Shield, Akamai, Fastly, Google Cloud Armor	Absorbing and filtering volumetric and application-layer DDoS traffic (Chapter 7.2)
Zero-trust network access	Cloudflare Access, Tailscale, Teleport, Google BeyondCorp Enterprise	Identity-verified, per-application access replacing a flat VPN (Chapter 8.5)
DNS providers and registrars	Cloudflare Registrar, Amazon Route 53, NS1	Authoritative DNS hosting and domain registration; evaluate registrar lock and MFA support first (Chapter 9.1)

Category	Representative Tools / Providers	Purpose
DNSSEC and DNS validation	DNSViz, dig +dnssec , registrar-native DNSSEC status pages	Verifying DS record publication and chain-of-trust validity (Chapter 10.1.2)
Monitoring and metrics	Prometheus with Grafana, Datadog	Node, host, and application-level metrics and dashboards (Chapter 17.4)
Centralized logging / SIEM	ELK/OpenSearch stack, Splunk	Log aggregation, correlation, and alert generation (Chapter 17.4)
Alerting and on-call	PagerDuty, Opsgenie	Multi-channel alert routing and escalation feeding incident response (Chapter 17.3, 17.5)

22. Appendix E: References and Further Reading

22.1 Key Standards and Frameworks

Two reference points recur across Parts II, III, and VI of this handbook closely enough to warrant a dedicated summary here rather than sending the reader back to the body text each time.

22.1.1 NIST Zero Trust Architecture (SP 800-207, SP 1800-35)

[NIST SP 800-207](#) (August 2020, Rose, Borchert, Mitchell, and Connelly) is the definitional document for zero trust architecture (ZTA): “an evolving set of cybersecurity paradigms that move defenses from static, network-based perimeters to focus on users, assets, and resources.” It rejects implicit trust based on network location or device ownership and requires every request authenticated and authorized against policy, the assume-breach posture running through Chapter 8. It also defines the logical components (policy engine, policy administrator, policy enforcement point) that ZTNA and service mesh products later implement in concrete form.

[NIST SP 1800-35](#), finalized in June 2025 by NIST’s National Cybersecurity Center of Excellence (NCCoE), is the companion practice guide: 19 example ZTA implementations built with 24 commercial technology partners, covering identity governance, micro-segmentation, and secure access service edge approaches, mapped against commonly used security standards. Read SP 800-207 once for the model in Chapter 8.1 through 8.3; use SP 1800-35 when a specific implementation decision needs a documented precedent.

22.1.2 Cloud Security Posture and Policy-as-Code (OPA, Checkov, Prowler)

Configuration and posture management (Chapter 4.7) rests on three tools named repeatedly enough to justify a shared reference. [Open Policy Agent \(OPA\)](#), a Cloud Native Computing Foundation graduated project since February 2021, decouples policy decisions from the applications that enforce them: a Kubernetes admission controller, a CI pipeline, and an API gateway can all query the same Rego-defined policy instead of each reimplementing its own rule logic. [Checkov](#), built by Bridgecrew and now maintained under Palo Alto Networks’ Prisma Cloud, scans Terraform, CloudFormation, Kubernetes manifests, ARM templates, Helm, and AWS CDK for misconfigurations before deployment, the pre-merge scanning step in Chapter 4.7.2. [Prowler](#), an open-source CSPM platform with over 14,000 GitHub stars and 300-plus contributors, runs continuous checks against live AWS, Azure, Google Cloud, Kubernetes, and several other environments, the runtime complement to Checkov’s pre-deployment scan.

Together these three cover the full posture lifecycle named in Chapter 4.7: OPA enforces policy at admission and in CI, Checkov catches misconfiguration in the IaC source before it is applied, and Prowler catches whatever reaches production anyway, including drift introduced outside of IaC entirely (Chapter 4.7.3).

Every other claim in this handbook that names a standard, a version, a date, or an incident is cited inline at the point it is made. Rather than duplicate that source list here, this appendix points to [references.md](#), the deduplicated bibliography for the full handbook, grouped into Standards and Frameworks (NIST SP 800-207 and SP 1800-35, the DNS and email RFCs behind DNSSEC, CAA, SPF, DKIM, and DMARC, and the OWASP SCSVS, SCSTG, and SCWE), Incidents and Case Studies (the Curve Finance DNS hijack and the MyEtherWallet BGP hijack among them), Tools (OPA, Checkov, Prowler, and the node, cloud, and secrets tooling named in Appendix D), and Further Reading. If a link anywhere in this handbook goes stale, check [references.md](#) for an updated URL or an archive copy before assuming the underlying claim no longer holds.

Key controls for Part 7

- Treat Appendix A as the canonical vocabulary for this handbook; link to it rather than redefining a term inline in a ticket or post-mortem.

- Run the node pre-deployment checklist (19.1) as a gate before any new RPC, archive, or validator node accepts traffic.
- Run the ongoing operations (19.2) and incident readiness (19.3) checklists on a fixed cadence, not only in response to a suspected problem.
- Run the registrar, DNS, and email checklists (20.1 through 20.3) before every domain or DNS configuration change, and keep the detection trigger list (20.4) memorized, not merely bookmarked.
- Confirm current licensing before adopting any tool from Appendix D; license terms in this space have changed before and will again.
- Add every new citation to `references.md` in the same change that introduces it (Appendix E.1).

Appendix C: References and Further Reading

The sources below are cited across the seven parts of this handbook, grouped by type for easier navigation.

Standards and Frameworks

- OWASP Smart Contract Security Testing Guide (SCSTG) / Smart Contract Weakness Enumeration (SCWE)
- OWASP Smart Contract Security Verification Standard (SCSVS)
- OWASP Access Control Cheat Sheet
- OWASP Authentication Cheat Sheet
- OWASP API Security Top 10, API4:2023 Unrestricted Resource Consumption
- OWASP Top 10 CI/CD Security Risks
- NIST CSF 2.0, ID.AM (Asset Management)
- NIST SP 800-53 Rev. 5, AC-6 (Least Privilege)
- NIST SP 800-53 Rev. 5, PS-4 (Personnel Termination)
- NIST SP 800-63B, Digital Identity Guidelines (Rev. 3)
- NIST SP 800-63B, Digital Identity Guidelines (Rev. 4)
- NIST SP 800-207, Zero Trust Architecture
- NIST SP 800-207, Zero Trust Architecture (PDF)
- NIST SP 800-162, Guide to Attribute Based Access Control (ABAC)
- NIST SP 1800-35, Implementing a Zero Trust Architecture
- NIST SP 800-41 Rev. 1, Guidelines on Firewalls and Firewall Policy
- NIST SP 800-52 Rev. 2, Guidelines for TLS Implementations
- NIST SP 800-81 Rev. 3, Secure Domain Name System Deployment Guide
- NIST SP 800-92, Guide to Computer Security Log Management
- CISA Zero Trust Maturity Model v2.0
- CISA Known Exploited Vulnerabilities (KEV) Catalog
- CISA Binding Operational Directive 18-01, Enhance Email and Web Security
- CISA Binding Operational Directive 22-01, Reducing the Risk of Known Exploited Vulnerabilities
- CISA, Insider Threat Mitigation
- CIS Benchmarks
- CIS Controls
- CIS Control 1, Inventory and Control of Enterprise Assets
- CIS Control 4, Secure Configuration of Enterprise Assets and Software

- CIS Control 8, Audit Log Management
- MITRE ATT&CK, Network Denial of Service (T1498)
- MITRE ATT&CK, Detection Strategy for T1498
- MITRE AADAPT Cyber Threat Framework for Digital Assets
- FIDO Alliance, FIDO2
- W3C Web Authentication (WebAuthn) Level 3
- AWS Shared Responsibility Model
- OMB Memorandum M-21-31, Improving the Federal Government's Investigative and Remediation Capabilities
- RFC 4033, DNS Security Introduction and Requirements
- RFC 4033, DNS Security Introduction and Requirements
- RFC 4034, Resource Records for the DNS Security Extensions
- RFC 4035, Protocol Modifications for the DNS Security Extensions
- RFC 4255, Using DNS to Securely Publish SSH Key Fingerprints
- RFC 4255, Using DNS to Securely Publish SSH Key Fingerprints
- RFC 4509, Use of SHA-256 in DNSSEC Delegation Signer (DS) Resource Records
- RFC 6376, DomainKeys Identified Mail (DKIM) Signatures
- RFC 6376, DomainKeys Identified Mail (DKIM) Signatures
- RFC 6594, Use of SHA-256/ECDSA in SSHFP Resource Records
- RFC 6698, DNS-Based Authentication of Named Entities (DANE) TLSA
- RFC 6781, DNSSEC Operational Practices, Version 2
- RFC 7208, Sender Policy Framework (SPF)
- RFC 7208, Sender Policy Framework (SPF)
- RFC 7479, Using Ed25519 in SSHFP Resource Records
- RFC 7489, Domain-based Message Authentication, Reporting and Conformance (DMARC)
- RFC 7489, Domain-based Message Authentication, Reporting and Conformance (DMARC)
- RFC 8446, The Transport Layer Security (TLS) Protocol Version 1.3
- RFC 8624, Algorithm Implementation Requirements for DNSSEC
- RFC 8659, DNS Certification Authority Authorization (CAA) Resource Record
- RFC 8659, DNS Certification Authority Authorization (CAA) Resource Record
- RFC 8945, Secret Key Transaction Authentication for DNS (TSIG)
- RFC 9162, Certificate Transparency Version 2.0
- RFC 9989, DMARC (updated multi-document specification)

Incidents and Case Studies

- Codecov Bash Uploader compromise (January 31 to April 1, 2021)
- AppSecco, SSRF, privileged AWS keys, and the Capital One breach
- Cloudflare, defending the internet: blocking a monumental 7.3 Tbps DDoS

- CoinMarketCap Academy, Infura outage and decentralization concerns
- Cointelegraph, Manta Network DDoS attack amid exchange listing
- Cybernews, NCX exchange data leak exposes wallets
- Cybernews, Squarespace DNS hijack targets crypto domains
- Decrypt, DNS attacks driving wallet-drainer redirects on DeFi protocols
- Decrypt, Curve Finance DNS record attack
- Decrypt, MetaMask and Ethereum apps down in Infura outage
- Electrek, Tesla cloud hijacked by cryptomining hackers
- Wikipedia, Knight Capital Group trading-code deployment failure
- GitHub Security Advisory GHSA-v9jh-j8px-98vq (CVE-2023-42319, Geth GraphQL DoS)
- Krebs on Security, GoDaddy employees used in attacks on cryptocurrency services
- Offchain Labs, post-mortem report on the May 2023 Ethereum mainnet finality incident
- Palo Alto Networks Unit 42, “Leaky Vessels” container escape vulnerabilities
- rekt.news, Curve Finance
- Okta Security, root cause analysis of the October 2023 support system breach
- TechCrunch, Vercel confirms security incident after breach at Context AI
- Vercel incident bulletin, April 2026 security incident
- BleepingComputer, DNS hijacks target crypto platforms registered with Squarespace
- BleepingComputer, hackers stole over \$20 million from misconfigured Ethereum clients (June 2018)
- CCN, AWS outage takes down Coinbase, Robinhood, Venmo and other platforms
- CCN, Solana survives 6 Tbps DDoS attack amid Sui downtime
- CertiK, BGP hijacking and the \$1.9M KLAYswap attack
- CNBC, Coinbase says hackers bribed staff to steal customer data
- Coinbase, protecting our customers and standing up to extortionists
- CoinDesk, BadgerDAO reveals details of how it was hacked for \$120M
- CoinDesk, \$150K stolen from MyEtherWallet users in DNS server hijacking (April 2018)
- CoinDesk, Compound Finance site compromised in phishing attack
- CoinDesk, AWS outage halts some crypto apps
- Dark Reading, Uber breach via external contractor MFA bombing attack
- Fastly, summary of the June 8 outage
- Halborn, the BadgerDAO hack explained (December 2021)
- The Block, Curve Finance’s front end targeted in DNS attack
- The Block, Ethereum infrastructure provider Infura is down
- ThousandEyes, Amazon Route 53 DNS and BGP hijack analysis
- ThousandEyes, inside the Fastly outage and lessons learned
- Uber, 2016 data incident disclosure (newsroom)

Tools and Documentation

- [Cloudflare, how DNSSEC works](#)
- [GitHub Docs, security hardening your deployments with OpenID Connect \(OIDC\)](#)
- [Open Policy Agent \(OPA\)](#)
- [Open Policy Agent \(OPA\) documentation and Rego](#)
- [Red Hat Enterprise Linux 9, auditing the system](#)
- [tfsec check library documentation](#)
- [Chainstack, RPC security and access rules](#)
- [HashiCorp Cloud Platform, Vault Secrets auto-rotation](#)
- [AWS IAM, security best practices](#)
- [OpenZeppelin Defender documentation](#)
- [Web3Signer, slashing protection](#)
- [ethereum.org, client diversity](#)
- [Geth \(go-ethereum\), security documentation](#)
- [OpenZeppelin Monitor \(GitHub\)](#)
- [OpenZeppelin Relayer \(GitHub\)](#)
- [Gitleaks \(GitHub\)](#)
- [Cartography \(GitHub\)](#)
- [Prowler \(GitHub\)](#)
- [driftctl \(GitHub\)](#)
- [Steampipe AWS plugin \(GitHub\)](#)
- [ICANNWiki, Domain Locking](#)
- [NHI Management Group, break-glass account glossary](#)
- [OpenBao](#)
- [Prowler](#)
- [SPIFFE](#)
- [SPIRE, about the project](#)
- [Suricata](#)
- [Tailscale](#)
- [Trivy, IaC scanning coverage](#)
- [TruffleHog](#)
- [Ubuntu, hardening automation for CIS benchmarks on Ubuntu 24.04 LTS](#)
- [Checkov](#)
- [Conftest](#)
- [HashiCorp Sentinel](#)
- [Pulumi CrossGuard documentation](#)
- [QuickNode, Ethereum endpoint security](#)
- [Red Hat, what is a configuration management database \(CMDB\)](#)

- [Tenderly, Alerting and Monitor products](#)
- [WireGuard](#)
- [Yubico, phishing-resistant MFA glossary](#)
- [Zeek](#)
- [Forta](#)
- [CyberArk, what is just-in-time access](#)

Further Reading

- [Cloudflare, Q3 2025 DDoS threat report](#)
- [Cloudflare, Q4 2025 DDoS threat report](#)
- [CryptoSlate, French regulator calls out DeFi centralization as cloud providers fall](#)
- [DNSChkr, DNSSEC adoption 2026](#)
- [PowerDMARC, Google and Yahoo bulk sender email authentication requirements](#)
- [Mailgun, state of email deliverability: the “Yahooogleg” bulk sender rules](#)
- [7BlockLabs, Ethereum/Erigon/Geth archive and full node disk size comparison \(2026\)](#)
- [BeyondTrust, guide to just-in-time privileged access management](#)
- [CyberArk, 2025 State of Machine Identity Security Report](#)
- [Figment, distributed validator technology and infrastructure resilience](#)
- [HashiCorp, rotated vs. dynamic secrets: which should you use](#)
- [Latacora, OIDC workload identity on AWS](#)
- [technologychecker.io, DNSSEC adoption](#)
- [AWS Security Blog, defense in depth: open firewalls, reverse proxies, SSRF, and the EC2 instance metadata service](#)